

Ingeniería de Software I

Tema1: Sistemas de Información / Gestión de Proyectos de Software

- **Introducción:** Mucho de los problemas del desarrollo del software son el resultado de una gestión débil; la gestión de proyectos implica la planificación, supervisión y control del personal, del proceso y los eventos que ocurren mientras evoluciona el software desde la fase preliminar a la implementación operacional.
El producto que se obtiene en la Gestión es un plan donde se define el proceso, las tareas a realizar, los mecanismos de seguridad y finalmente controlar el cambio y evaluar la calidad.
- **Componentes de la Gestión:** Los componentes de la gestión de proyectos son las cuatro **Ps.: Personal, Producto, Proceso y Proyecto.**
 - **Personal:** La gestión del personal está compuesta de varias áreas para garantizar la elección adecuada: Reclutamiento, selección, gestión de rendimiento, entrenamiento, etc. Áreas denominadas *“Modelo de Madurez de Gestión de Personal”*
 - **Producto:** Antes de planificar se deben realizar las siguientes tareas: Objetivos y ámbito del producto, soluciones alternativas, identificar soluciones técnicas y de gestión.
 - **Proceso:** Se establece la estructura desde donde detallar el plan de desarrollo, tareas, hitos, productos, etc. Además, en las fases genéricas se selecciona el modelo de proceso. *“Modelo CMM -Capability Maturity Model- para la Madurez de la Empresa en la utilización de un proceso.”*
 - **Proyecto:** En 1998 los datos de la industria del software indicaron que el 26% de los proyectos fallaron totalmente, del resto el 46% tuvieron desbordamientos en la planificación y el coste. Para evitar el fracaso se debe aplicar el sentido común en la planificación, supervisión y control del proyecto.
- **Participantes y Perfiles:** Atendiendo a las características de complejidad y magnitud del sistema que se está desarrollando, los perfiles de los participantes que intervienen deberán adaptarse y dimensionarse en cada momento. Los perfiles establecidos son: **Directivo, Jefe de Proyectos, Consultor, Analista y Programador.**
 - **Directivo**
 - **Perfil del Directivo:**
 - Alto Nivel en la dirección de la organización.
 - Conocimientos de los objetivos estratégicos y de negocio que se persiguen.
 - Autoridad para validar y aprobar cada uno de los procesos realizados durante el desarrollo del sistema.
 - Conocimiento del entorno y de la organización suficiente para proporcionar, a lo largo de todo el proyecto, unos requisitos del sistema adecuados, completos y suficientemente importantes como para considerarse en el catálogo definitivo de requisitos.
 - **Participantes del Directivo:**
 - **Comité de Dirección:** Provee los recursos necesarios para el objetivo propuesto; revisa y aprueba cada uno de los procesos.
 - **Comité de Seguimiento:** Resuelve las contingencias, asegura la disponibilidad de recursos humanos y participa colaborando en las actividades que sean necesarias.
 - **Directores de Usuarios:** Aportan información sobre las necesidades planteadas, validan los resultados; comprensión e incorporación de todos los requisitos con las prioridades adecuadas.
 - **Usuarios Expertos:** misma responsabilidad que los directores de usuarios pero con mayor nivel de detalles.
 - **Jefe de Proyecto:** Los participantes ejercen labores de coordinación y dirección de equipos humanos especializados en actividades propias. Los siguientes se corresponden con los participantes.
 - **Jefe de Proyecto:** realiza las siguientes tareas:

- Estima el esfuerzo para desarrollar el proyecto.
 - Determina la estructura de la estrategia.
 - Establece la planificación del proyecto.
 - Realiza las labores de seguimiento y control del proyecto.
 - Realiza la evaluación de resultados.
 - Coordina el equipo de proyecto.
 - Gestiona y resuelve las incidencias.
 - Realiza la actualización de la planificación inicial.
 - Realiza los informes de seguimiento.
 - Archiva la documentación de la gestión.
- **Responsable de Implementación**
 - Operación, Sistemas y Mantenimiento, participan en los procesos principales del desarrollo, ofreciendo su apoyo al Jefe de Proyecto.
 - Poseen mayor conocimiento de los aspectos organizativos y de procedimientos habituales en la organización en sus áreas de responsabilidades concretas.
 - Aseguran la disponibilidad de los recursos necesarios y la participación activa del equipo humano que coordinan.
- **Responsable de Seguridad y Calidad**
 - Aportan información relativa a las normas y procedimientos habituales en la organización.
 - Ofrecen asesoramiento sobre todos los aspectos de Seguridad y Calidad.
 - Coordinan a los integrantes de los equipos de Seguridad y Calidad.
- **Consultor:**
 - ***Perfil del Consultor:***
 - Asesora en los aspectos relativo al negocio (Consultor)
 - Asesora en temas informáticos (Consultor Informático)
 - Consultoría Informática: Tecnologías de la información y Sistemas de Información.
 - Conocimiento de las últimas tecnologías
 - Participa en la evaluación de las alternativas tecnológicas
 - Participa en la validación y selección de soluciones
 - Opinión experta, pericia o conocimiento relativo a los requisitos del negocio, técnico y de usuario.
 - ***Participantes del Consultor:***
 - **Técnicos y especialistas:** Los técnicos y especialistas en sistemas y comunicaciones cuentan con una visión más precisa de la tecnología existente en el momento, aportan también su experiencia práctica a la hora de valorar las alternativas tecnológicas, participan activamente durante la implementación. Y puesta en producción.
- **Analista:**
 - ***Perfil del Analista:*** Son los encargados de elaborar un catálogo detallado de requisitos que describen de forma precisa el sistema, este catálogo se obtiene mediante entrevistas y sesiones de trabajos, partir del catálogo de requisitos se elaboran los distintos modelos que sirven de base para el diseño. Ej.: Modelo de Datos, Modelo de Clases, Modelo de Comportamiento, Modelo de Funciones, etc.
 - ***Participantes del Analista:***
 - **Administrador de Base de Datos:** definen la estructura física de datos que utilizará el sistema a partir del modelo lógico de datos normalizado o del modelo de clases, definen la migración de datos, estiman los volúmenes de las estructuras de datos implicadas, definen la carga inicial de datos y gestionan el sistema en producción.

- **Equipo de Proyecto:** Participa a lo largo de todo el proceso de desarrollo y mantenimiento del sistema, se puede diferenciar los siguientes equipos: **Implementación, Operación, Mantenimiento, Arquitectura, Soporte Técnico y Seguridad.**
- **Equipo de Formación:** es el encargado de preparar e impartir la formación al equipo responsable de la implantación y operación del sistema, para ello elabora un plan de formación que incluye: **Cursos de formación y contenidos, Recursos humanos, Infraestructura, Contenido de formación que deberá recibir el usuario y Seguimiento de dicha formación.**
- **Grupo Aseguramiento de Calidad:** Desarrolla el plan de aseguramiento de la calidad reflejando las actividades de calidad a realizar, comprueba que se lleven a cabo las medidas de seguridad establecidas, es un grupo completamente independiente del equipo del proyecto.

- **Programador:**

- **Perfil del Programador:** Son miembros del equipo de proyecto, y su función es la de construir el código del producto en base al diseño técnico realizado por el analista-diseñador, es el encargado de realizar las pruebas unitarias del código.

➤ **Paradigmas de Organización:** La mejor estructura de equipo depende del estilo de gestión de una organización. Existen diferentes organigramas de equipo según diferentes autores, aunque cualquiera que sea la organización del equipo, el objetivo para todos los gestores de proyecto es colaborar para crear un equipo que presente cohesión.

- **Paradigma de Mantei** (the effect of programming team structures in programming tasks)

- **Descentralizado Democrático:** El equipo de ingeniería no tiene un jefe permanente. Se nombran coordinadores de tareas a corto plazo. La comunicación del equipo es horizontal. Estos equipos producen más y mejores soluciones. Es buena solución para problemas difíciles. Necesitan mucha comunicación. Al ser un equipo descentralizado necesita más tiempo para llevar a cabo una tarea.
- **Descentralizado Controlado:** El equipo de ingeniería tiene un jefe definido que coordina tareas específicas y jefes secundarios que tienen responsabilidades sobre sub-tareas. La resolución de problemas sigue siendo una actividad del grupo, pero la implementación de soluciones se reparte entre los subgrupos por el jefe de equipo. La comunicación entre subgrupos e individuos es horizontal. También hay comunicación vertical a lo largo de la jerarquía de control. Buenos para: problemas complejos, problemas grandes, cuando hay una modularidad alta.
- **Centralizado Controlado:** El jefe del equipo se encarga de la resolución de problemas a alto nivel y la coordinación interna del equipo. La comunicación entre el jefe y los miembros del equipo es vertical. Bueno para proyectos grandes. Bueno cuando hay alta modularidad. Buenos cuando producen software similar. Son menos innovadores.

- **Paradigma de Constantine** (work organization paradigms for project management and organization)

- **Cerrado (C):** Paradigma con jerarquía tradicional, buenos para proyectos similares, poca innovación.
- **Aleatorio (AL):** Estructura al equipo libremente y depende de la iniciativa individual de los miembros del equipo. Buenos cuando se requiere innovación y avances tecnológicos. No son buenos cuando se quiere un rendimiento ordenado.
- **Abierto (A):** Asigna unos controles al paradigma Aleatorio. El trabajo se desarrolla en colaboración, con mucha comunicación y toma de decisiones consensuadas. Buenos para problemas complejos. No son buenos cuando se requiere rendimiento.
- **Sincronizado (S):** Se basa en la compartimentación natural de un problema y organiza a los miembros del equipo para trabajar en partes del problema con poca comunicación activa entre ellos.

- **Coordinación y Comunicación:** La Escala, incertidumbre y la interoperabilidad son alguno de los motivos por los cuales los proyectos pueden tener problemas, los equipos deberán disponer de métodos efectivos para coordinar a las personas por medio de mecanismos de comunicaciones formales e informales.
 - **Formal – Enfoque Impersonal:** Incluye documentos de Ingeniería de Software y entregas, memorandos técnicos, hitos de proyecto, planificaciones del programa y herramientas de control.
 - **Formal – Procedimientos Interpersonales:** Se centra en las actividades de garantía de calidad aplicada a los productos de Ingeniería de Software.
 - **Informal – Procedimientos Interpersonales:** Reuniones de grupo para la divulgación de información y resolución de problemas
 - **Comunicación Electrónica:** Incluye correo electrónico, boletines de noticias electrónicos y videoconferencias.
 - **Red Interpersonal:** Discusiones informales entre los miembros del equipo y con personas que pueden aportar al proyecto aunque no pertenezcan al mismo.
- **Reglas para Conseguir un Equipo:** Los miembros deben confiar unos de otros, las habilidades deben adecuarse al problema, los inconformistas deben ser excluidos del grupo. El grupo debe estar fuertemente cohesionado de tal forma que el todo sea mayor que la suma de las partes.
- **Factores Tóxicos para el Equipo:** Atmosfera de trabajo frenética, alta frustración causada por factores tecnológicos, del negocio, o personales que provocan fricción entre los miembros del equipo, procedimientos mal coordinados, definiciones de roles confusos generando fallas en las responsabilidades, continua y repetida exposición al fallo, que conduce a la pérdida de la confianza y a una caída de la moral.
- **Proceso de Software y Métricas de Proyectos:** Las métricas del software se refieren a un amplio elenco de mediciones para el software. La medición se puede aplicar al proceso del software con el intento de mejorarlo sobre una base continua. Se puede utilizar en el proyecto del software para ayudar en la estimación, el control de calidad, la evaluación de productividad y el control de proyectos. Las mediciones ayudan en la toma de decisiones tácticas a medida que el proyecto evoluciona.
- **Medidas, métricas e indicadores:** Dentro del contexto de la ingeniería del software, una medida proporciona una indicación cuantitativa de la extensión, cantidad, dimensiones, capacidad o tamaño de algunos atributos de un proceso o producto.
 - El IEEE Standard Glossary of Software Engineering Terms [IEEE93] Define “Métrica: una medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo dado”
 - Un indicador es una métrica o una combinación de métricas que proporcionan una visión profunda del proceso del software, del proyecto de software o del producto en sí.
- **Métricas en el proceso y el dominio del proyecto**
 - **Indicadores de Proceso:** permiten a una organización de ingeniería del software tener una visión profunda de la eficacia de un proceso ya existente. También permiten que los gestores evalúen lo que funciona y lo que no. Intentan proporcionar valores que lleven a mejoras de los procesos de software a largo plazo.
 - **Indicadores de Proyecto:** permiten al gestor de proyectos de software:
 - Evaluar el estado del proyecto en curso;
 - Seguir la pista de los riesgos potenciales;
 - Detectar las áreas problemáticas antes de que sean críticas;
 - Ajustar el flujo y las tareas del trabajo;
 - Evaluar la habilidad del equipo de proyecto en cuanto a calidad de los productos de software.
 - **Mejora Estadística de Proceso del Software (MEPS):**
 - Todos los errores y defectos se caracterizan por origen.
 - Se registra el coste de corregir errores y defectos.
 - Se establece el número de errores y defectos y se organizan en forma descendente.

- Se computa el coste global de errores y defectos de cada categoría.
- Se desarrollan planes para modificar el proceso con el intento de eliminar la clase de errores y defectos que sean más costosos.

➤ **Métricas del proyecto:** la utilización de métricas para el proyecto tiene dos aspectos fundamentales:

- **En primer lugar**, estas métricas se utilizan para minimizar la planificación de desarrollo haciendo los ajustes necesarios que eviten retrasos y reduzcan problemas y riesgos potenciales.
- **En segundo lugar**, las métricas para el proyecto se utilizan para evaluar la calidad de los productos en el momento actual y cuando sea necesario, modificando el enfoque técnico que mejore la calidad.
- A medida que mejora la calidad, se minimizan los defectos, y al tiempo que disminuye el número de defectos, la cantidad de trabajo que ha de rehacerse también se reduce. Esto lleva a una reducción del coste global del proyecto.
- El modelo de métricas de proyecto [HET93] sugiere que todos los proyectos deberían medir:
 - **Entradas:** la dimensión de los recursos que se requieren para realizar el trabajo.
 - **Salidas:** medidas de las entregas o producto creados durante el proceso de ingeniería del software.
 - En realidad, este modelo se puede aplicar tanto al proceso como al proyecto.

➤ **Mediciones del Software:** las métricas del software se pueden categorizar de las siguientes maneras:

- Entre las medidas **Directas** del **Proceso** de la ingeniería del software se incluyen el coste y el esfuerzo aplicados.
- Entre las medidas **Directas** del **Producto** se incluyen las líneas de código (LDC) producidas, velocidad de ejecución, tamaño de memoria, y los defectos informados durante el período de tiempo establecido.
- Entre las medidas **Indirectas** se incluyen la funcionalidad, calidad, complejidad, eficiencia, fiabilidad, facilidad de mantenimiento y muchas otras capacidades.

➤ **Métricas orientadas al Tamaño:** para desarrollar métricas que se puedan comparar entre distintos proyectos, se seleccionan las líneas de código como valor de normalización. Con los rudimentarios datos contenidos en una tabla se pueden desarrollar para cada proyecto un conjunto de métricas simples orientadas al tamaño:

- Errores por KLDC (miles de líneas de códigos)
- Defectos por KLDC
- Esfuerzo por KLDC
- Páginas de documentación por KLDC
- Errores por Persona/Mes
- LDC por Persona/Mes
- Esfuerzo por página de Documentación

➤ **Métricas orientadas a la Función:** utilizan una medida de la funcionalidad entregada por la aplicación como un valor de normalización. Ya que la funcionalidad no se puede medir directamente, se debe derivar indirectamente mediante otras medidas directas. Las métricas orientadas a la función fueron propuestas por primera vez por **Allan Albrecht (IBM)**, quien sugirió una medida llamada punto de función.

➤ **Métricas para la calidad del software:** La calidad de un sistema, aplicación o producto es tan bueno como los requisitos que describen el problema, el diseño que modela la solución, el código que conduce a un programa ejecutable, y las pruebas que ejercitan el software para detectar errores. Un buen ingeniero del software utiliza mediciones que evalúan la calidad del análisis y los modelos de diseño, el código fuente, y los casos de prueba que se han creado al aplicar la ingeniería del software. Para lograr esta evaluación de la calidad en tiempo real, el ingeniero debe utilizar medidas técnicas que evalúan la calidad con objetividad, no con subjetividad.

- McCall y Cavano definieron un juego de factores de calidad como los primeros pasos hacia el desarrollo de métricas de la calidad del software. Estos factores evalúan el software desde tres puntos de vista distintos:
 - Operación del producto (utilizándolo),

- Revisión del producto (cambiándolo), y
 - Transición del producto (modificándolo para que funcione en un entorno diferente, portabilidad)
- **Corrección:** un programa debe operar correctamente o proporcionará poco valor a sus usuarios. La corrección es el grado en el que el software lleva a cabo su función requerida. (defectos por KLDC).
 - **Facilidad de Mantenimiento:** es la facilidad con la que se puede corregir un programa si se encuentra un error, se puede adaptar si su entorno cambia, o mejorar si el cliente desea un cambio de requisitos. No hay forma de medir directamente la facilidad de mantenimiento, por consiguiente, se deben utilizar medidas indirectas. (tiempo de cambio TMC).
 - **Integridad:** para medir la integridad, se tienen que definir dos atributos adicionales, amenaza y seguridad.
 - **Amenaza:** es la probabilidad de que un ataque de un tipo determinado ocurra en un tiempo determinado.
 - **Seguridad:** es la probabilidad de que se pueda repeler el ataque de un tipo determinado.
 - **Facilidad de uso:** la facilidad de uso es un intento de cuantificar lo amigable que puede ser con el usuario y se puede medir en función de cuatro características:
 - Habilidad intelectual y/o física requerida para aprender el sistema;
 - El tiempo requerido para llegar a ser moderadamente eficiente en el uso del sistema;
 - Aumento neto en productividad medida cuando alguien utiliza el sistema moderadamente y eficientemente.
 - Valoración subjetiva de la disposición de los usuarios hacia el sistema.
- **Eficacia de la Eliminación de Defectos (EED):** es una medida de la habilidad de filtrar las actividades de la garantía de calidad y de control al aplicarse a todas las actividades del marco de trabajo del proceso. Cuando un proyecto se toma en consideración globalmente, EED se define de la forma siguiente: $EED = E / (E + D)$ (4.4) donde E es el número de errores encontrados antes de la entrega del software al usuario final y D es el número de defectos encontrados después de la entrega.
- **Estimación y Planificación**
- **Estimación:** predicción de duración, esfuerzo y costes requeridos para realizar todas las actividades y construir todos los productos asociados al proyecto.
 - **Planificación:** la planificación es el primer paso en la realización de un proyecto informático y en gran parte, el responsable del éxito o fracaso del mismo. Organización de gente y tareas. Se incluyen los siguientes procesos:
 - Selección de una estrategia para la obtención de unos productos finales dados.
 - Definición de actividades a realizar para lograr ese objetivo.
 - Coordinación de concurrencia y solapamiento de dichas actividades.
 - Asignación de recursos a las mismas.
- **Planeación y Control de Actividades**
- **Planeación:** incluye a todas las actividades que se requieren para la selección del equipo de análisis de sistemas, la asignación de proyectos apropiados, la estimación de tiempo que cada tarea requiere para su ejecución y la programación del proyecto, de tal forma que las tareas se concluyen oportunamente.
 - **Control:** denota el uso de la retroalimentación para darle seguimiento al proyecto, esto incluye comparar lo planificado con lo realizado, tomar acciones adecuadas para acelerar o reprogramar las actividades y en consecuencia se concluyan a tiempo y la motivación de los miembros del equipo.
 - **Herramientas:** Gantt y PERT.
- **Aspectos que afectan la Estimación**
- **La complejidad del proyecto:** tiene un gran efecto en la incertidumbre, que es inherente en la planificación.
 - **El tamaño del proyecto:** puede afectar la precisión y la eficiencia de las estimaciones.

- **Grado de Incertidumbre estructural:** tiene también efecto en el riesgo de la estimación. La estructura se refiere al grado en el que los requisitos son definidos, la facilidad con la que pueden subdividirse funciones, y la naturaleza jerárquica de la información que debe procesarse.
- **El riesgo se mide por el grado de incertidumbre en las estimaciones.**
- **Elementos sobre los que hay que estimar:**
 - **Duración:** Tiempo (meses, año etc.)
 - **Esfuerzo:** unidad de esfuerzo (p/m, p/año, etc.)
 - **Costes:** Euros
 - **Tamaño:** Líneas de Código (LDC, KLDC) – Puntos de Función (PF)

➤ **Técnicas de Estimación: Descomposición**

- **Estimación del Esfuerzo**
 - Enfoque “divide y vencerás”
 - Descomponer en módulos
 - Tareas o funciones
 - **Modelo matricial:** Descomponer en funciones/tareas, realizar una matriz, estimar el esfuerzo para cada celda de la matriz, calcular la suma total y aplicar costes estándares de la empresa.
 - **Modelo jerárquico:** Descomponer las tareas/módulos en forma de árbol y calcular el esfuerzo de cada nodo de abajo hacia arriba.

➤ **Técnicas de Estimación: Empíricas**

- **Técnicas Empíricas:** Usa formulas derivadas empíricamente para predecir el esfuerzo en función de LCD,PF (atributos de software terminado)
 - **COCOMO (CONstructive COst MODEL):** Propuesto por *Barry Boehm* en 1981. “Software Engineering Economics”. Los datos empíricos se obtienen de una muestra limitada de proyectos. El modelo no es adecuado para todas las clases de software y en todos los entornos de desarrollo. Grandes variaciones si no se calibran bien.
 - **Modelo Básico:** mide el esfuerzo en función de las líneas de códigos.
 - **Modelo Intermedio:** mide además, fiabilidad, ejecución, almacenamiento, Base de datos, etc.
 - **Modelo Avanzado:** además, evalúa el impacto del coste de cada proceso del software.
 - **COCOMO II:** Se adapta las técnicas de Modelos de ciclo de vida no secuenciales y de prototipado rápido, modelos basados en reutilización de productos COTS, desarrollo basado en componentes, generación de aplicaciones y enfoques orientados a objetos y middleware distribuido.
 - **Utiliza:** Líneas de Código (SLOC), Puntos de Función (PF) y Puntos de Objeto (PO).
 - **Modelos de estimación:** Composición de aplicaciones (Usa PO). Diseño Preliminar (Usa PF y “Backfiring”). Nuevos modelos para reutilización.
 - **PUTMAN:** para las mediciones utiliza el esfuerzo durante todo el ciclo de vida en personas/años, número de líneas de código, tiempo de desarrollo en años y la constante del estado de la tecnología.
 - **Modelo de Puntos de Función:** Se basa en el diseño lógico para cualificar la funcionalidad externa que proporciona el software. Su objetivo es proporcionar métricas sobre: Productividad, Calidad, Coste Documentación y Líneas de Código. Ofrece un factor de normalización para comparar diferente software. Mide lo que el usuario pide y lo que el usuario recibe. Pasos del modelo:
 - Definición de los límites del sistema informático
 - **Definición de parámetros:** Entrada, Salida, Consultas, Datos lógicos Internos y Externos. Con estos parámetros se obtienen los Puntos de Función Ajustados, a cuyo valor hallado se le aplica un Factor de Ajuste en base a valoraciones subjetivas sobre la aplicación y su entorno.
 - Valoración de la Complejidad de cada parámetro.
 - Análisis de las características generales del sistema para determinar el factor de ajuste.

➤ **Técnicas de Estimación: Herramientas automáticas:** Implementan de forma automática técnicas de descomposición o modelos empíricos, permiten estimar costes, esfuerzo, riesgos, productividad, personal medio

por mes, etc., análisis de diversos escenarios, se requiere experiencia para su uso. Ej.: CoStar, USC COCOMO II, etc.

- **Planificaciones y Costes Globales:** tiene como cobertura al Hardware, Software, Personal, Viajes, Servicios, Gastos comunes, etc.

- **Planificación**

- **Alcance del sistema de software:** que va a ser parte de la aplicación y que no. Tareas a realizar/productos a generar. Work Breakdown Structure (WBS) Estructura de Desglose del Trabajo. Dividir el proyecto en partes más pequeñas Unidades de Trabajo (Work Packages). Orientado al Proceso Vs. Orientado al Proyecto.
- **Proceso de Desarrollo**
 - Ciclo de vida
 - Fases
 - Técnicas y métodos
 - Software/Hardware
 - Hitos
 - Resultados
- **Recursos y distribución:** Consideraciones a tener en cuenta para planificar: la duración de cada tarea, fecha de inicio y fecha de fin, recursos a asignar a cada tarea.
 - **Recursos Humanos:** Perfil requerido, disponibilidad, duración, fecha de comienzo, dedicación.
 - **Recursos de Hardware/Software:** Descripción, disponibilidad, duración, fecha de compra/uso.
- **Tiempos**
- **Formación del equipo de desarrollo**
- **Garantía de calidad**
- **Procedimientos de evaluación y seguimiento**
- **Gestión de configuraciones**
- **Análisis de riesgo**

- **Planificación de Actividades**

- **PERT (Técnica de revisión y evaluación del programa):** Se representan por una red de flechas y nodos que luego se evalúan para determinar cuáles son las actividades críticas y mejorar su programación si fuese necesario, como también para revisar el avance del proyecto. Es un diagrama que nos permite la identificación fácil del orden de la precedencia, la ruta crítica y en consecuencia las actividades críticas, además del cálculo sencillo de la duración de la holgura.
 - **Entradas:** tareas y relaciones, duración total de las tareas, esfuerzo dedicado a cada tarea, lo más pronto y lo más tarde que pueden empezar/acabar las tareas.
 - **Salidas:** Camino crítico, fecha inicio/fin más tempranas para cada tarea, fecha de inicio/fin más tardías para cada tarea. Holgura para cada tarea.
- **GANTT:** Es esencialmente un diagrama que contiene barras que representan cada una de las actividades, y cuya longitud representa la duración de la actividad respectiva. Un diagrama de GANTT de una dimensión es un calendario, siendo una técnica muy usada para la planeación de las actividades. Cuando las actividades se realizan en forma simultánea la resultante es un diagrama de dos dimensiones.
 - **Entradas:** tareas, duración de las tareas y fechas de inicio y fin de las tareas.
 - **Salidas:** Presentación de la planificación temporal de las tareas.

Tema2: La Información como Recurso de las Organizaciones

- **Organización:** es una estructura social formal estable que toma recursos del entorno y los procesa para producir salidas. Los entornos consumen los productos y servicios a cambio de las entradas que proporcionan.
 - **Componentes principales de la definición de organización (Galbraith 1977)**
 - Las organizaciones están compuestas de individuos y grupos de individuos
 - Buscan la realización de objetivos compartidos
 - Diferencia a través de la división del trabajo
 - Integrada por proceso de toma de decisión vinculados por información
 - Opera continuamente a través del tiempo
- **Definición de Administración:** es la conducción de una empresa, por medio del planeamiento, organización, coordinación y control de sus recursos humanos y materiales, hacia el logro de un objetivo determinado.
- **Teorías Organizativas**
 - **Clásica o Tradicional:** Escuela de la Administración Científica (Taylor 1910). Escuela de la Administración Industrial y General (Fayol 1919). Escuela Estructuralista (Max Weber 1920).
 - **Teoría Neoclásica:** Escuela de las Ciencias del Comportamiento Humano (Elton Mayo 1930).
 - **Teoría Moderna:** Escuela de las Ciencias Administrativas. Escuela de la Decisión (Simon y March). Investigación de Operaciones.
 - **TGS:** Enfoque de la Teoría General de los Sistemas (1980)
 - **EC:** Enfoque de Contingencia.
- **Modelo de Kast:** las Organizaciones son sistemas socio-técnicos abiertos compuestos de un cierto número de subsistemas. Dichos sistemas reciben insumos de energía, información y materiales del medio, los transforman y los regresan en forma de productos.
- **Modelo de Regulación:** El propósito del sistema de administración, y en consecuencia el del sistema de información, es regular y adaptar el comportamiento de la organización bajo las perturbaciones generadas por un medio ambiente cambiante y no controlable (Ashby).
 - **Funciones administrativas:**
 - **Procesamiento básico:** es la recolección, almacenamiento y transformación de datos acerca del estado y salidas de los procesos.
 - **Procesamiento de datos analíticos:** es un procesamiento de datos más complejos para modelar el comportamiento de la organización a través de los datos.
- **Reingeniería:** Es la revisión fundamental y el rediseño radical de procesos para alcanzar mejoras espectaculares en medidas críticas y contemporáneas de rendimiento, tales como costos, calidad, servicio y rapidez.
 - **Objetivos:**
 - Procesos reeducativos de fondo.
 - Habilidades para el puesto.
 - Orientaciones productivas.
 - Personal de los diferentes niveles tomen decisiones y por lo tanto estén debidamente facultados para ello.
 - Procesos que rompan con los actuales.
- **Control de Gestión:** es un proceso que sirve para guiar la gestión hacia los objetivos de la organización y un instrumento para evaluarla.

- **Características:** Despliegue de la estrategia en toda la organización, desarrolla actividades de planificación, control y diagnóstico, evalúa el desempeño de la organización, es un medio para movilizar el talento y la energía del colectivo hacia el logro de los objetivos de la organización, es un medio para gestionar el cambio.
- **Tópicos:** Es regulada por los objetivos, cumple con las siguientes actividades: formulación de objetivos, fijación de estándares, programa de acción, utilización de recursos, mediación de resultados, análisis de desviaciones, corrección del desempeño o mejora. Se distingue diferenciación entre el concepto de gestión, control de gestión y la función de control, pero no se observa la misma precisión al establecer fronteras. Hasta la década de 1970, el Control de Gestión es verificación de lo realizado. Subordinado a la planificación estratégica (según algunos autores). Ofrecen una visión dinámica, de cambio en el diseño de los Sistemas de Control de Gestión, los cuales deben modificarse a la velocidad con que cambian las estrategias.

➤ **Sistemas de Información**

- Sistema de Procesamiento de Datos
- Sistema Informático para la Administración (MIS)
- Sistema de Apoyo para la toma de decisiones (DSS)
- Sistema Expertos e Inteligencia Artificial (IA)

➤ **Sistemas Automatizados**

- **Elementos en común:** Hardware, Software, Personas, Datos, Procedimientos.
- **Sistemas en Línea**
- **Sistemas en Tiempo Real**

➤ **La Organización como Sistemas:** Están diseñadas para el cumplimiento de metas y objetivos específicos mediante el empleo de diversos recursos, incluyendo el factor humano, se integran como pequeños sistemas interrelacionados, las cuales realizan funciones especializadas, todos los sistemas y subsistemas se encuentran interrelacionados y son interdependientes. Para adaptarse las organizaciones tienen la necesidad de recurrir a gente, materias primas e información a través de sus límites y de intercambiar sus productos terminados, servicios o información hacia el mundo externo. La retroalimentación es un mecanismo para el control de un sistema.

➤ **Niveles de Administración**

- Administración de Operaciones
- Administración Media
- Administración Estratégica

➤ **Diseño de la Organización**

- Estructura Jerárquica
- Estructura por Proyecto
- Estructura Matricial
- Modelo de Procesamiento de la Información

➤ **Análisis de Decisiones**

- Para precisar los requisitos del análisis de decisiones se debe:
 - Identificar los objetivos de la Organización.
 - Conocer la Base de la Organización y tener conocimiento de las técnicas de recopilación de datos.
 - Establecer el enfoque de lo general a lo particular, que es decisivo, pues deben relacionarse todas las decisiones con sus objetivos.

- **Nivel Superior:** Objetivos muy amplios e inclusive ambiguos. (Decisiones carecen de estructura, o se encuentran semiestructuradas)
 - **Nivel Intermedio:** Tienen una mayor precisión, los objetivos son más específicos. (decisiones semiestructuradas, debido a la existencia de múltiples objetivos y a las diversas opciones de que disponen los gerentes de mandos medios)
 - **Nivel Operativo:** Objetivos más precisos: clasificaciones efectivas, determinación de intereses, reglas de cobro y de cancelación, cobranzas y políticas de cancelación. (Decisiones todavía son de carácter semiestructurado y estructurado)
 - **Alternativas para el Análisis de Decisiones:** Lenguaje Estructurado, Tabla de Decisiones y Arboles de Decisiones.
- **Sistema de Soporte/Apoyo a las Decisiones (Decisión Support System):** Son sistemas que proporcionan información y soporte para tomar una decisión.
- En general, podemos decir que un **DSS** es un sistema informático utilizado para servir de apoyo, más que automatizar, el proceso de toma de decisiones. La decisión es una elección entre alternativas basadas en estimaciones de los valores de esas alternativas. El apoyo a una decisión significa ayudar a las personas que trabajan solas o en grupo a reunir inteligencia, generar alternativas y tomar decisiones. Apoyar el proceso de toma de decisión implica el apoyo a la estimación, la evaluación y/o la comparación de alternativas. En la práctica, las referencias a **DSS** suelen ser referencias a aplicaciones informáticas que realizan una función de apoyo.
 - Un **DSS** se distingue del resto de los sistemas que puede tener un compañía porque lo integran los siguientes elementos: **Modelo de decisiones, Base(s) de Datos, Interfaz de usuario y análisis de escenarios What if (¿qué pasaría si?) y Goal seek (búsqueda de meta).**
 - Las decisiones se llevan a cabo según tres tipos de condiciones:
 - **Certeza:** significa que al hacer una elección conocemos todo de antemano.
 - **Incertidumbre:** implica lo opuesto; no sabemos nada acerca de las probabilidades o las consecuencias de nuestras decisiones.
 - **Riesgo:** las decisiones echas bajo riesgos consideran cierto conocimiento de nuestras alternativas (variables controlables), de lo que no podemos controlar sino solo estimar (variables ambientales) y de lo que serán las consecuencias (variables dependientes).
 - Es común que los tomadores de decisiones se caractericen por ser analíticos o heurísticos:
 - **Analíticos.** Se basa en la información que se adquiere y evalúa de manera sistemática para reducir alternativas y elegir con base en tal información. Hacen uso de las matemáticas para modelar problemas y de los algoritmos para resolverlos. Normalmente busca soluciones óptimas más que satisfactorias, se requiere que la información esté disponible, sea razonable, completa y precisa.
 - **Heurísticos.** Decide con la ayuda de ciertos lineamientos, y se basa fundamentalmente en la experiencia. Aprenden a actuar, utilizan el ensayo y el error para encontrar la solución, ya que se basan en su experiencia, su sentido común les sirve de guía.
 - **Decisión Estructurada:** Se hace de acuerdo a procesos específicos.
 - **Decisión no Estructurada:** Alto grado de libertad e incertidumbre. Decisiones para las cuales la información que se obtiene de un sistema computacional es solo una porción del conocimiento total requerido para tomar una decisión.
 - **Decisión Semi-estructurada:** Existen componentes estructurados (datos fijos, modelos o reglas de decisión), pero la decisión pertenece al humano.
- **Otros factores de la Organización**
- **Estilo de liderazgo:**
 - **Autocráticos:** del griego «αὐτο» (uno mismo) y «κράτος» (gobierno o poder), es un sistema de gobierno en la medida en que su autoridad recae sobre una sola persona que no tiene o no reconoce ningún tipo de regulación o limitación para ejercer su poder.

- **Promotores:** agentes promotores de cambios y desarrollas de habilidades y actitudes que inspiren y faciliten el trabajo en equipo para el logro de objetivos comunes, a través de la colaboración entusiasta y comprometida de sus colaboradores.
 - **Consideraciones tecnológicas**
 - Tecnología de operaciones
 - Tecnología de materiales
 - Tecnología del conocimiento
 - **Cultura de la Organización**
 - El simbolismo verbal
 - El simbolismo no verbal
 - **Relaciones humanas**
 - **Tipos de Usuarios de Sistemas**
 - Clasificación por categoría de trabajos
 - Clasificación en categoría por niveles de experiencia (novatos y expertos)
 - **Papeles del Analista de Sistemas**
 - El analista de sistema como consultor
 - El analista de sistema como especialista de apoyo
 - EL analista de sistema como agente de cambio
 - **Cualidades del analista de sistemas**
 - Solucionador de problemas.
 - Buen interlocutor
 - Experiencia en computación
 - Auto disciplinado y auto motivado
 - Administrar y coordinar
- **Sistema socio-técnico:** El término sistema **socio-técnico** fue originalmente usado para designar la interacción obrero – máquina en ambientes de trabajo industrial. Actualmente se ha extendido su alcance para abarcar las complejas interacciones entre la tecnología y las personas, así como sus consecuencias psicológicas y culturales. (observando siempre la razón con lo real y no pensado)
- Para Bennett (1990) a partir de los ochenta empieza la muerte del hombre organización, lo describía como un hombre colectivo, preocupado por su bienestar financiero, sus aspiraciones personales, su vida social y aún el carácter moral de la organización.
El comportamiento del hombre organización que puede ser un trabajador administrativo, contable o financiero de nivel medio o alto descansaba sobre las premisas de lo que obtenía de la organización a cambio de su trabajo. El hombre organización se sentía confiado acerca de la paternal benevolencia de la empresa, su trabajo lo creía sería a perpetuidad. Descansaba en la rígida estructura burocrática y jerárquica. La organización era para él su sistema social y sus propias costumbres, hábitos, sentimientos, entendimientos y cábalas.
Desde los sesentas hasta a finales de los setenta las organizaciones prometieron altos salarios, beneficios de por vida, prestigio social y seguridad. A cambio el hombre organización cedió parte de su autonomía, rindió su independencia a la organización por la lealtad corporativa.
El entorno cambia, y como el objetivo de la organización no es, en modo alguno, estático, “los objetivos de la organización tienden constantemente a adaptarse a los valores cambiantes...” (Simon, 1978, p109).
Los ochenta fueron tiempos de cambio y las purgas de personal empezaron. Los recortes de personal decimaron las áreas del hombre organización: publicistas, contadores, planificadores, asesores de relaciones públicas, diseñadores y otros asesores. La reorganización fue dramática, el desempleo se incrementó, desapareció la seguridad en el trabajo, la lealtad dejó de ser el elemento más importante para ser sustituida por la eficiencia y los resultados.
Todo ello provocó la muerte del hombre organización y del nacimiento del hombre moderno para quien el logro, el prestigio, el estatus y la remuneración económica empiezan a ser el móvil de sus vidas. (Bennett, 1990, p17-22)

En esas circunstancias, el “objetivo de la organización rara vez coincidirá exactamente con los objetivos personales incluso de aquellos participantes cuyo interés en la organización radica en la consecución de su finalidad” (Simon, 1978, p109).

El entorno económico y social estaba cambiando rápidamente. Industrias enteras dejan de ser competitivas, el objetivo por bajar los costos de operación conducen a una lucha despiadada y brutal por la competitividad. Los efectos se reflejarían en la pérdida de estabilidad de los trabajadores y la reducción significativa de sus paquetes de prestaciones (Bennett, 1990, p176-9)

El resultado fue el nacimiento del nuevo Hombre Organización, jóvenes ejecutivos que saben que la seguridad en el trabajo no existe, no pueden dar su lealtad a la organización y la desconfianza está siempre presente. Saben que el trabajo no será para toda la vida y buscan la experiencia y el entrenamiento que les sirva cuando sea necesario trasladarse a otra empresa. Sobre todo el nuevo Hombre Organización es independiente (Bennett, 1990, p254-7)

Ahora el criterio de eficiencia dicta, en tales organizaciones, la elección de aquella alternativa, entre todas las que dispone el individuo, que proporcione mayor rendimiento neto (en dinero) a la organización (Simon, 1978, p164).

Los nuevos modelos organizacionales significan anonimato y homogeneidad pero no autoritarismo como en la visión clásica. La muerte del Hombre organización permite el nacimiento del Hombre profesional (Bennis, 1966, 25).

Aparece así, el hombre racional que pregonan March y Simon, cuyo “análisis del conocimiento encaja en el marco más amplio de la teoría de la organización, no en sustitución de estos enfoques primitivos, sino como un complemento de los mismos” Estos enfoques intentaban “sustituir la fantasía por el hecho en la comprensión de la mente humana y el comportamiento humano en la organización (March y Simon, 1981, p 232-3).

➤ La Organización Inteligente

- La organización, especialmente la empresarial, es hoy un ámbito propicio para el surgimiento de teorías y propuestas.
- La era del conocimiento requiere la necesidad de ampliar y mejorar el capital intelectual que permita potenciar las ventajas competitivas de las organizaciones de nuestro tiempo.
- Las Organizaciones Inteligentes son aquellas capaces de aprender permitiendo así expandir sus posibilidades de crecimiento.
- No basta con adaptarse y sobrevivir sino sobre todo desarrollar la capacidad de crear. La construcción de una organización con auténtica capacidad de aprendizaje y creatividad se basa en el desarrollo de cinco disciplinas: dominio personal, trabajo en equipo, visión compartida, modelos mentales y pensamiento sistémico.

➤ Las Disciplinas para construir una Organización Inteligente

- Hay solo un camino para adaptarse al cambio constante: convertirse en una organización de aprendizaje continuo.
- Para esto es necesario dominar las Cinco Disciplinas que plantea Peter Senge, estas son: dominio personal, trabajo en equipo, visión compartida, modelos mentales y pensamiento sistémico.
- El pensamiento sistémico es la disciplina que ensambla las otras cuatro disciplinas del aprendizaje organizacional.
- El pensamiento sistémico es la piedra angular, el principal hilo conductor del texto.

• **Disciplinas Individuales**

- **Dominio Personal:** la capacidad de aclarar y profundizar constantemente nuestra visión personal.
- **Modelos Mentales:** la capacidad de desenterrar nuestras imágenes internas del mundo, examinarlas y abrirlas a la influencia de los demás.

- **Disciplinas Grupales**

- **Creación de una visión compartida:** la práctica de desenterrar imágenes de futuro compartidas que promuevan el auténtico compromiso.
- **Aprendizaje en equipo:** la capacidad de pensar juntos que se consigue mediante el dominio de la práctica del dialogo y el debate.
- **Pensamiento sistémico:** la disciplina que integra a las anteriores, uniéndolas en un conjunto coherente de teoría y práctica.

➤ **Percepción del todo y no solo del acontecimiento**

- Los cambios a corto plazo de los modelos mentales rutinarios y diarios, se acumulan a lo largo del tiempo y se convierten en cambios en las creencias a largo plazo, profundamente arraigados.
- El pensamiento sistémico, la percepción del todo estructural y no solo del acontecimiento parcial, pretende (y lo consigue), ayudar al enfoque estratégico a medio y largo plazo, en lugar de contentarse con la resolución de problemas (táctica) en el corto plazo.
- La modelización del pensamiento en grupos despersonaliza la asertividad y permite al equipo llegar a soluciones que no son de compromiso.

➤ **Impacto ético y social de los sistemas de información**

- En lo ético se debe tener presente los siguientes retos gerenciales:
 - **Entender los riesgos morales de la nueva tecnología.** La protección de la privacidad individual se ha convertido en un problema ético grave debido a la rapidez de los cambios tecnológicos, es importante que la gerencia realice un análisis del impacto ético y social de las nuevas tecnologías.
 - **Establecer políticas de ética corporativa que incluyan cuestiones de sistemas de información.** Es evidente que las corporaciones deben contar con una política de ética en el área de sistemas de información que contenga aspectos como privacidad, propiedad, responsabilidad formal, calidad de sistemas y calidad de vida.
- **Problemas éticos y sociales relacionados con los sistemas**
 - La ética se refiere a los principios morales que cada individuo puede usar para tomar decisiones que guíen su conducta, una nueva tecnología dará paso a una nueva cuestión ética cuya decisión implica una responsabilidad personal, es decir, acepta los costos y beneficios de nuestra decisión, la responsabilidad formal que implica quien realice las acciones, es decir, quien debe rendir cuentas y la responsabilidad legal que es una característica de los sistemas políticos.
- **Cinco dimensiones morales de la era de la información**
 - **Derechos y obligaciones de información:** ¿Qué derechos de información tienen los individuos y las organizaciones respecto a la información acerca de sí mismos? ¿Qué pueden proteger? ¿Qué obligaciones tienen los individuos y las organizaciones en lo concerniente a esta información?
 - **Derechos de propiedad:** ¿Cómo se protegerán los derechos tradicionales de propiedad intelectual en una sociedad digital en la que es difícil rastrear y justificar la propiedad, y es muy fácil hacer caso omiso de tales derechos?
 - **Responsabilidad formal y control:** ¿Quiénes pueden rendir cuentas y hacerse responsable por los daños a los derechos de información y propiedad individuales y colectivas?
 - **Calidad del Sistema:** ¿Qué estándares de calidad de datos y sistemas se deben exigir para proteger los derechos individuales y la seguridad de la sociedad?

- **Calidad de Vida:** ¿Qué valores deben preservarse en una sociedad basada en la información y los conocimientos? ¿Qué instituciones deben ser protegidas contra violaciones? ¿Qué valores y prácticas culturales apoya la nueva tecnología de información?

- **Responsabilidades**

- **Responsabilidad personal:** Aceptar posibles costos, deberes y obligaciones de las decisiones que uno toma.
- **Responsabilidad formal:** Los mecanismos para evaluar la responsabilidad de las decisiones tomadas y las acciones realizadas.
- **Responsabilidad legal:** es una característica de los sistemas políticos en los que hay leyes que permiten a los individuos ser compensados por los perjuicios infligidos en ellos por otros actores, sistemas u organizaciones.
- **Debido proceso:** Proceso en el que las leyes se conocen y son bien entendidas, y en el que existe la posibilidad de apelar a autoridades más altas para asegurar que las leyes se apliquen correctamente.

- **Análisis ético**

- **Identificar y describir claramente los hechos.** Averiguar quién hizo qué a quién, y dónde, cuándo y cómo.
- **Definir el conflicto o dilema e identificar los valores de orden más alto en cuestión.** Las cuestiones éticas, sociales y políticos siempre hacen referencia a valores más altos.
- **Identificar los grupos de interés.** Toda cuestión ética, social y política tiene grupos de interés: protagonistas del juego que están interesados en el desenlace.
- **Identificar las opciones razonables que se pueden tomar.** Tal vez ninguna de las opciones satisfaga todos los intereses implicados, pero es probable que algunas de ellas lo hagan mejor que otras.
- **Identificar las posibles consecuencias de las opciones.** Algunas opciones pueden ser éticamente correctas, pero desastrosas desde otros puntos de vista. Siempre es necesario preguntarse: “¿Qué pasaría si siempre se eligiera esta opción?”

- **Principios éticos candidatos**

- **Regla de Oro:** Tratar a los demás como se quiere que los demás lo traten a uno.
- **Imperativo Categórico de Immanuel Kant:** Si una acción no es correcta para todos, no es correcta para nadie.
- **Regla del Cambio de Descartes:** Si una acción no puede efectuarse rápidamente, no debe efectuarse nunca.
- **Principio Utilitarista:** Efectuar la acción que logra el valor más alto o mayor.
- **Principio de Aversión al Riesgo:** Efectuar la acción que produce el menor daño, o que cuesta menos.
- **Regla Ética “Nada es Gratis”:** Suponer que prácticamente todos los objetos tangibles e intangibles son propiedad de alguien más, a menos que exista una declaración específica que no diga que no es así.

➤ **Las dimensiones morales de los sistemas de información**

- **Derechos de Información**

- **Peligros para la privacidad relacionados con Internet Spamming,** la práctica de enviar correo electrónico y otras comunicaciones electrónicas no solicitadas.
- **Cuestiones éticas.** Las cuestiones de ética en esta era de la información son las siguientes ¿en qué condiciones debe invadir la privacidad de otros? ¿Qué justifica inmiscuirse en la vida de otros mediante vigencia subrepticia, investigación de mercados o cualquier otro mecanismo? ¿Se tiene que informar a las personas que se está dando la información de historial crediticio para fines de selección de empleados?
- **Cuestiones sociales.** El aspecto social de la privacidad tiene que ver con el nacimiento de expectativas de privacidad o normas de privacidad, así como actitudes públicas.

- **Cuestiones políticas.** El lado político de la privacidad tiene que ver con el desarrollo de estatutos que gobiernen las regiones entre quienes mantienen registros y los individuos.
- **Derechos de Propiedad**
 - **Secreto industrial.** Cualquier obra o producto intelectual elaborado para fines de negocios, siempre que no se base en información del dominio público. La limitación de la protección de los secretos industriales es que aunque prácticamente todos los programas de software con cierta complejidad contienen elementos únicos de algún tipo, es difícil impedir que las ideas del trabajo caigan dentro del dominio público cuando el software se distribuye ampliamente.
 - **Derecho de Autor.** Concesión otorgada por ley que protege a los creadores de una propiedad intelectual contra el copiado por parte de otros con cualquier fin, durante un periodo de 28 años.
 - **Patente.** Documento legal que otorga al titular durante 17 años, un monopolio exclusivo sobre las ideas en que se basa un invento. Está diseñada para asegurar que los inventores de máquinas o métodos nuevos sean recompensados por su labor, mientras su producto se usa ampliamente.
- **Calidad de Sistemas:** calidad de datos y errores del sistema; existe una barrera tecnológica que impide llegar al software perfecto y los usuarios deben ser conscientes de la posibilidad de una falla catastrófica debido a la calidad de los datos.
 - **Cuestiones éticas.** La cuestión ética central relacionada con la calidad que presentan los sistemas de información es en qué punto debe el productor ofrecer software o servicios para ser consumidos por otros.
 - **Cuestiones sociales.** La principal cuestión relacionada con calidad también tiene que ver con las expectativas.
 - **Cuestiones políticas.** La principal cuestión Política relacionada.
- **Calidad de Vida:** equidad, acceso, fronteras; el cambio hacia una computación altamente descentralizada aunado a una ideología de facultación y la descentralización de la toma de decisiones entre miles de trabajadores de las organizaciones.
 - **Delito por computadora.** La comisión de actos ilegales mediante el uso de una computadora, o contra un sistema de computación.
 - **Abuso de computadoras.** La comisión de actos relacionados con una computadora que si bien no son ilegales, se consideran faltos de ética
- **Enfermedades relacionadas por Computadoras**
 - **Lesión por tensión repetitiva (RSI).** Enfermedad ocupacional que se presenta cuando se obliga a grupos musculares a efectuar acciones repetitivas con altas cargas de impacto o miles de repeticiones con cargas de bajo impacto.
 - **Síndrome de Túnel del carpo (CTS).** Tipo de RSI en el que la presión sobre el nervio mediano que pasa por la estructura ósea “túnel del carpo” de la muñeca produce dolor.
 - **Síndrome de visión de computadora (CVS).** Condición de tensión ocular relacionada con el uso de pantallas de computadora, cuyos síntomas incluyen dolores de cabeza, visión borrosa y ojos resecos e irritados.
 - **Tecno-estrés.** Tensión inducida por el uso de computadoras, cuyos síntomas incluyen fastidio, hostilidad hacia las personas, impaciencia y nerviosismo.
- **Acciones Gerenciales:** un código de ética corporativo; consiste en desarrollar un conjunto de normas éticas específicas para sistemas de información en cada una de las 5 dimensiones morales:
 - **Derechos y obligaciones de información** (privacidad del correo electrónico y monitoreo del lugar de trabajo).

- **Derechos y obligaciones de propiedad** (licencia de software, propiedad de los datos).
- **Responsabilidad formal y control** (se designa a una sola persona, responsable de todo el sistema de información).
- **Calidad del sistema** (Niveles generales de la calidad de los datos y errores del sistema que se puede tolerar).
- **Calidad de vida** (El propósito de los sistemas es mejorar la calidad de vida de los clientes y empleados).

Tema 3: Ingeniería del Software

- **Software:** Se conoce como software al equipamiento lógico o soporte lógico de un sistema informático, que comprende el conjunto de los componentes lógicos necesarios que hacen posible la realización de tareas específicas, en contraposición a los componentes físicos que son llamados hardware. (Programas + Datos + Documentación)
 - **Definición:** Es el conjunto de los programas de cómputo, procedimientos, reglas, documentación y datos asociados, que forman parte de las operaciones de un sistema de computación.
 - **Características:** el software se desarrolla, no se estropea, un sistema de software debe ser fácil de mantener, construido y documentado para permitir cambios, debe ser fiable y hacer lo que se espera que haga, no debe fallar más a menudo de lo que se acordó en las especificaciones, debe ser eficiente y fácil de usar.
- **Evolución hacia la Ingeniería:** A mediados de los 60, la creación de un producto software se convertía en una tarea angustiosa, se hizo por tanto necesario introducir una serie de herramientas y procedimientos que facilitarían por un lado, la labor de creación de nuevo software y por otro, la comprensión y el manejo del mismo. Estos fueron los inicios de la Ingeniería del Software. Con el paso del tiempo, la evolución de estos métodos nos han llevado a reconocer la Ingeniería del Software como una verdadera disciplina, derivada de una investigación seria y de un estudio minucioso. Entre 1965 y 1970 se da la aceptación de los métodos de programación estructurada, a partir de 1970 hasta 1990, aunque se continuaba trabajando sin metodologías de programación, se introducen los lenguajes de programación orientada a objetos, nace el lenguaje C, se establecen empresas de servicios informáticos, se generan controles de producción y marketing, surgen las interfaces, base de datos, programación recurrente, mantenimiento, documentación entre otros, desde 1990 en adelante se genera software a partir de metodologías con profesionales capacitados estableciendo equipos de desarrollo, automatización en los procesos, utilización de herramientas CASE...
- **Crisis del Software:** La crisis del software se fundamentó en el tiempo de creación de software, ya que en la creación del mismo no se obtenían los resultados deseados, además de un gran costo y poca flexibilidad. Básicamente, la crisis del software se refiere a la dificultad en escribir programas libres de defectos, fácilmente comprensibles, y que sean verificables.
 - **Causas:** hardware más potente, mayor demanda, falta de metodologías y técnicas, uso inadecuado de recursos, sistemas más complejos, poca información de los desarrolladores.
 - **Síntomas y consecuencias:** baja productividad de los desarrolladores, no se cumple con las expectativas de los usuarios, la calidad no es la adecuada, los costos son elevados, los plazos estipulados no concuerdan con el progreso, la portabilidad y la eficiencia no son posibles de determinar.
 - **Solución:** Aplicar la Ingeniería del Software a la construcción de Sistemas Informáticos. La necesidad de un enfoque de ingeniería en el desarrollo del software fue propuesta en una conferencia de la OTAN en 1968.
- **Ingeniería del Software (Definición):** Según Bauer (1969) entendemos por Ingeniería del Software “el establecimiento y uso de principios de ingeniería robustos, orientados a obtener software económico, que sea fiable y funcione eficientemente sobre máquinas reales”. La Ingeniería del Software abarca tres elementos (métodos, herramientas y procedimientos) que facilitan el control de desarrollo software y suministran las bases para la construcción de software de alta calidad de forma productiva.
 - **Objetivos:** Construir instrumentos (Hardware, Software) que ayuden o faciliten al ser humano la realización de alguna tarea, conseguir un producto de Software fiable, de alta calidad y de bajo coste, conducir un proceso de desarrollo y mantenimiento de software de manera eficiente y con éxito. Mejorar la calidad de los productos, aumentar la productividad, facilitar el control y seguimiento del proceso de desarrollo, definir una disciplina que garantice la producción sistemática y el mantenimiento de productos de software desarrollados en plazo y costes estipulados.
 - **Disciplinas que la componen:** Economía, Ingeniería, Informática, Psicología y Gestión.

➤ **Fundamentos y Técnicas**

- **Métodos:** definen como construir el software desde el punto de vista técnico. (Planificación y estimación de proyectos, Análisis de Requisitos, Diseño, Codificación, Pruebas, Mantenimiento)
- **Herramientas:** proporcionan un soporte automático o semiautomático para los métodos (CASE, CAD, etc.)
- **Procedimientos:** punto de unión entre métodos y herramientas, definen la secuencia en la que se aplican métodos, como usar las herramientas, las entregas que se requieren, controles de seguimiento y calidad, guías para facilitar la labor de gestores y desarrolladores, etc.
- **Otras técnicas de apoyo:** Abstracciones: modelos de ciclo de vida, principios de las distintas fases, etc.; Representaciones: notaciones y lenguajes, diagramas de GANTT, lenguaje Z, etc.; Evaluaciones: mediciones (sobre proceso y producto).

➤ **Actividades del Ingeniero del Software:** El Ingeniero del Software debe trabajar en equipo, analizar y estudiar problemas, trabajar bajo restricciones de tiempo, costes y recursos, interactuar con clientes y usuarios, tomar decisiones, etc.

- **Actividades de Desarrollo:**
 - Decidir qué hacer (Análisis)
 - Decidir cómo hacerlo (Diseño)
 - Hacerlo (Codificación)
 - Probar el producto (Pruebas)
 - Usar el producto (Entrega/instalación)
 - Mantener el producto (Mantenimiento)
- **Actividades de Control:** se ocupan de evaluar y asegurar la calidad del software
 - Métricas
 - Garantía de calidad
 - Gestión de configuraciones
- **Actividades de Gestión**
 - Planificación y estimación
 - Seguimientos de los proyectos
 - Administración de proyectos
 - Dirección de proyectos
- **Actividades de Operación**
 - Entrega (e instalación)
 - Puesta en marcha
 - Formación a los usuarios

➤ **Ámbito del Software:** el ámbito de un proyecto de software debe ser unívoco y entendible a niveles de gestión y técnico. Los enunciados del ámbito del software deben estar delimitados. En todo Ámbito del Software debe analizarse el contexto, los objetivos de información, la función y el rendimiento.

➤ **Producto y Proceso:** Todo producto surge de una necesidad o problema que requiere de una solución, la cual es llevada a cabo por un proceso que engloba el Desarrollo, el Control, la Gestión y Operación.

- **Producto:** para desarrollar un producto, debe examinarse el problema a resolver justo al inicio del proyecto, establecer el ámbito del producto y delimitarlo teniendo en cuenta que se requieren estimaciones cuantitativas y un plan organizado, un análisis detallado de los requisitos del software que proporcionara la información necesaria para las estimaciones, comprendiendo que los requisitos pueden ser fluidos, cambiando regularmente a medida que progresa el proyecto.
- **Proceso:** las fases genéricas que caracterizan el proceso de software –definición, desarrollo y soporte- son aplicables a todo software. El problema es seleccionar el modelo de proceso apropiado para la ingeniería del software que debe aplicar el equipo del proyecto, entre los que podemos mencionar, modelo secuencial lineal, modelo de prototipado, modelo DRA, modelo incremental, modelo en espiral, modelo WIN-WIN, etc. Cuando

se selecciona un modelo de proceso, el equipo define entonces un plan de proyecto preliminar basado en un conjunto de actividades estructurales, para luego descomponer las distintas fases estableciendo las tareas y el personal requeridos.

- **Maduración del Producto y del Proceso:** la planificación de un proyecto comienza con la maduración del producto y del proceso. Todas las funciones que se deben tratar dentro de un proceso de ingeniería por el equipo de software deben pasar por el conjunto de actividades estructurales que se han definido por una organización de software.
 - **Comunicación con el cliente:** establecer los requisitos entre el desarrollador y el cliente.
 - **Planificación:** tareas requeridas para definir los recursos, planificación temporal, etc.
 - **Análisis de riesgo:** tareas requeridas para valorar los riesgos técnicos y de gestión.
 - **Ingeniería:** tareas requeridas para construir una o más representaciones de la aplicación
 - **Construcción y entrega:** tareas requeridas para construir, probar, instalar y proporcionar asistencia
 - **Evaluación del cliente:** tareas requeridas para obtener información de la opinión del cliente basadas en la evaluación del software.
- **Descomposición del proceso:** Una vez que se ha elegido el modelo de proceso, la **Estructura Común de Proceso (ECP)** se adapta a él. En todos los casos, el ECP puede adaptarse al paradigma. El ECP es invariable y sirve como base para todo el trabajo de software realizado por una organización. El ECP requiere de las siguientes tareas para la actividad de comunicación con el cliente:
 1. Revisar la petición del cliente.
 2. Planificar y programar una reunión formal con el cliente.
 3. Realizar una investigación para definir soluciones propuestas y enfoques existentes.
 4. Preparar un «documento de trabajo» y una agenda para la reunión formal.
 5. Realizar la reunión.
 6. Desarrollar conjuntamente mini-especificaciones que reflejen la información, función y características de comportamiento del software.
 7. Revisar todas las mini-especificaciones para comprobar su corrección, su consistencia, la ausencia de ambigüedades.
 8. Ensamblar las mini-especificaciones en un documento de alcance del proyecto.
 9. Revisar ese documento general con todo lo que pueda afectar.
 10. Modificar el documento de alcance del proyecto cuando se requiera.
- **Proyecto:** para gestionar un proyecto de software con éxito, debemos comprender que puede ir mal (para evitar estos problemas) y cómo hacerlo bien.
 - **Problemas detectados:** La gente del software no comprende las necesidades del cliente, el ámbito del producto está definido pobremente, los cambios están mal realizados, la tecnología elegida cambia las necesidades del negocio cambian, los usuarios se resisten, se pierden los patrocinadores, el equipo del proyecto carece del personal necesario, los gestores evitan buenas prácticas y sabias lecciones.
 - **Sugerencias:** Empezar con el pie derecho, mantenerse, seguimiento del progreso, tomar decisiones inteligentes, realizar un análisis <Postmortem> (después de finalizar el proyecto)
- **El Principio W5HH (Why, What, When, Who, Where, How, How):** Principio creado por Barry Boehm, este principio se basa en una serie de preguntas que conducen a una definición de características claves del Proyecto y el plan de proyecto resultante.
 - **¿Por qué está en desarrollo este sistema?** Ayuda a evaluar la validez de las razones para el trabajo de software.
 - **¿Qué se hará y cuando terminara?** Permite establecer la planificación e identificar tareas e hitos claves del proyecto.
 - **¿Quién es el responsable de una función?** Ayuda a definir el papel de cada integrante.

- **¿En dónde se ubica el centro de la organización?** Permite establecer la distribución de roles y responsabilidades en diferentes equipos o lugares si así se requiere.
 - **¿Cómo se realizará el trabajo en los sentidos técnico y de gestión?** Permite definir una estrategia técnica y de gestión de proyectos.
 - **¿Cuánto se necesita de cada recurso?** Permite especificar los recursos según las estimaciones realizadas con anterioridad.
- **Prácticas Críticas:** Deben establecerse ciertos criterios que permitan una gestión óptima en cuanto a rendimiento se refiere, entre ellos podemos mencionar, la Gestión del riesgo, el Coste empírico y estimación de la planificación, Gestión de proyecto basados en métricas, Seguimiento del valor agregado, Seguimiento de defectos frente a objetivos de calidad y Gestión del programa del personal.
- **Teoría de las 4 C's (Copérnico, Convencional, Conservador, Confianza)**
- **Copérnico:** el sistema que se está construyendo no es el centro del universo, ni va a estar aislado.
 - **Convencional:** los usuarios aceptan mejor las técnicas convencionales.
 - **Conservador:** si en un proyecto no se introducen demasiados cambios, se minimizan los riesgos y se maximizan las posibilidades de financiarlo con éxito.
 - **Confianza:** terminar los proyectos de software con éxito: manteniendo una disciplina en el desarrollo y revisando periódicamente los objetivos y resultados.
- **Principios de la Ingeniería del Software:** haz de la calidad tu razón de trabajar, una buena gestión es más importante que una buena tecnología, las personas y el tiempo no son intercambiables, seleccionar el modelo de ciclo de vida adecuado, entregar el producto al usuario lo más pronto posible, determinar y acotar el problema antes de escribir los requisitos, realizar un diseño, minimizar la distancia intelectual, documentar, las técnicas son anteriores a las herramientas, primero hazlo correcto, luego hazlo rápido, probar, introducir mejoras y modificaciones con cuidado, asunción de responsabilidades, la entropía (incertidumbre) del software es creciente, la gente es la clave del éxito.
- **Principios para una cultura de Ingeniería de Software:** Nunca dejes que tu jefe o cliente te convenza para hacer un mal trabajo, la gente necesita sentir que su trabajo es apreciado, la educación continua es responsabilidad de cada miembro del equipo, el compromiso del cliente es el factor más crítico en la calidad del software, tu mayor desafío es compartir la visión del producto final con el cliente, la mejora continua de tu proceso de desarrollo de software es posible y esencial, tener procedimientos escritos de desarrollo de software puede ayudar a crear una cultura compartida de buenas prácticas, la calidad es el principal objetivo; la productividad a largo plazo es una consecuencia de una alta calidad, haz que los errores los encuentre un colaborador y no un cliente, una clave en la calidad en el desarrollo de software es realizar iteraciones en todas las fases del desarrollo excepto en la codificación, la gestión de errores y solicitud de cambios es esencial para controlar la calidad y el mantenimiento, si mides lo que haces, puedes aprender a hacerlo mejor, haz lo que tenga sentido; no recurras a los dogmas, no puedes cambiar todo de una vez. identifica los cambios que se traduzcan en los mayores beneficios, y comienza a implementarlos.

Tema 4: Procesos del Software

El software es el conocimiento incorporado, y puesto que el conocimiento esta inicialmente disperso, el desarrollo del software implícito, latente e incompleto en gran medida, es un proceso social de aprendizaje. El proceso es un dialogo en el que se reúne el conocimiento y se incluye en el software para convertirse en software. El proceso proporciona una interacción entre los usuarios y los diseñadores, entre los usuarios y las herramientas de desarrollo, y entre los diseñadores y las herramientas de desarrollo (tecnología). Es un proceso iterativo donde la herramienta de desarrollo se usa como medio de comunicación, con cada iteración del dialogo se obtiene mayor conocimiento de las personas involucradas. Howard Baetjer

➤ **Ingeniería del Software:** una tecnología estratificada

- La ingeniería del software es el establecimiento y uso de principios robustos de la ingeniería a fin de obtener económicamente software que sea fiable y que funcione eficientemente sobre maquinas reales. El fundamento de la ingeniería del software es la capa de proceso.
- El proceso de la ingeniería del software es la unión que mantiene juntas las capas de tecnología y que permite un desarrollo racional y oportuno de la ingeniería del software.
- El proceso define un marco de trabajo para un conjunto de Áreas clave de proceso que se deben establecer para la entrega efectiva de la tecnología de la ingeniería del software. Las áreas claves del proceso forman la base del control de gestión de proyectos del software y establecen el contexto en el que se aplican los métodos técnicos, se obtienen productos del trabajo (modelos, documentos, datos, etc.) se establecen hitos, se asegura la calidad y el cambio se gestiona adecuadamente.

➤ **Métodos:** Los métodos de la ingeniería del software indican <como> construir técnicamente el software. Los métodos abarcan una gran gama de tareas que incluyen análisis de requisitos, diseño, construcción de programas, prueba y mantenimiento.

• **Tareas que componen los métodos**

- Planificación, Estimación de proyectos.
- Análisis de requerimiento de software y hardware
- Diseño de estructura de datos, Arquitectura de los programas.
- Procedimientos algorítmicos.
- Codificación, Prueba y Mantenimiento.

➤ **Herramientas:** las herramientas de la Ingeniería del Software proporcionan un enfoque automático o semiautomático para el proceso y para los métodos. Cuando se integran herramientas para que la información creada por una herramienta la puede utilizar otra, se establece un sistema de soporte para el desarrollo del software llamado **Ingeniería del Software Asistida por Computadora (CASE)**. Las Herramientas CASE son ayudas automatizadas para el desarrollo del software.

• **Las herramientas CASE se pueden clasificar de manera general en dos categorías:**

- **Herramientas de Alto Nivel (Front End):** Automatizan las fases de análisis y diseño lógico. Se pueden citar los productos comerciales, Rational, Designer 2000 de Oracle, Visible Analyst y EasyCase, Enterprise Architect, entre otros muchos.
- **Herramientas de Bajo Nivel (Back End):** Automatización de la codificación y posterior mantenimiento. Como ejemplo se pueden incluir todas las herramientas de RAD (Desarrollo Rápido de Aplicaciones), tales como Delphi o Visual Basic de Microsoft.
- **CASE-I (CASE Integrated):** herramientas que engloban todo el proceso de desarrollo software, desde análisis hasta implementación.

- Microsoft Project (Planificación)
- UML (Lenguaje Unificado de Modelado)
- RationalRase, Visio (Modelado UML)
- VisualAge Smalltalk 7.0 (AS/DS <Análisis Estructurado / Diseño Estructurado> con posibilidad de ADOO<Análisis y Diseño Orientado a Objetos>)
- Enterprise Architect (UML)
- Designer 2000
- Erwin (Base de Datos)
- MAGERIT (Seguridad)

➤ **Proceso:** Son los encargados de integrar los métodos y herramientas, además de definir la secuencia en la que se aplican los métodos, las entregas que requieren, los controles de calidad y las guías para el desarrollo.

➤ **Fases Genéricas de la Ingeniería del Software**

- **Fase de Definición:** Se centra sobre el **qué**. Es decir, durante la definición, el que desarrolla el software intenta identificar qué información ha de ser procesada, qué función y rendimiento se desea, qué comportamiento de sistema, qué interface van a ser establecidas, qué restricciones de diseño existen, y qué criterios de validación se necesitan para definir el sistema correcto.
 - Análisis del Sistema
 - Planificación del Proyecto
 - Análisis de Requisitos
- **Fase de Desarrollo:** Se centra en el **cómo**. Es decir, durante el desarrollo, un ingeniero del software intenta definir cómo han de diseñarse las estructuras de datos, cómo ha de implementarse la función dentro de una arquitectura de software, cómo han de implementarse los detalles procedimentales, cómo han de caracterizarse interfaces, cómo ha de traducirse el diseño de un lenguaje de programación (o lenguaje no procedimental) y cómo ha de realizarse la prueba.
 - Diseño del Software
 - Codificación
 - Prueba de Software
- **Fase de Mantenimiento:** Se centra en el cambio que va asociado a la corrección de errores, a las adaptaciones requeridas a medida que evoluciona el entorno del software y a cambios debidos a las mejoras producidas por los requisitos cambiantes del cliente.
 - **Corrección:** El mantenimiento correctivo cambia el software para corregir defectos que pueden ser descubiertos por el cliente o por el equipo de ingeniería.
 - **Adaptación:** El mantenimiento adaptativo produce modificaciones en el software para acomodarlo a los cambios de su entorno externo (hardware, sistema operativo, reglas de la empresa, etc.)
 - **Mejora:** EL mantenimiento perfectivo lleva al software más allá de sus requisitos originales debido a entre otras cosas, solicitudes del cliente para la adición de nuevas funciones que produzcan beneficios.
 - **Prevención o Reingeniería:** El mantenimiento preventivo hace cambios en programas de computadora a fin que se puedan corregir, adaptar y mejorar más fácilmente.

➤ **Paradigma de la Ingeniería del Software:** El modelo de proceso o paradigma de la ingeniería del software es la estrategia que comprenden métodos, herramientas y procesos. Los paradigmas intentan ordenar las actividades en el desarrollo del software. El ingeniero debe seleccionar un modelo de proceso para ingeniería de software según la naturaleza del proyecto y la aplicación, los métodos, las herramientas a utilizar, y los controles y entregas que se requieren.

➤ **Metodologías:** entre sus funciones básicas podemos mencionar, la definición del ciclo de vida más adecuado a las condiciones y características del desarrollo, determinar las fases dentro del ciclo de vida especificando su orden de ejecución, definir los resultados intermedios y finales y proporcionar un conjunto de métodos, de herramientas y técnicas para facilitar la tarea del ingeniero del software y aumentar su productividad.

- **Desde el punto de vista de la Gestión:** facilita la tarea de planificación, de control y seguimiento de un proyecto, mejora la relación coste/beneficio, optimiza el uso de recursos, facilita la evaluación de resultados y cumplimiento de los objetivos, mejora la comunicación cliente desarrollador y ayuda a la gestión del proyecto.
 - **Desde el punto de vista del Ingeniero:** ayuda a la comprensión del problema, optimiza el conjunto y cada una de las fases del proceso de desarrollo, facilita el mantenimiento del producto final y permite la reutilización de partes del producto.
 - **Desde el punto de vista del Usuario:** Garantiza un determinado nivel de calidad en el producto final y genera confianza en los plazos de tiempos fijados en la definición del proyecto.
- **Ciclo de Vida:** Conjunto de fases por las que pasa el sistema que se está desarrollando desde que nace la idea inicial hasta que el software es retirado o reemplazado ("muere"). Tiene dos puntos de vista, el de la transformación del producto y el del proceso que transforma el producto.
- **Modelo en cascada o lineal secuencial (metodología):** También llamado "Ciclo de vida básico" o "Modelo de cascada" tiene su origen en el "Modelo de cascada" ingeniado por Winston Royce, aunque omite los muchos bucles de este último. El Modelo Lineal Secuencial sugiere un enfoque sistemático o más bien secuencial del desarrollo de software que comienza en un nivel de sistemas y progresa con el análisis, diseño, codificación, pruebas y mantenimiento.
- **Análisis:** Es la fase en la cual se reúnen todos los requisitos que debe cumplir el software. En esta etapa es fundamental la presencia del cliente que documenta y repasa dichos requisitos.
 - **Diseño:** Es una etapa dirigida hacia la estructura de datos, la arquitectura del software, las representaciones de la interfaz y el detalle procedimental (algoritmo). En forma general se hace un esbozo de lo solicitado y se documenta haciéndose parte del software.
 - **Generación del código:** Es la etapa en la cual se traduce el diseño para que sea comprensible por la máquina. Esta etapa va a depender estrechamente de lo detallado del diseño.
 - **Pruebas:** Esta etapa se centra en los procesos lógicos internos del software, asegurando que todas las sentencias se han comprobado, y en la detección de errores.
 - **Mantenimiento:** Debido a que el programa puede tener errores, puede no ser del completo agrado del cliente o puede necesitar, eventualmente acoplarse a los cambios en su entorno. Esto quiere decir que no se rehace el programa, sino que sobre la base de uno ya existente se realizan algunos cambios.
- **Problemas del Modelo:** raramente los proyectos siguen este ciclo de vida, el cliente pocas veces establece todos los requerimientos al principio, el cliente no tiene un producto terminado hasta el final, el tiempo de desarrollo es muy largo.
- **Modelo de Prototipado (metodología):** El prototipo debe ser construido en poco tiempo, usando los programas adecuados y no se debe utilizar muchos recursos. El diseño rápido se centra en una representación de aquellos aspectos del software que serán visibles para el cliente o el usuario final. Este diseño conduce a la construcción de un prototipo, el cual es evaluado por el cliente para una retroalimentación; gracias a ésta se refinan los requisitos del software que se desarrollará. La interacción ocurre cuando el prototipo se ajusta para satisfacer las necesidades del cliente. Esto permite que al mismo tiempo el desarrollador entienda mejor lo que se debe hacer y el cliente vea resultados a corto plazo.
- **Fases dentro del ciclo de vida del prototipado**
 - Fase de Recolección de Requisitos
 - Fase de Diseño Rápido
 - Fase de Construcción del Prototipo
 - Fase de Evaluación del Prototipo
 - Refinamiento Sucesivo
 - Terminación

- **Uso del Modelo:** es utilizado cuando el cliente no puede especificar todos los requerimientos al principio, facilita un modelo al programador.
 - **Tipos de Prototipos:** Totales, Parciales, Interfaces, Modelos y Estructuras de datos.
 - **Problemas del Modelo:** El cliente suele utilizar el prototipo aunque no es un producto de software terminado, los módulos ineficientes presentados como prototipos suelen convertirse en partes del sistema.
- **Modelo DRA – Desarrollo Rápido de Aplicaciones (metodología):** consiste en un desarrollo rápido de aplicaciones basado en el modelo lineal secuencial, pero donde se enfatiza un ciclo de desarrollo extremadamente corto. Es una adaptación a alta velocidad del modelo lineal secuencial, donde se puede aumentar la velocidad haciendo uso de componentes. Si se comprende bien los requisitos y se limita el ámbito del proyecto, el proceso DRA permite al equipo de desarrollo crear un sistema completamente funcional, dentro de periodos cortos de tiempo.
- **Problemas del modelo DRA:** para proyectos grandes se necesitan recursos suficientes para formar los equipos necesarios, no todas las aplicaciones son susceptibles de aplicar este modelo, cuando el grado de interoperabilidad con programas ya existentes es alto, no es apropiado.
- **Modelos Evolutivos:** los modelos evolutivos se caracterizan porque permiten a los ingenieros del software, desarrollar de manera interactiva, nuevas versiones del software cada vez más completas.
- **Desarrollo Evolutivo:** Existen dos tipos de desarrollo evolutivo:
- **Desarrollo Exploratorio:** el objetivo es trabajar con el cliente para explorar sus requerimientos y entregar un sistema final. El desarrollo empieza por las partes del sistema que se comprenden mejor. El sistema evoluciona agregando nuevos atributos propuestos por el cliente.
 - **Prototipos Desechables:** el prototipo se centra en experimentar con los requerimientos del cliente que no se comprenden del todo para establecer una definición mejorada de los requerimientos del sistema.
 - **Problemas:** el proceso no es visible. Los administradores tienen que hacer entregas regulares para medir el progreso. A menudo los sistemas tienen una estructura deficiente, los cambios continuos tienden a corromper la estructura del software, incorporar cambios en él se convierte cada vez más en una tarea difícil y costosa.
- **Modelo Evolutivo – Incremental:** El modelo Incremental combina elementos del modelo lineal secuencial (aplicados repetidamente) con la filosofía interactiva de construcción de prototipos.
- **Modelo Evolutivo – Espiral:** propuesto originalmente por Boehm, es un modelo de proceso de software evolutivo que conjuga la naturaleza iterativa de construcción de prototipos con los aspectos controlados y sistemáticos del modelo lineal secuencial. Ideal para realizar versiones incrementales de manera rápida, el software se desarrolla en una serie de versiones incrementales.
- **Modelo Evolutivo – Espiral Victoria-Victoria:** al modelo ya mencionado, se agrega la funcionalidad de comunicación con el cliente: en la comunicación ideal, el desarrollador pregunta al cliente y este le facilita suficiente información para continuar, en la comunicación real, el cliente y el desarrollador, entran en un proceso de negociación, donde el cliente puede ser preguntado para sopesar la funcionalidad, rendimiento y otras características. Con este modelo se esfuerzan en obtener un beneficio mutuo **Victoria – Victoria**. Dicho modelo fue definido por Boehm como un conjunto de actividades de negociación al principio de cada paso alrededor de la espiral.
- **Modelo Evolutivo – Desarrollo Concurrente:** dado por Davis y Sitaram, se puede representar en forma de esquema como una serie de actividades técnicas importantes, tareas y estados asociadas a ellas. El modelo define una serie de acontecimientos que dispararan transiciones de estado a estado para cada una de las actividades de la ingeniería del software. Este modelo se utiliza a menudo como el paradigma de desarrollo de aplicaciones cliente/servidor.

- **Modelo Basado en Componentes:** la tecnología de objetos proporciona el marco de trabajo técnico para un modelo de proceso basado en componentes de Ingeniería de Software. El paradigma orientado a objetos enfatiza en la creación de clases que encapsulan tanto los datos como los algoritmos que se utilizan para manejar los datos. El modelo enfatiza la reutilización de componentes. Es evolutivo por naturaleza y exige un enfoque iterativo para la creación de software. Se configuran las aplicaciones desde componentes preparados de software. La reutilización según estudios establece que se reduce el ciclo de vida en un 70%, reduce el costo del proyecto en un 84% y aumenta la productividad.
- **Modelo de Métodos Formales:** comprende el conjunto de actividades que conducen a la especificación matemática del software de computadora, permite que un ingeniero especifique, desarrolle y verifique un sistema aplicando una notación rigurosa y matemática. Con este modelo se consigue software libre de errores, es de difícil aplicación, pero para software de alta seguridad es muy adecuado. Se lo conoce como Ingeniería de Software de Sala Limpia.
- **Modelos Ágiles / Programación Extrema:** El desarrollo ágil de software refiere a métodos de ingeniería del software basados en el desarrollo iterativo e incremental, donde los requisitos y soluciones evolucionan mediante la colaboración de grupos auto-organizados y multidisciplinarios. Existen muchos métodos de desarrollo ágil; la mayoría minimiza riesgos desarrollando software en lapsos cortos. El software desarrollado en una unidad de tiempo es llamado una iteración, la cual debe durar de una a cuatro semanas. Cada iteración del ciclo de vida incluye: planificación, análisis de requisitos, diseño, codificación, revisión y documentación. Una iteración no debe agregar demasiada funcionalidad para justificar el lanzamiento del producto al mercado, sino que la meta es tener una «demo» (sin errores) al final de cada iteración. Al final de cada iteración el equipo vuelve a evaluar las prioridades del proyecto. Los métodos ágiles enfatizan las comunicaciones cara a cara en vez de la documentación. Los métodos ágiles también enfatizan que el software funcional es la primera medida del progreso. Combinado con la preferencia por las comunicaciones cara a cara. Los métodos similares al ágil son: Scrum (1986), Crystal Clear (cristal transparente), programación extrema (en inglés eXtreme Programming o XP, 1996), desarrollo de software adaptativo, feature driven development, Método de desarrollo de sistemas dinámicos (en inglés Dynamic Systems Development Method o DSDM, 1995). Kent Beck creó el método de Programación Extrema (usualmente conocida como XP) en 1996.
- **Proceso Unificado de Desarrollo del Software:** es un proceso basado en componentes que ha sido propuesto por la industria, dispone de un Lenguaje de Modelado Unificado (UML), que es utilizado para construir el sistema y las interfaces que conectarán los componentes. Combina un desarrollo incremental e iterativo, definiendo la función del sistema aplicando un enfoque basado en escenarios. Se puede decir que es un proceso:
 - **Dirigido por los casos de uso:** basándose en el modelo de casos de uso y en su análisis los desarrolladores crean los modelos de diseño e implementación que lleven a cabo los casos de uso.
 - **Centrado en la arquitectura:** la arquitectura de un sistema de software se describe mediante diferentes vistas del sistema de construcción
 - **Iterativo e Incremental:** el desarrollo de un proyecto se divide en iteraciones, encargadas de pequeñas partes del trabajo y que dan como resultado un crecimiento del producto, incremento. Estas iteraciones se planifican y se prueban cada vez que terminan.
- **Métrica V3** es una metodología de planificación, desarrollo y mantenimiento de sistemas de información. Es un instrumento para la sistematización de las actividades que dan soporte al ciclo de vida del software. Contempla el desarrollo de sistemas de información para las distintas tecnologías que actualmente conviven. En la elaboración de Métrica 3 se han tenido en cuenta los métodos de desarrollo más extendidos, así como los últimos estándares de ingeniería del software y calidad, además de referencias específicas en cuanto a seguridad y gestión de proyectos. Cubre el desarrollo estructurado y orientado a objetos. Facilita los procesos de apoyo y de organización a través de interfaces: Gestión de proyectos; Gestión de configuración; Aseguramiento de calidad y seguridad. La automatización de sus actividades es posible ya que existe una amplia variedad de herramientas de ayuda al desarrollo. Existe una herramienta que adapta Métrica 3 a las condiciones específicas de cada proyecto, permitiendo el control y seguimiento desde diferentes perfiles.

- **Metodología DUM** es una metodología evolutiva e incremental de desarrollo del software que ha sido creada en el departamento de Lenguajes y Ciencias de la Computación de la Universidad de Málaga. Basada en un enfoque iterativo incremental esta metodología realiza una especificación exhaustiva de todas las actividades y tareas que se realizan en las diferentes fases, prestando especial atención por alcanzar un nivel superior de madurez según el marco CMMI/Carnegie Mellon. Las fases en las que se subdivide la metodología DUM son las siguientes: fase preliminar, fase de Inicio, fase de elaboración, fase de construcción, fase de transición; y finalmente, una fase de mantenimiento.
- **Fase Preliminar:** En esta fase el cliente proporciona los dos elementos básicos para comenzar un proyecto: una petición formal del mismo; y una definición del problema al que debe dar respuesta el Sistema a desarrollar. En base a estos dos elementos la organización de desarrollo definirá un primer equipo encargado del inicio del proyecto.
 - **Fase de Inicio:** En la Fase de Inicio se determina si el problema planteado tiene solución, lo cual se hace desde un punto de vista genérico. Si al final de la Fase de Inicio se determina que el problema tiene solución, DUM denominará el Sistema a desarrollar un Sistema Realizable.
 - **Fase de Elaboración:** En la Fase de Elaboración se determina si es posible desarrollar el Sistema teniendo en cuenta las restricciones impuestas por el cliente, y se obtiene un proyecto particular después de aplicarle las restricciones del cliente al proyecto genérico.
 - **Fase de Construcción:** Terminada la Fase de Elaboración se cuenta con una especificación detallada de qué debe hacer el Sistema, de una arquitectura formada por los elementos del Sistema que guiará cómo cumplirá el Sistema con su cometido, y de una versión inicial del Sistema en base a su arquitectura que será la base sobre la que se construirá el resto del Sistema.
 - **Fase de Transición:** Durante la Fase de Transición se realiza la prueba del Sistema con el fin de adaptar el mismo a un entorno de producción realizándose las modificaciones que se estimen necesarias. A este respecto será necesario desarrollar la estructura necesaria para facilitar en la medida de lo posible dicho proceso.
 - **Fase de Mantenimiento:** Tras la finalización del proyecto será necesario establecer un acuerdo respecto al mantenimiento, que puede ser llevado a cabo por la misma organización de desarrollo o por otra distinta.

Tema 5: Ingeniería de Requerimientos

- **¿Qué es requisito?:** "... es una condición o capacidad a la que el sistema (siendo construido) debe conformar. puede ser definido como: Una capacidad del software necesaria por el usuario para resolver un problema o alcanzar un objetivo... Una capacidad del software que debe ser reunida o poseída por un sistema o componente del sistema para satisfacer un contrato, especificación, estándar, u otra documentación formal". [ZULOAGA96]
- **Requerimientos:** En la ingeniería de sistemas y la ingeniería de software, la Ingeniería de requisitos o Ingeniería de requerimientos comprende todas las tareas relacionadas con la determinación de las necesidades o de las condiciones a satisfacer para un software nuevo o modificado, tomando en cuenta los diversos requisitos de las partes interesadas, que pueden entrar en conflicto entre ellos. El propósito de la ingeniería de requerimientos es hacer que los mismos alcancen un estado óptimo antes de alcanzar la fase de diseño en el proyecto. Los buenos requisitos deben ser medibles, comprobables, sin ambigüedades o contradicciones, etc.
 - **Administración de los Requerimientos:** "... un enfoque sistemático para obtener, organizar y documentar los requisitos del sistema, así como mantener un acuerdo entre el cliente y el equipo del proyecto en los cambios a los requisitos. Debemos considerar que la clave para una efectiva administración de requisitos es: Mantener un enunciado claro de los requisitos, junto con atributos para cada tipo de requisitos y su seguimiento con otros requisitos o elementos del proyecto." [TORRES]
 - **Requerimientos de software:** Se refiere a lo que debe hacer y lo que no debe hacer el sistema, se pueden subdividir en requerimientos funcionales y requerimientos no funcionales.
 - **Requerimientos de hardware:** Dependen del volumen de operaciones, cantidad de usuarios y de las aplicaciones que deben de estar en uso cuando se implemente el sistema. Se pueden distinguir tipos de requerimientos de hardware, requerimientos de rendimiento, de interfaz, de ambiente.
 - **Requerimientos del Usuario:** los requerimientos del usuario son declaraciones, en lenguaje formal y en diagramas, de los servicios que espera que el sistema proporcione y de las restricciones bajo las cuales debe funcionar. Deben describirse los requerimientos funcionales y no funcionales de tal forma que sean comprensibles por los usuarios del sistema sin conocimiento técnico detallado. Únicamente deben especificar el comportamiento externo del sistema y deben evitar, tanto como sea, las características de diseño del sistema. Posibles problemas al redactar un documento en lenguaje formal: Falta de claridad, confusión entre los requerimientos funcionales, no funcionales, las metas e información de diseño y establecimiento separado de requerimientos que pueden ser resumidos en un solo de requerimientos.
 - **Requerimientos del Sistema:** establecen con detalle las funciones, servicios y restricciones operativas del sistema, son versiones extendidas de los requerimientos del usuario que son utilizados por los ingenieros de software como punto de partida para el diseño del sistema, agregan detalle y explican como el sistema debe proporcionar los requerimientos del usuario. Deben describir el comportamiento externo del sistema y sus restricciones operativas. El documento de requerimientos del sistema (algunas veces denominado especificación funcional) debe ser preciso. Debe definir, exactamente qué es lo que se va a implementar. Puede ser parte del contrato entre el comprador del sistema y los desarrolladores del software.
 - **Notaciones para la especificación de requerimientos:**
 - **Lenguaje natural estructurado:** este enfoque depende de la definición de formularios o plantillas estándares para expresar la especificación de requerimientos.
 - **Lenguajes de descripción de diseño:** ese enfoque utiliza un lenguaje similar a uno de programación, pero con características más abstractas, para especificar los requerimientos por medio de la definición de un modelo operativo del sistema. Este enfoque no se utiliza ampliamente en la actualidad, aunque puede ser útil para especificar interfaces.
 - **Notaciones gráficas:** para definir los requerimientos funcionales del sistema, se utiliza un lenguaje gráfico, complementando con anotaciones de texto. Uno de los primeros lenguajes gráficos fue el SADT (Ross, 1977) (Schoman y Ross, 1977). Actualmente, se suelen utilizar las descripciones para casos de uso (Jacobson, 1993) y los diagramas de secuencia (Stevens y Pooley, 1999)
 - **Requerimientos Funcionales:** son declaraciones de los servicios que debe proporcionar el sistema, de la manera en que este debe reaccionar a entradas particulares y de cómo se debe comportar en situaciones particulares, estos requerimientos dependen del tipo de software que se desarrolle, tomado por la

organización al redactar requerimientos. Para un sistema de software se pueden expresar de diferentes formas. En algunos casos, se puede especificar explícitamente lo que el sistema no debe hacer.

- **Requerimientos No Funcionales:** son restricciones de los servicios o funciones ofrecidas por el sistema. Incluyen restricciones de tiempo, sobre el proceso de desarrollo y estándares. Los requerimientos no funcionales a menudo se aplican al sistema en su totalidad. Normalmente apenas se aplican a características o servicios individuales del sistema. Pueden especificar el rendimiento del sistema, la protección, la disponibilidad, capacidades de dispositivos de entrada y salida, representaciones de datos y otras propiedades emergentes. Esto significa que a menudo son más críticos que los requerimientos funcionales particulares. También pueden ser utilizados para restringir el proceso de desarrollo de un sistema. Todo esto surge por las necesidades del usuario, debido a las restricciones económicas, las políticas de la organización, la necesidad de interoperabilidad entre sistemas, entre otros.
 - **Requerimientos del Dominio:** Se derivan del dominio de aplicación del sistema más que de las necesidades específicas del usuario, normalmente incluyen terminología especializada del dominio o referencias a conceptos de dominio, pueden ser requerimientos funcionales nuevos, restringir los existentes o establecer como se deben ejecutar cálculos particulares. Los requerimientos del dominio son importantes debido a que a menudo reflejan los fundamentos del dominio de aplicación. Si estos requerimientos no se satisfacen, es imposible hacer que el sistema trabaje de forma satisfactoria.
 - **Especificación de la Interfaz:** Casi todos los sistemas software deben funcionar con otros sistemas que ya están implementados e instalados en el entorno, esto implica que las interfaces de los sistemas implementados deben especificarse de forma precisa para que los sistemas nuevos puedan interactuar sin problemas, estas especificaciones deben definirse al principio del proceso e incluirse en el documento de requerimientos. Existen tres tipos de interfaces que pueden definirse:
 - Interfaces de procedimientos en las cuales los programas o subsistemas existentes ofrecen una variedad de servicios a los que se accede invocando a los procedimientos de la interfaz (APIs)
 - Estructuras de datos que pasan de un subsistema a otro.
 - Representaciones de datos (como el orden de los bits) establecidas para un subsistema existente. Estas interfaces son muy comunes en sistemas en tiempo real embebido.
- **El documento de requerimientos del software:** algunas veces denominado especificación de requerimientos de software o SRS, es la declaración oficial de que deben implementar los desarrolladores del sistema, debe incluir tanto los requerimientos del usuario para el sistema como una especificación detallada de los requerimientos del sistema. El documento tiene un conjunto de usuarios que lo utilizan, la diversidad de posibles usuarios significa que el documento de requerimientos tiene que presentar un equilibrio entre la comunicación de los requerimientos a los clientes, la definición de los requerimientos en el detalle exacto para los desarrolladores y probadores, y la inclusión de información sobre la posible evaluación del sistema. El nivel de detalle que se debe incluir en un documento de requerimientos depende del tipo de sistema que se desarrolle y el proceso de desarrollo utilizado.
- **El Proceso de Elicir Requisitos:** Etimológicamente significa “Sacar algo de algo”. El proceso de elicir requisitos puede descomponerse en las actividades de: hallazgo, de hechos, reunión e integración de la información. Un procedimiento para obtener la información consta de los siguientes pasos:
- Identificación de las fuentes relevantes e los requisitos
 - Preguntar cuestiones apropiadas para obtener y comprender sus necesidades
 - Analizar la información reunida buscando implicaciones, inconsistencias o aspectos no resueltos
 - Confirmar la comprensión de los requisitos con los usuarios
 - Sintetizar las especificaciones adecuadas para los requisitos
- **Técnicas de Educación:** Han evolucionado desde el procedimiento general:
- Definiendo procesos detallados
 - Proponiendo preguntas o categorías de preguntas
 - Estableciendo formatos estructurados para las reuniones

- Describiendo comportamiento ideal, individuales o de grupo.
- Construyendo patrones para organizar y registrar la información
- El resultado tangible de educir requisitos es un conjunto de requisito que puede ser utilizado por el equipo de desarrollo de software

➤ **Problemas de la Educción de Requisitos**

- **Problemas de Alcance:** las técnicas de educación tienen que ser suficientemente amplias como para establecer las condiciones de los límites del sistema objetivo. Los contextos que afectan al proceso de ingeniería de requisitos son:
 - **Organización:** los requisitos de la organización proporcionan una comprensión de la propia organización y del papel que representa el sistema en el contexto de la organización. Los factores implicados son las presentaciones de las entradas al sistema, los usuarios de las salidas del sistema y las formas que el sistema cambia la forma de trabajar en la organización.
 - **Entorno:** para educir requisitos del usuario es necesario disponer de una descripción precisa de los usuarios y de su entorno. Los factores de entorno incluyen las restricciones de hardware y software, madurez del dominio, interfaces y el papel del sistema dentro de otro sistema más grande.
 - **Proyecto:** los factores del contexto del proyecto contienen: atributos de los diferentes grupos, tales como usuarios, patrocinadores, desarrolladores, estilo, jerarquía de grupo como así también las restricciones impuestas por las personas implicadas, por ejemplo, la dirección puede imponer restricciones de tiempo, coste, etc.
- **Problemas de Comprensión:** como consecuencia de una pobre comunicación entre usuario y analista, los requisitos son ambiguos, incompletos, inconsistentes e incorrectos. Los aspectos abarcados son los siguientes:
 - Las comunidades implicadas en la educación (patrocinadores, clientes, desarrolladores, analistas, etc.) tienen distintas formaciones y experiencias.
 - El lenguaje usado para expresar requisitos puede ser demasiado formal o informal para reunir las necesidades de cada uno de los grupos. Inicialmente, se usa lenguaje natural lo que provoca ambigüedad y mala interpretación.
 - La gran cantidad de información que se educa tiene que organizarse de alguna forma.
- **Problemas de Volatilidad:** Los requisitos evolucionan con el tiempo. Las necesidades del usuario durante el desarrollo de un sistema pueden madurar a causa del conocimiento adicional fruto del desarrollo o de presiones del entorno o de la organización que no son previstas. Si no se incorporan estos cambios, los requisitos originales están incompletos e inconsistentes. Las causas más conocidas son: las necesidades de los usuarios evolucionan, los requisitos son resultado de muchas contribuciones con distintas necesidades y objetivos y la complejidad de la organización.

➤ **Técnicas de Recogida:** Las técnicas más comunes son: **Entrevistas, Desarrollo Conjunto de Aplicaciones (JAD), Prototipado, Tormenta de Ideas, Observación, Estudio de Documentación, Cuestionarios, Despliegue de la Función Calidad (DFC).** Estas técnicas se denominan Técnicas para Facilitar las Especificaciones de la aplicación (TFEA) (Facilitated Application Specification Techniques)

➤ **Técnicas basadas en la actividad del Analista:** preguntar, observar e inferir, discutir y formular, negociar respecto del conjunto de estándares vigentes, estudiar e identificar el problema, descubrir a través de proceso creativos y postular una solución.

➤ **El muestreo:** es el proceso por el cual se seleccionan de manera sistemática elementos representativos de una población: Definir con precisión los datos que se van a recopilar o a describir, delimitar la población sujeta a selección de muestras, elegir el tipo de muestra y decidir el tamaño de la muestra.

➤ **Tipos de información que se obtienen durante la investigación**

- **Análisis de datos cuantitativos:** la información obtenida es a través de informes corporativos, informes de desempeño, registros y formas para la captura de datos.

- **Análisis de documentos cualitativos:** se obtiene la información a través de memorándum, avisos pegados en tableros y áreas de trabajo, manuales de procedimientos y de políticas de la empresa.
- **La Entrevista:** es una conversación dirigida con un propósito específico, que se basa en un formato de preguntas y respuestas. La Planeación de la Entrevista contiene, lectura de antecedentes, establecimientos de los objetivos de la entrevista, la selección de los entrevistados, la preparación del entrevistado y la selección de tipo y estructura de las preguntas.
- **Preguntas Abiertas:** ofrecen libertad de expresión y simplifican las cosas al entrevistado, permiten al entrevistador utilizar el vocabulario del entrevistado, proporcionan una riqueza de detalle, revela nuevas alternativas sobre preguntas no consideradas, hace más interesante la entrevista, entre otras cosas, pero posee ciertas desventajas como la generación de información irrelevante, la posible pérdida del control de la entrevista, respuestas con demasiado tiempo alejándose del enfoque principal y puede dar la impresión que el entrevistador no se ha preparado correctamente para entrevistar.
 - **Preguntas Cerradas:** Permiten solo dos tipos respuestas, fundamentalmente Si y No, tiene como ventaja una respuesta directa, ahorra tiempo, permite mantener el control de la entrevista, cubre rápidamente varios aspectos y obtiene datos de relevancia, pero, por otro lado puede tornar aburrida la encuesta, perder la riqueza de los detalles y de las ideas centrales generando un clima tenso entre el entrevistado y el entrevistador.
 - **Preguntas de Sondeo:** El propósito del sondeo es ir más allá de la respuesta inicial para obtener un mayor significado, y aclarar o ampliar los puntos del entrevistado. El sondeo se puede realizar mediante preguntas cerradas o abiertas. La mayoría de los entrevistadores principiantes son reticentes acerca del sondeo, y en consecuencia, aceptan respuestas superficiales. Si se hace de una manera sistemática y determinada, le servirá para mostrar a su interlocutor que escucha lo que dice, pensando al respecto y respondiendo adecuadamente.
 - **Errores en las preguntas:** las preguntas tendenciosas tienden a dirigir al entrevistado hacia la respuesta que Ud. quisiera escuchar. Las preguntas dobles son aquellas que en una sola contienen, de hecho, dos preguntas diferentes. Normalmente no deben utilizarse por que se corre el riesgo que el entrevistado solo conteste una de ellas. El orden de las preguntas en una secuencia lógica es Inductivo (piramidal), Deductivo (embudo) y combinación de ambos (diamante)
- **Realización de la Entrevista**
- **Cuestionarios:** No debe profundizarse una pregunta y/o analizar términos difíciles, esto implica para el analista que las preguntas deben ser completamente transparentes, el flujo del cuestionario convincente, se debe anticipar a las respuestas de las preguntas, y que el manejo del cuestionario sea planificado con todo detalle. Deberá contarse con preguntas abiertas y cerradas, establecer una redacción adecuada, seleccionar un vocabulario acorde al utilizado por la organización, realizar redacciones sencillas y preguntas cortas.
 - **Uso de Escalas en los Cuestionarios:** escala es el proceso de asignar números u otros símbolos a un atributo o características con el fin de poder medirlo. Con frecuencias las escalas son arbitrarias y no llegan a ser únicas.
 - **Elaboración de Escalas:**
 - **Escalas arbitrarias:** mide lo que se intenta medir.
 - **Escalas por consenso:** involucra a un grupo de jueces, quienes ayudan a evaluar los tópicos que el analista eligió para el cuestionario.
 - **Análisis de temas:** involucra la evaluación de los tópicos por un pequeño grupo de quienes responderán, similar al grupo que recibirá el cuestionario definitivo. Cada tópico se analiza en función de su conveniencia.
 - **Factorización:** es el procedimiento estadístico por medio del cual se agrupan objetos similares.
 - **Parámetros de Desempeño:** la **Validez** es el grado con el que la pregunta determina lo que el analista intenta medir. La **Confiabilidad** se refiere al grado en que su aplicación repetida al mismo sujeto u objeto produce iguales resultados.
 - **Diseño y Aplicación de Cuestionarios:** un buen cuestionario debe tener un formato acorde, disponer de suficiente espacio en blanco para las respuestas, utilizar círculos en las respuestas, tomar los objetivos como lineación y mantener un estilo consistente, un orden jerárquico de las preguntas, las más importantes primero agrupando por tema y tendencias asociativas. La decisión de a quien se dará el cuestionario se toma al

momento de establecer los objetivos, se deberá reunir a todas las personas en un mismo lugar, entregar personalmente el cuestionario en blanco y recogerlos luego, enviar por correo los cuestionarios a los empleados de sucursales remotas estableciendo fechas límites e instrucciones.

- **Uso de Arreglos Q:** esta metodología consiste en la estructuración de un arreglo-Q, el cual fuerza a que las respuestas se apeguen a una distribución normal, que es adecuada para agrupar a los que responden, con base en sus opiniones sobre un tópico particular. A partir de las entrevistas, las observaciones y el muestreo de datos de archivos, el analista crea una serie de tarjetas con conceptos para que los que responden las ordenen. Cada tarjeta contiene por separado un concepto; en teoría, al agrupar las tarjetas se tiene todo el dominio posible de actitudes sobre el tópico en cuestión.
- **Brainstorming (Lluvia de Ideas):** es una técnica de grupo sencilla para la generación de ideas, una sesión de Brainstorming funciona bien con 4 a 10 personas, una persona es el líder, pero el papel del líder se reduce a, prácticamente, abrir la reunión más que a restringirla o controlarla. Es útil en la generación de múltiples puntos de vistas del problema y la formulación del problema en formas diferentes, ayuda en la estimulación de la imaginación contribuyendo a que los usuarios sean conscientes de sus necesidades, a construir una más completa figura de las necesidades del usuario, evita la tendencia de centrarse en áreas estrechas demasiado pronto, es una técnica fácil de aprender.
 - **Fase de preparación:** identificación de participantes, clientes, usuarios e ingenieros de software, designación de un líder, planificación de la sesión con los participantes y preparación de la sala.
 - **Fase de generación:** la sesión es iniciada por el líder, que expresa una instrucción general del problema. Los participantes expresan libremente, un turno o espontáneamente, dependiendo del criterio del líder. No hay críticas sobre las ideas, se fomentan las ideas no convencionales, así como embellecer o combinar las ideas de otros. Las ideas deben estar visibles durante la sesión.
 - **Fase de consolidación:** se revisan las ideas para clasificarlas, se descartan las ideas que no sean utilizables, se discuten las ideas restantes con el objetivo de ordenarlas imponiendo prioridades, se registran todas esas ideas, con sus propiedades u otros comentarios.
- **Entorno PIECES:** El objetivo que persigue este modelo es asegurar que tanto el analista de sistemas como los usuarios se involucren en el análisis de cada una de las categorías esenciales en relación con el dominio del problema y que las respuestas contribuyan significativamente a la definición de los requerimientos del sistema. En este sentido, las categorías de clasificación PIECES se describen a continuación:
 - **Prestaciones:** o rendimiento abordan temas relacionados con el desempeño del sistema ante los usuarios. Aquí se consideran asuntos de rendimiento en términos de la cantidad de trabajo realizado sobre un cierto período de tiempo y el tiempo promedio de respuesta obtenido ante la solicitud o transacción iniciada por el usuario.
 - **Información:** aborda aspectos íntimamente vinculados con el proceso de captura, procesamiento, presentación y almacenamiento de los datos en el sistema.
 - **Economía:** se ocupa de la evolución del proyecto en términos de costos operacionales así como de cualquier otro elemento que pueda vincularse o impactar económicamente el sistema.
 - **Control:** está estrechamente relacionado con cuestiones de seguridad del sistema, así como en la redacción requerida en los datos entrantes.
 - **Eficiencia:** se ocupa de evaluar el grado de certeza con la que se realizan las cosas. El impacto de la eficiencia por lo general se mide en uno de tres niveles: Corporativo, departamental o individual.
 - **Servicio:** aborda aspectos funcionales del sistema así como otros requerimientos relacionados con su implementación, mantenimiento, adiestramiento y documentación.
- **Análisis de Mercado:** es una actividad común de la mayoría de empresas que venden productos. Las empresas necesitan conocer el mercado antes de construir el producto. Hay diversos aspectos en el análisis de mercado:
 - **El análisis de mercado:** observa cuidadosamente los productos similares de los competidores, se identifica lo bueno a copiar y lo malo a evitar.
 - **La investigación de mercado:** implica recoger datos estadísticos sobre los productos comprados y se identifican las tendencias en esos datos para predecir las necesidades de futuros productos.

- **Cuestionarios de clientes:** diseñados cuidadosamente por expertos, pueden educir información muy detallada de las necesidades de compradores y usuarios potenciales de un paquete de software.
- **Análisis de Factores Críticos:** esta aproximación de educción consiste en identificar y concentrarse en un pequeño conjunto de factores críticos de los que depende la efectividad del sistema. Comprende los siguientes pasos: Comprender la operación del sistema, identificar los factores críticos para que el sistema sea efectivo, identificar la fuerza y la debilidad del sistema respecto de cada uno de estos factores, identificar áreas de problemas y oportunidades, reunir los detalles relevantes que mejoren el rendimiento y formular requisitos usando estos detalles.
- **JAD (Desarrollo de Aplicaciones Conjunta):** promueve la cooperación, comprensión y equipo de trabajo entre compradores, usuarios y desarrolladores. Proporciona un proceso que facilita la creación de una visión compartida de lo que el sistema debería ser.
- **Principios en los que se apoya el JAD:**
 - **Dinámica de Grupo:** en la que se enriquecen, complementan e integran las visiones parciales que tienen los participantes en el proceso de analizar el problema.
 - **Uso:** utilización de ayudas visuales para mejorar la comunicación y la comprensión.
 - **Mantenimiento:** de un proceso de organización racional, en el que los papeles de cada uno de los participantes están perfectamente definidos.
 - **Documentación:** Filosofía de Documentación “lo que ves es lo que obtienes” con formularios de documentos estándares que se rellenan y se endosan por todos los participantes de una sesión.
- **Fases de un plan JAD:** Definición del Proyecto, Investigación, Preparación, La Sesión JAD y el Documento Final.
- **Prototipos:** (Bernard Boar, James Martin – Desarrollo heurístico): Consiste en capturar un conjunto de necesidades e implementarlas rápidamente con la intención declarada de expandirlas y refinarlas iterativamente al ir aumentando la comprensión que del sistema tienen el usuarios y quien lo desarrolla.
- **Enfoques para el desarrollo de los prototipos**
 - **Prototipos de remiendo:** tiene que ver con la construcción de un sistema que si bien funciona, se encuentra remendado o parchado.
 - **Modelo a escala no funcional:** se construye a escala, con el objeto de evaluar ciertos aspectos de diseño.
 - Primer modelo a escala completa: comúnmente llamado piloto
 - **Características esenciales:** incluye algunas, pero no todas las características que tendrá el sistema final.
- **Desarrollo de Prototipos:**
 - **Trabajar con módulos manipulables:** permiten relacionarnos con las características y la construcción es independiente de otros módulos del sistema.
 - **Construir con rapidez:** una vez obtenidos los requerimientos, se construyen los módulos funcionales del prototipo
 - **Modificaciones en el Prototipo:** los módulos deben tener baja dependencia para que sean fácilmente modificables.
 - **Enfatizar la interfaz del usuario:** lo que se trata de obtener es que el usuario plantee más allá de sus requerimientos de información.
- **Ventajas:** modificaciones del sistema en etapas tempranas del desarrollo, oportunidad de detener el desarrollo de un sistema que no sirve y la posibilidad de desarrollar un sistema que se ajuste a las necesidades y expectativas del usuario.
- **Desventajas:** administración difícil como un proyecto, se lo considera un sistema, tanto por los usuarios como los analistas.
- **Despliegue de la Función de Calidad (DFC):** es una técnica de gestión de calidad que traduce las necesidades del cliente en requisitos técnicos del software, fue desarrollada en Japón y aplicada en la industria Mitsubishi, hace énfasis en entender lo que resulta importante para el cliente. Identifica tres tipos de requisitos:

- **Requisitos Normales:** se declaran los objetivos y metas en una reunión con el cliente. Si estos requisitos están presentes el cliente está satisfecho.
 - **Requisitos Esperados:** estos son implícitos al sistema, y el cliente no siempre los declara.
 - **Requisitos Innovadores:** son características que van más allá de las expectativas del cliente y suelen ser muy satisfactorias.
- **Escenarios:** Un escenario es un retrato significativo y detallado de un plausible mundo futuro, suficientemente gráfico, en el que los planificadores pueden claramente ver y aprehender los problemas, desafíos y oportunidades que tales cosas y sus circunstancias pueden presentar. Un escenario no es una predicción de un pronóstico específico por sí mismo; preferentemente, es una plausible descripción de qué puede ocurrir. Los escenarios describen eventos y tendencias y cómo ellas pueden evolucionar. especial atención sobre los procesos causales y los puntos para la toma de decisiones (Kahn 1967). Ningún escenario es siempre probable; la probabilidad de un escenario debe ser analizada minuto a minuto. La ocurrencia no es la medida de un buen escenario; preferentemente, esta medición debe hacerse considerando que éstos respondan a: Plausibilidad (un camino racional desde aquí hasta allá); Consistencia interna; Descripción de los procesos causales y Sean útiles en la toma de decisiones.
- **Casos de Uso:** los casos de uso son una técnica que se basa en escenarios para la obtención de requerimientos que se introdujeron por primera vez en el método Objeto (Jacobson et al. 1993). Actualmente se han convertido en una característica fundamental de la notación de UML, que se utiliza para describir modelos de sistemas orientados a objetos. En su forma más simple, un caso de uso identifica el tipo de interacción y los actores involucrados.
- **Etnografía:** La etnografía conocida también como ciencia del pueblo, es el estudio sistemático de personas y culturas. La etnografía es un método de investigación que consiste en observar las prácticas culturales de los grupos sociales y poder participar en ellos para así poder contrastar lo que la gente dice y lo que hace. Es una de las herramientas investigativas y algunos autores la consideran incluso como una rama de la antropología social o cultural, en un principio este método se utilizó para analizar a las comunidades aborígenes, actualmente se aplica también al estudio de cualquier grupo que se pretenda conocer mucho mejor. La etnografía es especialmente efectiva para descubrir dos tipos de requerimientos: Los requerimientos que se derivan de la forma en la que la gente trabaja realmente más que de la forma en la que las definiciones de los procesos establecen que debería trabajar y los requerimientos que se derivan de la cooperación y conocimiento de las actividades de la gente.
- **IEEE-STD-830-1998: Especificaciones de los Requisitos del Software:** El formato de Especificación de Requisitos Software (ERS) según la última versión del estándar IEEE 830. Según IEEE, un buen Documento de Requisitos, pese a no ser obligatorio que siga estrictamente la organización y el formato dados en el estándar 830, se deberá incluir, de una forma o de otra, toda la información presentada en dicho estándar. El estándar de IEEE 830 no está libre de defectos ni de prejuicios, y por ello ha sido justamente criticado por múltiples autores y desde varios puntos de vista, llegándose a cuestionar incluso si es realmente un estándar en el sentido habitual que tiene el término en otras ingenierías.
- **Introducción:** En esta sección se proporcionará una introducción a todo el documento de Especificación de Requisitos Software (ERS). Consta de varias subsecciones:
 - **Propósito:** En esta subsección se definirá el propósito del documento ERS y se especificará a quien va dirigido el documento.
 - **Ámbito del sistema:** Se podrá dar un nombre al futuro sistema, se explicará lo que el sistema hará y lo que no hará, se describirán los beneficios, objetivos y metas que se espera alcanzar con el futuro sistema y se referenciarán todos aquellos documentos de nivel.
 - **Definiciones:** En esta subsección se definirán todos los términos acrónimos y abreviaturas utilizadas en la ERS.
 - **Referencia:** En esta subsección se mostrará una lista completa de todos los documentos referenciados en la ERS.
 - **Visión General:** Esta subsección describe brevemente los contenidos y la organización del resto de la ERS.

- **Descripción General:** En esta sección se describen todos aquellos factores que afectan al producto y a sus requisitos. No se describen los requisitos, sino su contexto.
 - **Perspectiva del Producto:** Esta subsección debe relacionar el futuro sistema (producto software) con otros productos, si el producto es totalmente independiente de otros productos, también debe especificarse aquí. Si la ERS define un producto que es parte de un sistema mayor, esta subsección relacionara los requisitos del sistema mayor con la funcionalidad del producto descrito en la ERS, y se identificarán las interfaces entre el producto mayor y el producto aquí descrito.
 - **Funciones del Producto:** En esta subsección de la ERS se mostrara un resumen, a grandes rasgos, de las funciones del futuro sistema.
 - **Características de los Usuarios:** Esta subsección describirá las características generales de los usuarios del producto, incluyendo nivel educacional, experiencia y experiencia técnica.
 - **Restricciones:** Esta subsección describirá aquellas limitaciones que se imponen sobre los: desarrolladores del producto, Políticas de la empresa, Limitaciones del hardware, Interfaces con otras aplicaciones, entre otros.
 - **Suposiciones y Dependencias:** Esta subsección de la ERS describirá aquellos factores que, si cambian, pueden afectar a los requisitos.
 - **Requisitos Futuros:** Esta subsección esbozara futuras mejoras al sistema, que podrán analizarse e implementarse en un futuro.
 - **Requisitos Específicos:** Esta sección contiene los requisitos a un nivel de detalle suficiente como para permitir a los diseñadores diseñar un sistema que satisfaga estos requisitos, y que permita al equipo de pruebas planificar y realizar las pruebas que demuestren si el sistema satisface, o no, los requisitos.
 - **Interfaces Externas:** Se describirán los requisitos que afecten a la interfaz de usuario, interfaz con otros sistemas (hardware y software) e interfaces de comunicaciones.
 - **Funciones:** Esta subsección (quizá la más larga del documento) deberá especificar todas aquellas acciones (funciones) que deberá llevar a cabo el software.
 - **Requisitos de rendimiento:** Se detallaran los requisitos relacionados con la carga que se espera tenga que soportar el sistema.
 - **Restricciones de diseño:** Todo aquello que restrinja las decisiones relativas al diseño de la aplicación: Restricciones de otros estándares, limitaciones del hardware, etc.
 - **Atributos del Sistema:** Se detallaran los atributos de calidad del sistema: Fiabilidad, mantenibilidad, portabilidad, y, muy importante, la seguridad.
 - **Otros:** Cualquier otro requisito que no encaje en otra sección.
 - **Apéndices:** Pueden contener todo tipo de información relevante para la ERS pero que, propiamente, no forme parte de la ERS. **Por ejemplo:** Formatos de entrada/salida de datos, por pantalla o en listados. Resultados de análisis de costes. Restricciones acerca del lenguaje de programación.
- **Trazabilidad:** “Es la capacidad de describir y de seguir la vida de un requisito, tanto en dirección hacia adelante y hacia atrás, es decir, desde sus orígenes, a través de su desarrollo y especificación, a su despliegue y uso subsecuentes, y a través de períodos de refinamiento y de la iteración en curso en cualesquiera de estas fases” [GOTEL].
- Permite ver donde se originó un requerimiento. En otras palabras, todos los documentos de requerimientos, documentos de diseño, código y guiones de prueba deberán apuntar a su fuente de origen.
 - **Trazabilidad Bidireccional:** contar con un mecanismo integral de trazabilidad facilita la finalización de las tareas relacionadas con pruebas y documentación.
 - **Vertical:** garantiza que todos los requerimientos sean diseñados, y que todos los diseños se codifiquen y prueben. El mecanismo de trazabilidad no solo resalta las pruebas que se deben actualizar o repetir sino que también señala los documentos que se deben revisar.
 - **Horizontal:** Es la habilidad de relacionar las secciones/componentes de la misma fase entre si e identificar las dependencias que haya entre ellas, dependencia/relación de un componente/clase con otros componentes/clases. Permite fácilmente detectar si hay conflictos entre requerimientos, diseño, lógica de codificación o casos de prueba. Cuando se genera un control de cambios sobre un requerimiento, la trazabilidad horizontal brinda la opción de identificar si tendrá un impacto sobre algún otro aspecto de los requerimientos.

- **Matriz de Trazabilidad:** Durante el ciclo de vida del desarrollo de software de un proyecto existen dos fases muy importantes: definición de requisitos; y el diseño y ejecución de pruebas. En proyectos grandes o complejos es difícil poder saber que test ejecutados o diseñados cubren cada una de las especificaciones o requerimientos del proyecto. Es por este motivo que existe lo que se conoce como la matriz de trazabilidad. La matriz de trazabilidad es una herramienta que se utiliza para saber que requerimientos quedan cubiertos por una prueba.
- **Procesos de la ingeniería de requerimientos:** Fases principales en el proceso de ingeniería de requerimientos:
 - **Estudio de viabilidad:** Se estima si las necesidades del usuario se pueden satisfacer con las tecnologías actuales de software y hardware. La fase de evaluación de la información identifica la información requerida para comprender la evaluación y recopilación de la información, y la redacción de informes, normalmente, se debería intentar completar un estudio de viabilidad en dos o tres semanas. Una vez que se tiene la información, se redacta el informe del estudio de viabilidad. En el informe, se pueden proponer cambios en el alcance, el presupuesto y la confección de agendas del sistema y sugerir requerimientos adicionales de alto nivel para éste.
 - **Obtención y análisis de requerimientos:** Es el proceso de obtener los requerimientos del sistema por medio de la observación de los sistemas existentes, discusiones con los usuarios potenciales y proveedores, el análisis de tareas, etcétera. En esta actividad, los ingenieros de software trabajan con los clientes y los usuarios finales del sistema para determinar el dominio de la aplicación, qué servicios debe proporcionar el sistema, el rendimiento requerido del sistema, las restricciones hardware, etcétera.
 - **Especificación de requerimientos:** Es la actividad de traducir la información recopilada durante la actividad de análisis en un documento que define un conjunto de requerimientos. En este documento se pueden incluir dos tipos de requerimientos: los requerimientos del usuario, que son declaraciones abstractas de los requerimientos del cliente y del usuario final del sistema, y los requerimientos del sistema, que son una descripción más detallada de la funcionalidad a proporcionar.
 - **Validación de requerimientos:** Esta actividad comprueba la veracidad, consistencia y completitud de los requerimientos. La validación de requerimientos es importante debido a que los errores en el documento de requerimientos pueden conducir a importantes costes al repetir el trabajo cuando son descubiertos durante el desarrollo o después de que el sistema esté en uso.
 - **Revisiones de requerimientos:** Una revisión de requerimientos es un proceso manual que involucra a personas tanto de la organización del cliente como de la del contratista, ellos verifican el documento de requerimientos en cuanto a anomalías y omisiones, alternatively, se puede organizar como una actividad más amplia con diferentes personas que verifican diferentes partes del documento. Las revisiones de requerimientos pueden ser informales o formales. Los revisores también pueden comprobar la:
 - **Verificabilidad.** ¿Puede probarse el requerimiento de modo realista?
 - **Comprensibilidad.** ¿Las personas que adquieren el sistema o los usuarios finales comprenden correctamente el requerimiento?
 - **Rastreabilidad.** ¿Está claramente establecido el origen del requerimiento?
 - **Adaptabilidad.** ¿Es adaptable el requerimiento? Es decir, ¿puede cambiarse el requerimiento sin causar efectos de gran escala en los otros requerimientos del sistema?
- **Gestión de requerimientos:** permite la generación de una visión común del proyecto, revisión de los aspectos funcionales y técnicos, obteniendo el suficiente dominio de manera que se pueda administrar y prever cualquier circunstancia presentada durante su desarrollo.
 - **Requerimientos duraderos.** Son requerimientos relativamente estables que se derivan de la actividad principal de la organización y que están relacionados directamente con el dominio del sistema.
 - **Requerimientos volátiles.** Son requerimientos que probablemente cambian durante el proceso de desarrollo del sistema o después de que éste se haya puesto en funcionamiento. (Ej. requerimientos resultantes de las políticas gubernamentales sobre sanidad).

- **Planificación de la gestión de requerimientos:** La planificación es una primera etapa esencial del proceso de gestión de requerimientos. La gestión de requerimientos tiene un coste elevado. Para cada proyecto, la etapa de planificación establece el nivel de detalle necesario en la gestión de requerimientos. Durante la etapa de gestión de requerimientos, habrá que decidir sobre:
- **La identificación de requerimientos.** Cada requerimiento se debe identificar de forma única de tal forma que puedan ser remitidos por otros requerimientos de manera que pueda utilizarse en las evaluaciones de rastreo.
 - **Un proceso de gestión del cambio.** Éste es el conjunto de actividades que evalúan el impacto y coste de los cambios.
 - **Políticas de rastreo.** Estas políticas definen las relaciones entre los requerimientos, y entre éstos y el diseño del sistema que se debe registrar y la manera en que estos registros se deben mantener.
 - **Ayuda de herramientas CASE.** La gestión de requerimientos comprende el procesamiento de grandes cantidades de información sobre los requerimientos. Las herramientas que se pueden utilizar van desde sistemas de gestión de requerimientos especializados hasta hojas de cálculo y sistemas sencillos de bases de datos.

Tema 6: Modelos de Sistema

Una técnica ampliamente usada es documentar la especificación del sistema como un conjunto de modelos del sistema. Estos modelos son representaciones gráficas que describen los procesos del negocio, el problema a resolver y el sistema que tiene que ser desarrollado. Debido a las representaciones gráficas usadas, los modelos son a menudo más comprensibles que las descripciones detalladas en lenguaje natural de los requerimientos del sistema. Ellos constituyen también un puente importante entre el proceso de análisis y diseño. Pueden usarse modelos en el proceso de análisis para comprender el sistema existente que debe ser reemplazado o mejorado, o para especificar el nuevo sistema que sea requerido. Pueden desarrollarse diferentes modelos para representar el sistema desde diferentes perspectivas.

- **Diagramas del UML:** El UML está compuesto por diversos elementos gráficos que se combinan para conformar diagramas. Debido a que el UML es un lenguaje, cuenta con reglas para combinar tales elementos. La finalidad de los diagramas es presentar diversas perspectivas de un sistema, a las cuales se les conoce como modelo. Recordemos que un modelo es una representación simplificada de la realidad; el modelo UML describe lo que supuestamente hará un sistema, pero no dice cómo implementar dicho sistema.
- **Elaboración del modelo de los requerimientos:** El objetivo es describir los dominios de la información, función y comportamiento que se requieren para un sistema basado en computadora.
 - **Elementos de un modelo:**
 - Elementos basados en escenarios (casos de uso UML – Lenguaje Unificado de Modelado).
 - Elementos basados en Datos (estructuras)
 - Elementos basados en Clases (DC – Diagramas de Clases)
 - Elementos basados en Objetos (DO – Diagramas de Objetos)
 - Elementos basados en la Relación de Entidades (DER – Diagramas Entidad Relación)
 - Elementos de comportamiento (DTE – Diagramas de Transición de Estados)
 - Elementos Orientados al Flujo (DFD – Diagramas de Flujo de Datos)
 - **Requerimientos de las negociaciones.**
 - Identificación de participantes
 - Determinación de las condiciones “para ganar” de los participantes.
 - Negociación de las condiciones para ganar
 - **Validación de los requerimientos**
- **Modelos de Contexto:** En una de las primeras etapas de la obtención de requerimientos y del proceso de análisis se deben definir los límites del sistema. Esto comprende trabajar conjuntamente con los stakeholders (actores/participantes) del sistema para distinguir lo que es el sistema y lo que es el entorno del sistema. Se debería tomar estas decisiones al inicio del proceso para delimitar los costes del sistema y el tiempo necesario para el análisis. En algunos casos, el límite entre un sistema y su entorno está relativamente claro. Por ejemplo, en el caso de que un sistema automático reemplace a un sistema existente manual o computarizado, el entorno del nuevo sistema es normalmente el mismo que el entorno del sistema existente. En otros casos, hay más flexibilidad, y usted decide lo que constituye el límite entre el sistema y su entorno durante el proceso de ingeniería de requerimientos.
 - **Diagrama de Contexto:** El diagrama de contexto es un caso especial del diagrama de flujo de datos, en donde una sola burbuja representa todo el sistema. El diagrama de contexto muestra a través de flujos de datos las interacciones existentes entre los agentes externos y el sistema, sin describir en ningún momento la estructura del sistema de información. En este tipo de diagrama, el sistema de información debe representarse como un único proceso de muy alto nivel con entradas y salidas hacia los agentes externos que lo limitan, de forma equivalente a una caja negra. Teniendo en cuenta que este diagrama debe de ser comprensible, no es posible representar todos los flujos de datos del sistema en él, sino más bien debe representarse en él una visión general del sistema desde la perspectiva de los propietarios de sistemas.

- **Modelos de comportamiento:** Los modelos de comportamiento (**Modelos de Flujo de Datos, Modelos de Máquinas de Estado**) se utilizan para describir el comportamiento del sistema en su totalidad. La mayoría de los sistemas de negocio están fundamentalmente dirigidos por los datos. Están controlados por las entradas de datos al sistema con relativamente poco procesamiento de eventos externos. Un modelo de flujo de datos puede ser todo lo que se necesite para representar el comportamiento de estos sistemas. Por el contrario, los sistemas de tiempo real a menudo están dirigidos por eventos con un mínimo procesamiento de datos.
- **Modelos de Flujo de Datos:** los modelos de flujo de datos se utilizan para mostrar como fluyen los datos a través de una secuencia de pasos de procesamiento. Estos pasos de procesamiento o transformaciones representan procesos software o funciones cuando los diagramas de flujo de datos se utilizan para documentar un diseño de software. Sin embargo, en un modelo de análisis, el procesamiento se puede llevar a cabo por las personas o por las computadoras. Los modelos de flujo de datos son valiosos debido a que realizan un seguimiento y documentan como los datos asociados a con un proceso particular fluyen a través del sistema, y esto ayuda a los analistas a comprender el proceso. Tienen la ventaja de que, a diferencia de otras notaciones de modelado, son sencillos e intuitivos. Normalmente es posible explicarlos a los usuarios potenciales del sistema, quienes pueden entonces participar en la validación del análisis. Los DFDs pueden distribuirse en varios niveles, a mayor nivel, mayor detalle.
- **Diagrama de Flujo de Datos**
 - **Herramientas para Definir el Ambiente:**
 - **Declaración de propósitos:** Es una declaración textual breve y concisa del propósito del Sistema, dirigida al administrativo superior. También algún analista sugieren que la declaración de propósito debe incluir un resumen de beneficios tangibles y cuantificables a tal vez un análisis de costo-beneficios.
 - **Diagrama de Contexto:** Las personas, organizaciones y sistemas con los que se comunica nuestro sistema, se conoce como terminadores. Los datos que recibe del mundo exterior. Los datos que el sistema produce y que se envían al mundo exterior. Los almacenes de datos que el sistema comparte con los terminadores. La frontera entre el sistema y el resto del mundo.
 - **Lista de Acontecimientos:** es un listado sencillo de los acontecimientos del ambiente a los cuales debe responder el sistema. Al crear esta lista se debe distinguir entre un acontecimiento y un flujo relacionado con un acontecimiento. La lista de acontecimientos debe incluir no solo las interacciones normales entre el sistema y sus terminadores sino también situaciones de falla.
 - **Componentes:**
 - **El Proceso.** (Burbujas, función o transformación). Muestra una parte del Sistema que transforma entradas en
 - salidas. La burbuja se debe describir con un nombre o frase. Es conveniente la utilización de Verbos y objetos.
 - **El Flujo.** Describe el movimiento de bloques o paquetes de información de una parte del Sistema a otra.- Los flujos representan “datos en movimientos”.
 - **El Almacén.** Se utiliza para modelar una colección de paquetes de datos en reposo. Se representa con dos líneas paralelas. Los almacenes se conectan por flujos a los procesos.
 - **El Terminador.** Entidad. Es una persona, un Departamento, un Grupo, que están dentro o fuera de la Organización. En otros casos, la entidad puede ser otro Sistema con la cual existe una comunicación (retro-alimentación). (Las relaciones que pudieran existir entre las entidades, no se reflejan en un D.F.D. (no son partes del Sistema que se está estudiando).-
- **Modelos de Máquina de Estados:** Un modelo de máquina de estados de un sistema supone que, en cualquier momento, el sistema está en uno de varios estados posibles. Cuando se recibe un estímulo, éste puede disparar una transición a un estado diferente.
 - Componentes
 - **Estado:** El estado representa situaciones durante la vida de un objeto. Se representa con un rectángulo que tiene sus esquinas redondeadas.

- **Transición:** Una flecha representa el pasaje entre diferentes estados de un objeto. Se etiqueta con el evento que lo provoca y con la acción resultante.

➤ **Modelos de Datos:** La mayoría de los sistemas software grandes utilizan bases de datos de información de gran tamaño. Una parte importante del modelado de sistemas es la definición de la forma lógica de los datos procesados por el sistema. Estos se denominan a menudo modelos semánticos de datos. La técnica de modelado de datos más ampliamente usada es el modelado Entidad-Relación, modelo de clases, modelo de objetos.

- **Diagrama Entidad Relación:** Un diagrama o modelo entidad-relación (a veces denominado por sus siglas en inglés, E-R "Entity relationship", o del español DER "Diagrama de Entidad Relación") es una herramienta para el modelado de datos que permite representar las entidades relevantes de un sistema de información así como sus interrelaciones y propiedades. Permite la conversación con el grupo de Administradores de datos, la comunicación con el Grupo de Administración de Bases de Datos y enfatiza las relaciones con los almacenes de los D.F.D.

➤ **Modelos de Objetos:** Una aproximación orientada a objetos para el proceso de desarrollo del software en su totalidad se usa actualmente de forma generalizada, en particular para el desarrollo de sistemas interactivos. Esto significa expresar los requerimientos de los sistemas utilizando un modelo de objetos, diseñar utilizando objetos y desarrollar el sistema en un lenguaje de programación orientado a objetos, como por ejemplo Java o C++. Los modelos de objetos desarrollados durante el análisis de requerimientos pueden utilizarse para representar tanto los datos del sistema como su procesamiento, dichos modelos combinan algunos de los usos de los modelos de flujo de datos y los modelos semánticos de datos. Los modelos de objetos también son útiles para mostrar cómo se clasifican las entidades en el sistema y se componen de otras entidades.

- **Diagrama de Clases:** Los diagramas de clases describen la estructura estática de un sistema, las cosas que existen y que nos rodean se agrupan naturalmente en categorías, una clase es una categoría o grupo de cosas que tienen atributos (propiedades) y acciones similares. Un rectángulo es el símbolo que representa a la clase, y se divide en tres áreas. Un diagrama de clases está formado por varios rectángulos de este tipo conectados por líneas que representan las asociaciones o maneras en que las clases se relacionan entre sí.

- **Componentes de un Diagrama de Clases:** Las clases se representan con rectángulos divididos en tres áreas: la superior contiene el nombre de la clase, la central contiene los atributos y la inferior las acciones.

- **Asociaciones:** Son las que representan a las relaciones estáticas entre las clases. El nombre de la asociación va por sobre o por debajo de la línea que la representa.
- **Multiplicidad:** Las notaciones utilizadas para señalar la multiplicidad se colocan cerca del final de una asociación. Estos símbolos indican el número de instancias de una clase vinculadas a una de las instancias de la otra clase.
- **Agregación:** es una relación en la que la Clase "Todo" juega un rol más importante que la Clase "Parte", pero las dos clases no son dependientes una de otra. Se grafica con un rombo diamante vacío contra la Clase "Todo". En la agregación se denota: "Es parte de ..."
- **Composición:** Es un tipo especial de agregación que denota una fuerte posesión de la Clase "Todo", a la Clase "Parte". Se grafica con un rombo diamante relleno contra la clase que representa el todo.
- **Generalización:** Es otro nombre para herencia. Se refiere a una relación entre dos clases en donde una Clase "Específica" es una versión especializada de la otra, o Clase "General". En la generalización se denota "Es un..."

- **Diagrama de Objetos:** Los Diagramas de Objetos están vinculados con los Diagramas de Clases. Un objeto es una instancia de una clase, por lo que un diagrama de objetos puede ser visto como una instancia de un Diagrama de Clases. Los diagramas de objetos describen la estructura estática de un sistema en un momento particular y son usados para probar la precisión de los diagramas de clases.

- **Representación:**

- **Nombre de los objetos:** Cada objeto es representado como un rectángulo, que contiene el nombre del objeto y su clase subrayadas y separadas por dos puntos.

- **Atributos:** Como con las clases, los atributos se listan en un área inferior. Sin embargo, los atributos de los objetos deben tener un valor asignado.
- **Modelado Basado en Escenarios:** Para modelar el comportamiento de los objetos, se tiene que mostrar cómo se utilizan las operaciones proporcionadas por los objetos. Para ello se utilizan los escenarios que son representados como casos de uso UML. Una forma de modelar los comportamientos es utilizar diagramas de secuencia UML que muestran la secuencia de acciones implicadas en un caso de uso. Además de los diagramas de secuencia, UML también incluye diagramas de colaboración que muestran la secuencia de mensajes intercambiados por los objetos. Estos diagramas son similares a los diagramas de secuencia, por lo que no se tratan aquí.
- **Diagrama de Casos de Uso:** Un caso de uso es una descripción de las acciones de un sistema desde el punto de vista del usuario. Es una herramienta valiosa dado que es una técnica de aciertos y errores para obtener los requerimientos del sistema, justamente desde el punto de vista del usuario. Los diagramas de caso de uso modelan la funcionalidad del sistema usando actores y casos de uso. Los casos de uso son servicios o funciones provistas por el sistema para sus usuarios.
 - **Representación:**
 - **Sistema:** Se representa con un rectángulo estableciendo los límites del sistema que contiene los casos de uso. Los actores se ubican fuera de los límites del sistema.
 - **Casos de Uso:** Se representan con óvalos. La etiqueta en el óvalo indica la función del sistema.
 - **Actores:** Los actores son los usuarios de un sistema.
 - **Relaciones:** Las relaciones entre un actor y un caso de uso, se dibujan con una línea simple. Para relaciones entre casos de uso, se utilizan flechas etiquetadas "incluir" o "extender." Una relación "incluir" indica que un caso de uso es necesitado por otro para poder cumplir una tarea. Una relación "extender" indica opciones alternativas para un cierto caso de uso.
- **Diagrama de Actividades:** En un diagrama de actividades se muestra un proceso de negocio o un proceso de software como un flujo de trabajo a través de una serie de acciones. Estas acciones las pueden llevar a cabo personas, componentes de software o equipos. Puede usar un diagrama de actividades para describir procesos de diversos tipos, como los ejemplos siguientes:
 - Un proceso de negocio o un flujo de trabajo entre los usuarios y el sistema.
 - Los pasos realizados en un caso de uso.
 - Un protocolo de software, es decir, las secuencias de interacciones permitidas entre los componentes.
 - Un algoritmo de software.
- **Diagrama de Secuencias:** Los diagramas de clases y los de objetos representan información estática. No obstante, en un sistema funcional, los objetos interactúan entre sí, y tales interacciones suceden con el tiempo. El diagrama de secuencias UML muestra la mecánica de la interacción con base en tiempos.
 - **Representación:**
 - **Rol de la Clase:** El rol de la clase describe la manera en que un objeto se va a comportar en el contexto. No se listan los atributos del objeto.
 - **Activación:** Los cuadros de activación representan el tiempo que un objeto necesita para completar una tarea.
 - **Mensajes:** Los mensajes son flechas que representan comunicaciones entre objetos. Las medias flechas representan mensajes asincrónicos. Los mensajes asincrónicos son enviados desde un objeto que no va a esperar una respuesta del receptor para continuar con sus tareas
 - **Líneas de Vida:** Las líneas de vida son verticales y en línea de puntos, ellas indican la presencia del objeto durante el tiempo.
 - **Destrucción de Objetos:** Los objetos pueden ser eliminados tempranamente usando una flecha etiquetada "<<destruir>>" que apunta a una X.
 - **Loops:** Una repetición o loop en un diagrama de secuencias, es representado como un rectángulo. La condición para abandonar el loop se coloca en la parte inferior entre corchetes [].

- **Diagrama de Colaboraciones:** El diagrama de colaboraciones describe las interacciones entre los objetos en términos de mensajes secuenciados. Los diagramas de colaboración representan una combinación de información tomada de los diagramas de clases, de secuencias y de casos de uso, describiendo el comportamiento, tanto de la estructura estática, como de la estructura dinámica de un sistema.
- **Representación:**
 - **Rol de la Clase:** El rol de la clase describe cómo se comporta un objeto. Los atributos del objeto no se listan.
 - **Rol de las Asociaciones:** Los roles de asociación describen cómo se va a comportar una asociación en una situación particular. Se usan líneas simples etiquetadas con un estereotipo.
 - **Mensajes:** Contrariamente a los diagramas de secuencias, los diagramas de colaboración no tienen una manera explícita para denotar el tiempo, por lo que entonces numeran a los mensajes en orden de ejecución. La numeración puede anidarse; por ejemplo, número 1: 1.1, 1.2, 1.3, etc.

Tema 7: Análisis y Diseño Estructurado

- **AS/DS (SA/SD Structured Analysis and Structured Design, Análisis Estructurado y Diseño Estructurado):** El Análisis Estructurado (SA) en ingeniería de software y su técnica aliada, Diseño estructurado (SD), son métodos para analizar y convertir requisito (sistemas) de negocio dentro de especificaciones y en última instancia, Programa informático, configuraciones de hardware y procedimientos manuales relacionados. Las técnicas de análisis y diseño estructurados son herramientas fundamentales de análisis de sistemas, y desarrolladas a partir de análisis de sistemas clásicos de los años 1960 y 1970.

Un problema puede ser modelizado atendiendo a las siguientes perspectivas:

- **Descripción Estática del problema:** Identificando la estructura estática de los objetos y sus relaciones se obtiene una especie de fotografía que proporciona una imagen estática del dominio del problema.
 - **Perspectiva Dinámica del problema:** Se describen aspectos del sistema que cambian con el tiempo, y que, caracterizan los distintos estados que pueden presentar un problema, así como las condiciones y eventos que permiten transitar entre estados.
 - **Vista Funcional del problema:** Se describe como se transforman los datos en el contexto del problema.
- **Los Modelos del Sistema:** En los Modelos del Sistema se encuentra el Modelo Esencial que corresponde a la actividad del Análisis y el Modelo de Implementación que corresponde a la etapa de Diseño.
- **Modelo Esencial o de Análisis:** Incluye los siguientes modelos:
 - **El Modelo del Ambiente:** Declaración de los objetivos. Creación de un Diagrama de Contexto y de una Lista de Eventos, describe los estímulos que reciben el sistema y las respuestas generadas por los estímulos. Definición del Diccionario de Datos inicial. Tabla de Estimulo-Respuesta.
 - **El Modelo de Comportamiento:** Creación de un Diagrama de Flujo de Datos, y un Diagrama de Entidad Relación por cada uno de los eventos de la Lista de Eventos. Los un Diagramas de Flujo de Datos por eventos se unen en un único un Diagrama de Flujo de Datos (el Modelo Funcional) y los Diagramas de Entidad Relación por eventos se unen en un único Diagrama de Entidad Relación (el Modelo de Datos). Se acostumbra, también, modelar el comportamiento externo del sistema con Diagrama de Transición de Estados, árboles de pantallas o menús, etc. La creación simultánea del modelo de datos, modelo funcional y modelo de interfaz o comportamiento externo, ayuda en la validación y completitud del modelo esencial (descubriendo, por ejemplo, eventos no considerados).
 - **Modelo de Diseño o Implementación:** La actividad de diseño implica la realización de una serie de modelos, los cuales son: Modelo de Usuario, Modelo de Distribución (Modelo de Procesadores, Modelo de Tareas), Modelo de Programas.
- **El Modelo de Implementación o Modelo de Diseño:** A partir de esta etapa, el modelo esencial es instanciado en una tecnología dada. Se debe considerar ahora, las imperfecciones de la tecnología y determinar: la cantidad de procesadores necesarios, las cualidades de estos procesadores, el tamaño de disco necesario de acuerdo al volumen de la información a ser almacenada, etc. Luego se diseña la solución sobre la base de esas restricciones tecnológicas. La creación del modelo de implementación se fundamenta en la creación de tres modelos, uno de ellos en forma independiente (el modelo de implementación usuario o de la interfaz hombre-máquina) y los otros dos en forma encadenada en un proceso incremental de refinamiento e incorporación de detalles
- **El Modelo de Implantación del Usuario (Modelo de Diseño):** Es el punto de inflexión entre la etapa de análisis y la etapa de diseño. El modelo de implementación del usuario especifica un conjunto de restricciones que el usuario deseará imponer al grupo de desarrollo y condicionarán al diseñador. Define la interfaz hombre-máquina que es modelada en todos sus detalles, estilo, formato de pantallas, formato de informes y listados, diseño de pantallas para el ingreso de datos y presentación de resultados, estilo de mensajes de error, secuencialidad, etc. Los aspectos más importantes que se especifican en el modelo de implementación del usuario son:

- **Delimitación de la frontera de automatización:** distribución del modelo esencial entre personas y máquinas: el usuario puede tomar diferentes actitudes frente a este punto, pero lo que debe tenerse presente es que siempre es el usuario el que finalmente tiene la responsabilidad de fijar la frontera de automatización. El usuario puede definir frontera automática, manual o dejar que el desarrollador decida.
 - **Detalle de la interacción humano-máquina:** especifica todos los aspectos del diseño de la interfaz entre el sistema y el entorno como por ejemplo: Elección de dispositivos de E/S, formato de las entradas y salidas que fluyen entre los terminadores y el sistema, etc.
 - **Actividades de apoyo manual que se podrían requerir:** actividades 'no esenciales' que deben agregarse al sistema por no disponerse de una tecnología perfecta e ideal. Pueden representarse como burbujas adicionales en el modelo esencial. Los casos típicos son: Controles de posibles fallas humanas/técnicas y Operación del sistema en producción.
 - **Restricciones operativas que el usuario desea imponer al sistema:** son restricciones que afectarán la configuración de Hardware, sistema operativo, telecomunicaciones, lenguaje de programación. Algunos de los aspectos típicos son: Volumen de los datos, Tiempo de respuesta, Restricciones de seguridad, etc.
- **El Modelo de Distribución (Modelo de Diseño):** Describe todas las decisiones relativas a la arquitectura de hardware (modelo de procesadores) y a la estructuración general de la arquitectura de software (modelo de tareas). Se incorporan, en los modelos creados hasta este punto algunas Distorsiones (requerimientos no esenciales) destinadas a optimizar el uso de esa tecnología. El criterio fundamental es: Minimizar todo lo posible las distorsiones agregadas.
 - **El Modelo de Procesadores (Modelo de Distribución):** Asigna el modelo esencial a distintos procesadores y determina la arquitectura de comunicación entre ellos. Implica la asignación de procesos y almacenes a los procesadores. El modelo comportamental (modelo de datos, modelo funcional y modelo de comportamiento externo o de interfaz) es subdividido por procesadores. Se aplican criterios cualitativos (por ejemplo: necesidad de monitores de alta resolución gráfica) y cuantitativos (por ejemplo: velocidad del procesador, volumen de información almacenada, etc.) para seleccionar los procesadores, sistemas operativos, software y hardware de red, etc. Las distorsiones agregadas corresponden a la partición del DFD, ERD, DTE en procesadores, refinamiento de procesos y entidades o depósitos de datos (para asociar parte en un procesador y parte en otro) y a la incorporación de procesos para el control de la comunicación entre procesadores (siempre que la tecnología no solucione el problema de manera transparente). Según la cantidad de procesadores utilizados y las formas de comunicación entre ellos se tienen distintas configuraciones.
 - **Centralizada:** Asigna el modelo esencial completo a un único procesador central.
 - **Descentralizada:** Se asignan partes del modelo esencial a diferentes procesadores los cuales trabajan en forma independiente. En el caso de almacenes que deban ser compartidos por procesos asignados a diferentes procesadores, los mismos deberán duplicarse, y mantenerse copias actualizadas en cada procesador.
 - **Mixta:** Puede darse una combinación de los casos anteriores. Es común la existencia de un sistema central que consolida toda la información de la organización y que en diferentes unidades operativas que no esté conectadas a dicho procesador central existan sistemas satélites que implementan algunos procesos con almacenes con datos locales.
 - **Distribuida:** Se asignan partes del modelo esencial a diferentes procesadores los cuales están comunicados de alguna forma y sobre los que corre un sistema operativo distribuido. En este caso el usuario ve al conjunto de procesadores como un único recurso computacional.
 - **Cliente/Servidor:** Se distribuyen partes del proceso en diferentes procesadores. El esquema más genérico de distribución cliente-servidor distribuye el modelo del sistema en tres niveles: presentación, lógica del negocio, y acceso a base de datos.

Tipos de configuración de comunicación entre procesadores:

- **Conexión directa** entre procesadores (canal / red local / otros)
- **Enlace de telecomunicaciones** entre procesadores

- **Enlace indirecto:** los datos son transferidos de un procesador a otro por medio de almacenamiento masivo

Factores que influyen en la configuración de procesadores: Costo, Eficiencia, Seguridad (procesadores y datos en lugares seguros), Confiabilidad (separar los procesos en varios procesadores) y Restricciones políticas y operacionales.

- **El Modelo de Tareas (Modelo de Distribución):** Los modelos resultantes de la creación del modelo de procesadores son estudiados por separado (un procesador por vez), para determinar tareas diferentes (que serán programas diferentes de manera tal que se pueden ejecutar concurrentemente o no). La distorsión agregada en esta etapa representa la subdivisión del modelo funcional de un procesador (el DFD) en distintos DFDs (uno por tarea) agrupando procesos batch, interactivos o de tiempo real, partes del DFD aisladas del resto (comunicación solamente a través de depósitos de datos), etc. Además, es probable que sea necesario agregar procesos de control de concurrencia y sincronización para el acceso a recursos compartidos (como por ejemplo los depósitos de datos).
 - **El Modelo de Programas (Modelo de Diseño):** Para cada tarea debe desarrollarse un modelo de programa. De esto se encarga principalmente el Diseño Estructurado. La estructura del programa que implementa cada una de las tareas resultantes de las etapas de modelado de procesadores y tareas, es diseñada mediante la aplicación de las técnicas y estrategias descritas por el Diseño Estructurado (por ejemplo: Análisis de Transformaciones y Transacciones) y mejorada con la aplicación de criterios de calidad (por ejemplo: Cohesión, Acoplamiento, etc.).
- **Diagrama de Flujo de Datos:** Un diagrama de flujo de datos (DFD) es una representación gráfica del flujo de datos, Entradas y Salidas, a través de un sistema de información. Un diagrama de flujo de datos también se puede utilizar para la visualización de procesamiento de datos (diseño estructurado). Es una práctica común para un diseñador dibujar un contexto a nivel de DFD que primero muestra la interacción entre el sistema y las entidades externas.
- **Niveles de un DFD:**
 - **Nivel 0: Diagrama de contexto:** En el diagrama de contexto se caracterizan todas las interacciones que realiza un sistema con su entorno (entidades externas), estas pueden ser otros sistemas, sectores internos a la organización, o factores externos a la misma. Se dibuja un sólo proceso que representa al sistema en cuestión y se escribe su nombre en dicha burbuja como un sustantivo común más adjetivos. De él solamente parten los flujos de datos que denotan las interrelaciones entre el sistema y sus agentes externos.
 - **Nivel 1: Diagrama de nivel superior:** En el diagrama de nivel superior se plasman todos los procesos que describen al proceso principal. En este nivel los procesos no suelen interrelacionarse directamente, sino que entre ellos debe existir algún almacenamiento o entidad externa que los una.
 - **Nivel 2: Diagrama de detalle o expansión:** En un diagrama de nivel 2 o mayor, comienzan a explotarse las excepciones a los caminos principales de la información dado que aumenta progresivamente el nivel de detalle. De aquí en adelante se permiten los flujos entre procesos.
- **Diagrama de Entidad Relación:** Un diagrama o modelo entidad-relación (a veces denominado por sus siglas en inglés, E-R "Entity relationship", o del español DER "Diagrama de Entidad Relación") es una herramienta para el modelado de datos que permite representar las entidades relevantes de un sistema de información así como sus interrelaciones y propiedades. Los DER hacen énfasis en las relaciones entre los datos, permite conocer en detalle información que hay en cada almacén, además de conocer la relación que existe entre almacenes.
- **Componentes:** El modelo de datos entidad-relación está basado en una percepción del mundo real que consta de una colección de objetos básicos, llamados entidades, y de relaciones entre esos objetos.
 - **Entidad:** Representa una "cosa" u "objeto" del mundo real con existencia independiente, es decir, se diferencia unívocamente de otro objeto o cosa, incluso siendo del mismo tipo, o una misma entidad.
 - **Atributos:** Los atributos son las características que definen o identifican a una entidad. Estas pueden ser muchas, y el diseñador solo utiliza o implementa las que considere más relevantes.

- **Relación:** Describe cierta dependencia entre entidades o permite la asociación de las mismas. Se pueden establecer Conjuntos de Relaciones que consisten en una colección, o conjunto, de relaciones de la misma naturaleza.
- **Diagrama de Transición de Estados:** el diagrama de transición de estados nos permite expresar el comportamiento del sistema dependiente del tiempo. Los sistemas de este tipo manejan fuentes externas de datos de alta velocidad y deben proporcionar una respuesta y datos de salida de manera suficientemente rápida. Una parte importante de la especificación de tales sistemas es la descripción de que sucede cuando. Los modelos para estudiar el comportamiento de un sistema dependiente del tiempo son importantes aplicarlos para una categoría especial de sistemas conocidos como sistemas de tiempo real. Como ejemplos podríamos citar; control de procesos, sistemas de permutación telefónica, sistemas de adquisición de datos a alta velocidad y sistemas de control militares.
- **Componentes:**
 - **Estados del sistema:** Representa un estado en el que se puede encontrar el sistema. Un estado es un conjunto de circunstancias o atributos que caracterizan a una persona o cosa en un tiempo dado; forma de ser; condición. Un estado representa algún comportamiento del sistema que es observable y que perdura durante algún periodo finito.
 - **Cambios de Estados:** Los sistemas de información que modelamos pueden tener docenas de estados diferentes, para establecer un cambio de estado, el sistema tiene determinadas reglas de comportamiento, entonces ciertas clases de cambios de estado tendrán sentido y por tanto serán válidos. La forma de representar los cambios de estados validos en el DTE son con flechas conectando pares relevantes de estados. La mayoría de los sistemas tienen un estado inicial reconocible y un estado final reconocible.
 - **Condiciones y Acciones:** Las condiciones que causan un cambio de estado y las acciones que el sistema toma cuando cambia el estado. Las condiciones y acciones se representan junto a las flechas que conecta dos estados relacionados.
- **Diagramas particionados:** En un sistema complejo puede haber docenas de estados distintos del sistema: tratar de representarlos todos en un mismo diagrama sería difícil, si no imposible. Por lo tanto, tal como se usaron los niveles en la construcción de DFD, se pueden usar las particiones con los DTE.
- **Diccionario de Datos:** Un diccionario de datos es un conjunto de metadatos que contiene las características lógicas y puntuales de los datos que se van a utilizar en el sistema que se programa, incluyendo nombre, descripción, alias, contenido y organización. En un diccionario de datos se encuentra la lista de todos los elementos que forman parte del flujo de datos (DFD) de todo el sistema. Los elementos más importantes son flujos de datos, almacenes de datos y procesos. El diccionario de datos guarda los detalles y descripción de todos estos elementos. Describe el significado de los flujos y almacenes que se muestran en los D.F.D., describe la composición de almacenes (paquetes), especifica los valores y unidades relevantes de piezas elementales de información de los flujos de datos y en los almacenes de datos, describe los detalles de las relaciones entre almacenes en un diagrama de Relación-Entidad.
- **Descripción de un Elemento de Datos:**
 - **Nombre:** El nombre debe elegirse de manera tal que tenga la mayor significación posible para el usuario.-
 - **Descripción:** Podrá ser un breve esbozo del significado del elementos de datos y puede incluir un típico ejemplo.-
 - Alias, Elementos de datos relacionados, Rango de valores y significado de los valores, longitud, codificación y otras informaciones de edición: Rutinas de validación, consistencia, congruencia, etc.
- **Especificaciones de Proceso:** Descripción de que es lo que sucede en cada burbuja primitiva de un nivel más bajo de un D.F.D. La especificación de procesos describe las reglas sobre cómo realizar el proceso para transformar las entradas en salidas. Indican el proceso a realizar, la transformación de datos, no el algoritmo (que se selecciona en la etapa de diseño).
- **Herramientas para describir la lógica de los procesos:**

- **Tablas de decisión**
- **Árboles de decisión**
- **Pre y post-condiciones**
- **Lenguaje estructurado:** Vocabulario (restringido) de una lengua (español, inglés, etc.), verbos imperativos, términos definidos en el DD, palabras reservadas para formulación lógica (mayúsculas), sintaxis de la programación estructurada.

➤ **Balanceo de Modelos:** Cuando se modelan tres aspectos distintos de un sistema (funciones, datos y tiempos), además de modelar las características detalladas del sistema en un diccionario de datos y un conjunto de especificaciones de procesos, es fácil desarrollar diversas interpretaciones diferentes e inconsistentes de esa misma realidad. El Balanceo de Modelos logra generar Modelos y Diagramas consistente con una realidad específica, eliminando todos los errores que puedan existir entre modelos y diagramas diferentes, pero relacionados a una misma realidad.

- **Balanceo del Diagrama de Flujos de Datos y el Diccionario de Datos:** Cada flujo de datos y cada almacén de datos deben estar definidos en el diccionario de datos. Si falta la definición se considera indefinido. De manera inversa, cada dato y cada almacén que se define en el diccionario de datos debe aparecer en alguna parte del DFD. Si no aparece dicho dato es un fantasma.
- **Balanceo del Diagrama de Flujos de Datos y las Especificaciones de Proceso:** Cada burbuja del DFD debe asociarse con un DFD de nivel inferior o con una especificación de proceso, pero no ambas. Cada especificación de proceso debe tener una burbuja de nivel inferior asociada en el DFD. Las entradas y salidas de los DFD deben coincidir con ingreso y salidas de datos en las Especificaciones de proceso (GET, PUT, READ, WRITE, DELETE, etc.).
- **Balanceo de las Especificaciones de Proceso con el Diagrama de Flujos de Datos y el Diccionario de Datos:** Cada referencia de un dato en las especificaciones de proceso debe satisfacer una de las siguientes reglas: coincide con el nombre de un flujo de datos o almacén conectado a la burbuja descrita por la especificación de proceso, es un término local, definido explícitamente en la especificación de proceso, aparece como componente en una entrada del diccionario de datos para un flujo o almacén conectado con la burbuja.
- **Balanceo del Diccionario de Datos con el Diagrama de Flujos de Datos y las Especificaciones de Proceso:** El Diccionario de datos es consistente con el resto del modelo si obedece la siguiente regla: Cada entrada del diccionario de datos debe tener referencia en una especificación de proceso, un DFD, u otro diccionario de datos.
- **Balanceo del Diagrama de Entidad-Relación con el Diagrama de Flujos de Datos y las Especificaciones de Proceso:** Existen algunas relaciones que deben darse para que el sistema global sea completo, correcto y consistente: cada almacén del DFD debe corresponder con un tipo de objeto, una relación o un tipo de objeto asociativo, los nombres de objetos en el DER y los nombres de almacenes de datos en el DFD deben coincidir, las entradas del diccionario de datos deben aplicarse tanto al modelo de DFD como al DER.
- **Balanceo del Diagrama de Flujos de Datos y el Diagrama de Transición de Estados:** la condición de estado puede considerarse balanceada con el diagrama de flujo de datos si se cumple con las siguientes reglas: cada burbuja de control del DFD se asocia con un diagrama de transición de estados como su especificación de proceso, cada condición del diagrama de transición de estados debe corresponderse con el flujo de datos de entrada al proceso de control asociado con el diagrama de transición de estados, cada acción en el diagrama de transición de estados debe corresponder con un flujo de control de salida del proceso de control asociado con dicho diagrama.

➤ **Diagramas de Estructura:** Los diagramas de estructura (DE) sirven para el modelamiento top-down de la estructura de control de un programa descrito a través de un árbol de invocación de módulos. Fueron presentados en la década de los 70 como la principal herramienta utilizada en diseños estructurados, por autores como Constantine, Myers, Stevens e Yourdon. Un diagrama de estructura permite modelar un programa como una jerarquía de módulos. Cada nivel de la jerarquía representa una descomposición más detallada del módulo del nivel superior. La notación usada se compone básicamente de tres símbolos: Módulos, Invocaciones y Cuplas.

Tema 8: Análisis Orientado a Objetos

“Es un método de análisis que examina los requisitos desde la perspectiva de las clases y objetos que se encuentran en el vocabulario del dominio del problema”

- **Modelo:** Un modelo es una representación de un objeto, sistema o idea, de forma diferente al de la entidad misma. El propósito de los modelos es ayudarnos a explicar, entender o mejorar un sistema. Un modelo de un objeto puede ser una réplica exacta de éste o una abstracción de las propiedades dominantes del objeto. Un modelo sirve para:
 - Verificar una entidad física antes de su construcción
 - Comunicarnos con otros usuarios
 - Visualizar
 - Disminuir la complejidad
- **Modelos Abstractos:** examen selectivo de ciertos aspectos del problema, eliminando los aspectos que no son importantes. La abstracción sirve para un propósito que nos determina lo que no es importante, delimitando nuestro universo. Es posible obtener múltiples abstracciones de la misma cosa. Todas las abstracciones son incompletas o inadecuadas, un buen modelo captura los aspectos cruciales del problema y omite el resto.
- **Técnica de Modelado de Objetos:** OMT es una metodología de análisis y diseño orientados a objetos, contiene una notación uniforme, interconexión entre modelos limitada y explícita. Sus fases son: Definición del Problema, Análisis, Diseño, Implementación. OMT no cubre la fase de Mantenimiento y Prueba. La metodología OMT emplea tres clases de modelos para describir el sistema:
 - **Modelo de objetos:** Describe la estructura estática de los objetos del sistema (identidad, relaciones con otros objetos, atributos y operaciones). El modelo de objetos proporciona el entorno esencial en el cual se pueden situar el modelo dinámico y el modelo funcional. El objetivo es capturar aquellos conceptos del mundo real que sean importantes para la aplicación. Se representa mediante diagramas de objetos que contendrán clases de objetos las cuales se disponen en jerarquías que comparten estructura, comportamientos comunes y que están asociadas con otras clases.
 - **Modelo dinámico:** Describe los aspectos de un sistema que tratan de la temporización y secuencia de operaciones tales como: Sucesos (eventos), estados y acciones. Captura el control del sistema. Se representa gráficamente mediante diagramas de estado.
 - **Modelo funcional:** Describe las transformaciones de valores de datos (funciones, correspondencias, restricciones y dependencias funcionales) que ocurren dentro del sistema. Captura lo que hace el sistema, independientemente de cuando se haga o de la forma en que se haga. Se representa mediante diagramas de flujo de datos
 - **Relación entre los modelos:** cada modelo describe un aspecto del sistema manteniendo referencias al resto de los modelos, el Modelo de Objetos define las estructuras de datos sobre las que actuarán el Modelo Dinámico y el Modelo Funcional. Los sucesos serán las operaciones de los objetos definidas en el Modelo de Objetos. Las propiedades del sistema que no puedan ser correctamente expresadas en ningún modelo, deberán expresarse en lenguaje natural como alternativa válida.
- **Visión General del Análisis**
 - **Definición del Problema:** escribir y obtener una descripción inicial del problema
 - **Construir un Modelo de Objetos:** identificar las clases, comenzar el diccionario de datos con la descripción de las clases, sus atributos y operaciones, añadir asociaciones entre las clases, añadir los atributos a las clases y a las asociaciones, organizar y simplificar las clases utilizando la herencia. Agrupar las clases en módulos, basados en el acoplamiento y funciones relacionadas. El modelo de Objetos está compuesto por el Diagrama del Modelo de Objetos más el Diccionario de Datos
 - **Desarrollo de un Modelo Dinámico:** preparar guiones de las secuencias de interacción más frecuentes, identificar sucesos entre objetos y preparar una traza de sucesos por cada guion, preparar un diagrama de

Flujo de Sucesos para el sistema completo, desarrollar un diagrama de estados para cada clase que tenga un comportamiento dinámico significativo, comprobar la consistencia y compleción de los sucesos compartidos entre los diagramas de estado. El Modelo Dinámico se compone de, Diagrama de Estado más Diagrama de Flujo de Sucesos Global.

- **Construir un Modelo Funcional:** identificar valores de entrada y salida, utilizar diagramas de Flujo de Datos para mostrar las dependencias funcionales, describir lo que hace cada función, identificar las restricciones, especificar los criterios de optimización. El modelo Funcional se compone de Diagramas de Flujo de Datos más Restricciones.
 - **Verificar, iterar y refinar los tres modelos:** añadir operaciones claves descubiertas en la preparación del Modelo Funcional, verificar que las clases, atributos, operaciones y asociaciones sean consistentes y completas al nivel de abstracción elegido, desarrollar guiones más detallados (que pasaría si), iterar los pasos anteriores, según sea, necesario para completar el análisis. Documento de Análisis está compuesto por la Definición del Problema, el Modelo de Objetos, el Modelo Dinámico y el Modelo Funcional.
- **Diseño del Sistema:** organizar el sistema en subsistemas, identificar la concurrencia inherente al problema, ubicar subsistemas a los procesadores y tareas, seleccionar la estrategia apropiada para almacenar los datos, identificar los recursos globales y determinar los mecanismos para controlar el acceso, considerar condiciones de frontera, establecer opciones de prioridades. El Documento de Diseño de Sistema se compone de Estructura de la arquitectura básica del sistema, así como las decisiones de estrategia de alto nivel.
- **Diseño de Objetos:** Establecer las operaciones para cada suceso del Modelo Dinámico y las operaciones de proceso para el Modelo Funcional, seleccionar el algoritmo de implementación, seleccionar estructuras de datos adecuadas, definir nuevas clases y estructuras internas si se necesitan, optimizar los caminos de acceso a datos, redistribuir y ajustar las clases y operaciones para incrementar la herencia, abstraer los comportamientos, utilizar delegación, determinar la representación exacta de los atributos de los objetos, empaquetar clases y asociaciones en módulos. Documento de Diseño se compone del Modelo de Objetos Detallado, el Modelo Dinámico Detallado y el Modelo Funcional Detallado.
- **Modelo Funcional:** Especifica lo que sucede, consta de múltiples DFDs que muestran el flujo de valores de entrada y salidas, operaciones y almacenes internos. Se compone de Diagramas de Flujo de Datos (DFD) (Actores, Procesos, Almacenes de Datos), Flujos de Datos, Descomposición por Niveles. Especificación de Operaciones. Consistencia con Otros Modelos.
- **Diagrama de Flujos (DFD):** Un DFD muestra la relación funcional de los valores que calcula el sistema.
 - **Procesos:** es una transformación de datos, definen patrones de entradas y salidas, se representan mediante elipses que contienen una descripción de la transformación, normalmente un nombre.
 - **Actores:** son objetos activos que conducen el DFD produciendo o consumiendo valores (terminadores), definen los límites (el contexto) del sistema, pueden ser usuarios del sistema u elementos de hardware en sistemas Tiempo Real. Los actores son representados por un cuadrado con el nombre del actor.
 - **Flujo de Datos:** representan valores o conjuntos de valores que se transmiten de un cálculo a otro, conectan la salida de un actor o proceso con la entrada de otro, pueden descomponerse, duplicarse o unirse según la situación de transmisión.
 - **Almacenes de Datos:** son objetos pasivos que almacenan datos (leer, cargar, eliminar datos), suelen ser conjuntos de datos heterogéneos a los que se puede acceder en orden diferente al de inserción, se representan por un par de líneas paralelas que contienen el nombre del almacén.
 - **Flujos de Control:** Un flujo de control es un valor booleano que afecta a si un proceso es o no ejecutado. El flujo de control no es un valor de entrada al proceso en sí. Los flujos de control pueden ser útiles en ocasiones, pero duplican información del modelo dinámico.
 - **Niveles de Descomposición:** El proceso de descomposición usualmente es descendente (top-down), cualquier proceso puede descomponerse como un nuevo DFD que detalla su funcionamiento. Se pueden estudiar separadamente cada DFD en cualquier nivel del árbol.

- **Restricciones:** Una restricción muestra la relación entre dos objetos al mismo tiempo o bien entre distintos valores del mismo objeto en instantes diferentes. Las restricciones pueden aparecer en todas las clases del modelo. Las restricciones de objetos especifican que algunos objetos dependen entera o parcialmente de otros objetos.
- **Especificación de Operaciones:** Los procesos de los diagramas de flujo deben ser implementados eventualmente como operaciones que se aplican a objetos. Todo proceso atómico del más bajo nivel es una operación. Los procesos de nivel superior también se pueden considerar operaciones. Toda operación se podrá especificar de diferentes maneras, entre las que están: funciones matemáticas, tablas de valores de entrada y salida, ecuaciones que especifican la salida dependiendo de la entrada, condiciones previas y posteriores, tablas de decisión, pseudocódigo, lenguaje natural.
- **Consistencia con otros Modelos:** Los procesos se corresponden con operaciones del Modelo de Objetos, los Procesos indican relaciones funcionales entre clases, los almacenes de datos son clases de objetos pasivos, los actores son clases del Modelo de Objeto y, sus flujos de datos, operaciones, los flujos de datos son valores del Modelo de objeto.

➤ **Modelo Dinámico:** Los aspectos del sistema que están relacionados con el tiempo y con los cambios constituyen el modelo dinámico. Los conceptos más importantes del modelado dinámico son los sucesos, que representan estímulos externos, y los estados, que representan los valores de los objetos. El diagrama de estados va a representar los sucesos y los estados que se dan en el sistema. El Modelo Dinámico consiste en múltiples Diagramas de Estado, uno por cada clase con comportamiento dinámico significativo.

- **Suceso:** estímulo individual de un objeto a otro. Ocurre en un determinado instante y no tiene duración.
 - **Estado:** valores de los atributos y enlaces de un objeto en un instante dado.
 - **Diagrama de Estado:** patrón de sucesos, estados y transiciones entre estados para una clase determinada.
- **Sucesos:** los sucesos pueden darse consecutivamente de forma lógica (causa-efecto) o no (concurrentes-al mismo tiempo). Son un medio de transmisión de información de un objeto a otro, unidireccional.
 - **Clases de sucesos:** indica estructura y comportamiento comunes. Pueden contener atributos para expresar la información que manejan.
 - **Escenarios y trazas de sucesos**
 - **Escenarios:** secuencia de sucesos que tienen lugar durante una ejecución particular del sistema, el ámbito del escenario es variable, pueden incluirse todos los sucesos del sistema o solo aquellos que sean significativos, pueden enviarse sucesos concurrentes. La descripción del escenario es el primer paso del modelo dinámico, al que debe seguir la identificación de los objetos emisores y receptores de sucesos.
 - **Diagrama de Traza de Sucesos (DTS):** especifican la secuencia de sucesos en el tiempo, en la etapa de diseño deberán refinarse los DTSs.
 - **Estados:** abstracción de los valores de los atributos y enlaces de un objeto, especifican la respuesta de un objeto a los sucesos que llegan (acción/cambio de estado), representan intervalos de tiempo, en su definición, se ignoran los atributos que no intervienen en el comportamiento del objeto.
 - **Diagrama de Estado:** Relaciona Sucesos y Estados, el siguiente Estado depende del Suceso recibido y del Estado actual, las transmisiones para abandonar un Estado deben corresponder a sucesos diferentes, si un Suceso no genera una transición que abandone el Estado, se ignora, los estados no definen por completo todos los valores del objeto, pueden representarse bucles continuos o ejecuciones no repetitivas, los diagramas que representan ejecuciones no repetitivas tienen estados iniciales y finales, se utilizan como subrutinas referenciadas desde diversos diagramas de más alto nivel.
 - **Condiciones:** Una condición es una función Booleana lógica que tiene a objetos como valores. Las condiciones se pueden utilizar como protecciones en las transiciones. Una transición con protección se dispara cuando se produce su suceso, pero sólo si la condición de protección es verdadera.
 - **Operaciones:** están asociadas a las transiciones entre Estados, se describen de dos maneras, las Acciones que son operaciones instantáneas asociadas a un suceso u operaciones de control interno y las Actividades que son operaciones con duración en el tiempo, asociadas a un Estado.

- **Diagrama de Estado anidados:** una actividad en un Estado puede expandirse como un Diagrama de Estado de nivel inferior, en el que cada Estado representa una etapa de la actividad, los Sucesos también pueden expandirse en Diagramas de Estados subordinados. los Diagramas de Estados pueden estructurarse de forma similar a la de los objetos:
 - **Generalización:** actividades de anidamiento que permiten organizar los Estados y Sucesos de forma jerárquica.
 - **Generalización de Estados:** en general, los Estados en un Diagrama Anidado, pueden interactuar con otros Estados, pueden tener sub-estados que hereden sus transiciones, a menos que las “reemplacen” por otras equivalentes.
 - **Generalización de Sucesos:** los Sucesos pueden organizarse en una jerarquía de generalización en la que se heredan los atributos de los Sucesos, permiten utilizar distintos niveles de abstracción en diferentes partes del Modelo.
 - **Agregación:** permite la descomposición de un Estado en componentes ortogonales, siendo equivalente a la concurrencia de los Estados.
 - **Concurrencia:** un Diagrama de Estados para un sistema compuesto es una colección de diagramas, uno por cada componente, la agregación implica Concurrencia, los diagramas de estado de los componentes suelen ser independientes, pero pueden interactuar, la concurrencia dentro de un objeto se comprende como dividir un objeto en subconjuntos de atributos y enlaces, cada uno con su propio diagrama.
 - **Acciones de Entrada y Salida:** acciones que se ejecutan cuando se entra o se sale de un Estado por cualquier suceso, estas acciones se simplifican cuando todas las transiciones impliquen esas mismas acciones para la entrada o la salida, poseen un orden de ejecución: Acciones de transición entrantes, Acciones de entrada, Actividades, Acciones de salida y Acciones de transición salientes.
 - **Acciones Internas:** un Suceso puede probar que se ejecute una acción sin cambiar de Estado, en la notación se refleja como suceso/acción.
 - **Sincronización de Actividades Concurrentes:** a veces, un objeto puede realizar actividades de forma concurrente, los pasos internos de las actividades no han de estar sincronizados, pero todas ellas han de completarse para pasar al siguiente estado: División del Control y Fusión del Control.
 - **Relación Entre el Modelo de Objetos y el Modelo Dinámico:** El diagrama de Estado describe todo o una parte del comportamiento de un objeto de una clase, los Estados son los equivalentes a los valores de los atributos y enlaces de las clases, los Sucesos pueden representarse como operaciones en el Modelo de Objetos.
- **Modelo de Objetos:** captura la estructura estática del sistema, mostrando, objetos, relaciones entre objetos, atributos y operaciones, es el más importante, puesto que el sistema se construye alrededor de objetos.
- **Objeto:** Concepto, abstracción o cosa con fronteras definidas y significado para nuestro problema, permite una mejor comprensión del mundo y proporciona la base para una implementación sobre el ordenador, no existe una representación exacta, todos los objetos tienen una identidad y son distinguibles.
 - **Clase:** Describe grupos de objetos con propiedades (atributos) similares, comportamiento (operaciones) comunes, relaciones con otros objetos y semántica común, cada objeto sabe cuál es su clase, ya que es una instancia de la misma, elemento esencial para la abstracción y generalización.
 - **Diagrama de Objetos:** Notación gráfica para modelar los objetos, clases y sus relaciones, define dos tipos de diagramas, de clases y de objetos (instancias)
 - **Diagrama de clases:** esquema, patrón o plantilla para describir muchos casos posibles de datos, describe clases de objetos.
 - **Diagrama de objetos:** describe cómo se relacionan un grupo particular de objetos entre sí.
 - **Atributos:** Valor de un dato dentro de un objeto, cada atributo tiene un valor para cada objeto, el nombre de un atributo es único dentro de una clase, debería ser un dato ‘puro’, no un objeto (no tiene identidad): si un objeto necesita otro objeto habrá que modelarlo como asociación, además del nombre podemos especificar el Tipo y el Valor por defecto.

- **Operaciones y Métodos:** Función o transformación que se aplica 'a' o 'por' los objetos en una clase, tienen un objeto destino como argumento implícito (el objeto actual).
 - **Polimorfismo:** la misma operación puede aplicarse a clases diferentes dentro de una jerarquía de herencia.
 - **Método:** implementación de una operación para una clase, las operaciones con métodos en varias clases deben compartir la misma signatura (tipos de sus argumentos y valor que devuelven).
- **Enlaces y Asociaciones:**
 - **Enlace:** conexión física o conceptual entre objetos.
 - **Asociación:** grupo de enlaces con la misma estructura y semántica común. Las asociaciones pueden ser binarias, ternarias o de órdenes superiores. Los nombres de las asociaciones son opcionales en la notación y se escriben en cursiva. El sentido de la asociación es inherentemente bidireccional, pero pueden ser solo directas o inversas.
- **Multiplicidad:** Especifica cuántos objetos de una clase pueden relacionarse con un único objeto de una clase asociada. Se especifica mediante un subconjunto (posiblemente infinito) de enteros no negativos: 0..n, o bien uno o varios intervalos no conexos: 1..4, 7.
- **Atributos de los enlaces:** Propiedades de los enlaces de una asociación, opcionales en los enlaces uno a uno y uno a muchos, obligatorio en los enlaces muchos a muchos.
- **Roles:** suelen aparecer como 'nombres' en los enunciados de los problemas, proporcionan una forma de ver las asociaciones como recorrido desde un objeto hacia otro conjunto de objetos. Su uso es opcional pero suele ser más claro su uso que los nombres de asociaciones, es necesario su utilización en relaciones entre objetos de la misma clase, o cuando existe más de una asociación entre el mismo par de clases.
 - **Rol:** uno de los extremos de una asociación. En relaciones n-arias: un rol en cada extremo.
 - **Nombre de Rol:** nombre que identifica de forma única el final de una asociación. Atributo derivado cuyo valor es un conjunto de objetos relacionados. Deben ser únicos en las asociaciones de una clase.
- **Ordenación:** Se utiliza en la parte 'muchos' de una asociación en la que los objetos deben estar ordenados.
- **Cualificadores:** atributo especial que reduce la multiplicidad efectiva de una asociación, es una forma de asociación ternaria, se expresa como una etiqueta dentro de un pequeño recuadro adosado al extremo del multiplicidad "muchos".
- **Agregación:** forma especial de asociación, relación "parte-todo" o "es-parte-de", los objetos que representan los componentes se asocian con un objeto que representa todo el compuesto propiedades: transitividad, antisimetría, propagación a algunas partes de algunas propiedades del compuesto.
- **Agregados Recursivos:** contienen, directa o indirectamente, un caso de la misma clase.
- **Propagación de las operaciones:** aplicación automática de una operación a una red de objetos cuando se aplica operación a un objeto inicial, la propagación de una operación a las partes es un buen indicador de agregación.
- **Generalización y Herencia:** Recursos para compartir semejanzas entre clases, manteniendo las diferencias. Generalización, especialización y herencia son relaciones y procesos transitivos.
 - **Generalización:** Relación entre una clase y una versión menos refinada de ella, es una relación "is-a (es un)", la relación inversa es la de especialización, la clase más general es la superclase y la menos general la subclase.
 - **Procedimiento:** Asignamos a la superclase los atributos y operaciones comunes al grupo de sus subclases, decimos que cada subclase "HEREDA" las características de su superclase. una clase es antecesora de otra si es una de sus superclases, una clase es sucesora de otra si es una de sus subclases. Las subclases añaden sus propios atributos y operaciones a los heredados de sus antecesoras. Los nodos terminales del árbol jerárquico son las instancias (objetos) de las subclases más bajas del árbol. Estas instancias heredan todos los atributos y operaciones de sus clases antecesoras. En ocasiones, una subclase puede especificar un valor de un atributo o una operación, aunque le vengán suministrados por sus superclases: Los valores por defecto de los atributos y los métodos de las operaciones, normalmente, nunca se modifican las signaturas.

- **Redefinición (override):** Una subclase 'reemplaza' una característica de su superclase definiendo una característica con el mismo nombre, esto permite especificar un comportamiento que dependa de la subclase, aplicar de forma más rigurosa las especificaciones de una característica, mejorar el rendimiento. No se debe reemplazar la signature de una característica, debe preservar, los tipos de los atributos, número y tipos de argumentos de funciones y tipo del valor devuelto por una operación.
- **Clases Abstractas:** Las clases abstractas sirven para definir un comportamiento y/o estructura común a un conjunto de subclases. Una clase abstracta no es instanciable. Organizan características comunes de varias clases (a). Pueden definir el protocolo para una operación sin proporcionar un método correspondiente (operación abstracta).
- **Herencia múltiple:** Permite a una clase tener más de una superclase, y heredar las características de todos sus padres se utiliza para obtener una mayor potencia a la hora de definir nuevas clases, oportunidades adicionales de reutilización y acercamiento a la forma natural de pensar.
- **Claves candidatas:** Es un conjunto mínimo de atributos que definen de manera unívoca un objeto. Una clase puede tener más de una clave candidata, cada una de las cuales tendrá distintas combinaciones y número de atributos. La etiqueta de un objeto es siempre una clave candidata de una clase, para las asociaciones son claves candidatas una o más combinaciones de objetos relacionados.
- **Restricciones:** Es una relación funcional entre entidades (clase, objeto, atributo, enlace o asociación) dentro del Modelo de Objetos que limita los valores que la entidad puede tomar. Se aplica también sobre los enlaces.

Tema 9: Otros métodos de la Ingeniería del Software

- **Métodos Formales:** “Método Formal es cualquier técnica que trate la construcción y/o el análisis de modelos matemáticos que contribuyen a la automatización del desarrollo de sistemas informáticos”
- **Un poco de historia:** En 1967, fue Robert Floyd quien propuso utilizar lo que se denominó método de aserciones intermedias como una manera de estudiar las propiedades de los programas. Estas ideas fueron perfeccionadas por Hoare dando lugar al método axiomático (precondiciones E/S) donde introdujo la idea de "invariante". En 1976, Edsger Dijkstra, presentó un método formal llamado precondition más débil, basado en la transformación de predicados WP (weakest precondition). La idea principal era invertir los métodos de ambos de tal manera que se pudiera derivar la precondition a partir de la post-condición.
 - **Propiedades:** Carencia de ambigüedad, Consistencia (reglas de inferencia), Completitud.
 - **Ventajas:** se comprende mejor el sistema, la comunicación con el cliente mejora ya que se dispone de una descripción clara y no ambigua de los requisitos del usuario, el sistema se describe de manera más precisa, se asegura matemáticamente que las especificaciones sean correctas, mayor calidad y productividad del software.
 - **Problemática:** la falta de madurez en la práctica es la causa de la imposibilidad de utilizarlos a nivel industrial tal y como se utilizan otros métodos de ingeniería del software. El desarrollo de herramientas que apoyen el método formal es complicado y los programas resultantes son incómodos para los usuarios, se considera que la aplicación de métodos formales encarece los productos y ralentiza su desarrollo.
 - **El papel de los métodos formales en la ingeniería del software:** Los métodos formales se basan en el empleo de técnicas, lenguajes y herramientas definidos matemáticamente para cumplir objetivos tales como facilitar el análisis y construcción de sistemas confiables independientemente de su complejidad, delatando posibles inconsistencias o ambigüedades que de otra forma podrían pasar inadvertidas. La formalización matemática del Software es el enfoque más apropiado para conseguir mejorar su calidad, va adquiriendo cada vez más fuerza.
 - **Clasificación:** La clasificación más común se realiza en base al modelo matemático subyacente en cada método, de esta manera podrían clasificarse en:
 - **Especificaciones basadas en lógica de primer orden y teoría de conjuntos:** permiten especificar el sistema mediante un concepto formal de estados y operaciones sobre estados. Los datos y relaciones/funciones se describen en detalle y sus propiedades se expresan en lógica de primer orden. La semántica de los lenguajes está basada en la teoría de conjuntos. Los métodos de este tipo más conocidos son: Z, VDM y B.
 - **Especificaciones algebraicas:** proponen una descripción de estructuras de datos estableciendo tipos y operaciones sobre esos tipos. Para cada tipo se define un conjunto de valores y operaciones sobre dichos valores. Las operaciones de un tipo se definen a través de un conjunto de axiomas o ecuaciones que especifican las restricciones que deben satisfacer las operaciones. Métodos más conocidos: Larch, OBJ, TADs.
 - **Métodos de Verificación:** Entre los métodos de verificación más utilizados, se encuentran:
 - **Aserciones E/S:** Basado en la lógica de Hoare. El programa, en lógica de Hoare, se especifica mediante aserciones que relacionan las entradas y salidas del programa. Se garantiza que si la entrada actual satisface las restricciones de entrada (precondiciones) la salida satisface las restricciones de salida (post-condiciones). Se utiliza una expresión del tipo $P \{ \text{programa} \} Q$, siendo P y Q aserciones de la lógica, para indicar que si P es cierto antes de la ejecución del programa y dicho programa termina, entonces Q es cierto tras la ejecución de dicho programa. Este método permite tanto la corrección parcial como total de los programas.
 - **Precondition más débil:** Básicamente, consiste en dada una post-condición POST, encontrar, operando hacia atrás, un programa S tal que $WP(S, POST)$ (la precondition) se satisfaga en un amplio conjunto de situaciones.
 - **Inducción estructural:** La inducción estructural es una técnica de verificación formal que se basa en el principio de inducción matemática.
 - **Conceptos de los Métodos Formales**

- **Estado:** En el contexto de métodos formales, un estado es el dato almacenado al cual accede el sistema y que es alterado por éste.
- **Precondición:** Relación que se debe cumplir con los datos de entrada para que la operación se pueda aplicar.
- **Post-condición:** Relaciones que se cumplen después de ejecutar la operación. No debe incluir detalles de implementación.
- **Matemáticas en el desarrollo del software:** Propiedades de las matemáticas útiles para quienes desarrollan sistemas: Describir de forma concisa y exacta una situación física, un objeto o el resultado de una acción, se debería presentar una especificación matemática de un sistema, y elaborar la solución en base a una arquitectura de software que implemente la especificación, proporcionan una transición suave entre las actividades de ingeniería del software, admite la abstracción y es un medio excelente para el modelado. Dado que es un medio exacto, hay pocas probabilidades de ambigüedad, y las especificaciones se pueden verificar matemáticamente para descubrir contradicciones e incompletitud; y, por último, la vaguedad desaparece completamente, utilizar para representar niveles de abstracción en la especificación de sistema de forma organizada.
- **Preliminares Matemáticos:** conocimiento de notación matemática (conjunto, sucesiones y la notación lógica que se emplea en el cálculo de predicados)
 - **Conjuntos y especificación constructiva:** Un conjunto es una colección de objetos o elementos (piedra angular). Los elementos de un conjunto son únicos (es decir, no se permiten los duplicados). Forman un grupo pequeño de elementos dentro de llaves, separando mediante comas sus elementos. El orden en que aparecen los elementos dentro de un conjunto no es relevante. El número de elementos del conjunto se conoce con el nombre de cardinalidad. Un conjunto especial es el conjunto vacío $\emptyset$. En matemáticas normales este operador se corresponde con el cero. El conjunto vacío tiene la propiedad de que es un subconjunto de todos los conjuntos restantes. Hay dos formas de definir los conjuntos, una es por enumeración de los elementos y otra es por especificaciones constructivas empleando expresiones booleanas, definiendo con reglas explícitas la construcción del conjunto. ($n : N \mid n < 3$)
 - **Operadores de Conjuntos:** $\in$ Pertenece $\cup$ Unión $\cap$ Intersección $\subset$ Incluido $\setminus$ diferencia
 - **Operadores Lógicos:** Otro componente importante de los métodos formales es la lógica: el álgebra de las expresiones verdaderas y falsas. Los operadores matemáticos equivalentes son los siguientes:
 $\wedge$ y $\vee$ o $\neg$ negación $\rightarrow$ implica
 - **Cuantificación Universal:** es una manera de confeccionar una afirmación acerca de los elementos del conjunto que resulta ser verdadera para todos los miembros de ese conjunto. $\forall$ Para todo ...
 - **Sucesiones:** Una sucesión es una estructura matemática que modela el hecho consistente en que sus elementos están ordenados. Una sucesión S es un conjunto de parejas cuyos elementos oscilan entre 1 y el elemento de mayor número. Los objetos que forman los primeros elementos de las parejas se conocen como dominio de una sucesión, y la colección de segundos elementos como intervalo de la sucesión. A las sucesiones se les puede aplicar todos los operadores de conjuntos. La concatenación es un operador binario que forma una sucesión que se construye añadiendo el segundo operador al final de su primer operador.
 - **Otros operadores:** se pueden aplicar a las sucesiones los siguientes operadores: cabeza (primer elemento), cola (todos menos el primer elemento), ultimo (último elemento), frente (todos menos el último elemento)
- **Lenguajes Formales de Especificación:** un lenguaje de especificación formal suele estar compuesto de tres componentes primarios:
 - **una sintaxis** que define la notación específica con la cual se representa la especificación;
 - **una semántica** que ayuda a definir un «universo de objetos» que se utilizará para describir el sistema;
 - **un conjunto de relaciones** que definen las reglas que indican cuáles son los objetos que satisfacen correctamente la especificación.
- **Uso del lenguaje Z:** Las especificaciones en Z se estructuran como un conjunto de esquemas. Un esquema es, en esencia, una especificación formal análoga a la subrutina o el procedimiento de un lenguaje de programación. Del mismo modo que los procedimientos y las subrutinas se utilizan para estructurar un sistema, los esquemas se utilizan para estructurar una especificación formal. Uso del lenguaje Z Las especificaciones en Z se estructuran como un conjunto de esquemas. Un esquema es, en esencia, una especificación formal análoga a la subrutina o el procedimiento de un lenguaje de programación. Del mismo

modo que los procedimientos y las subrutinas se utilizan para estructurar un sistema, los esquemas se utilizan para estructurar una especificación formal.

- **Ingeniería de del Software de Sala Limpia:** Propuesta por primera vez para la Ingeniería del Software por parte de Mills y sus colegas durante los años 80. La filosofía que subyace tras la ingeniería del software de sala limpia consiste en evitar la dependencia de costosos procesos de eliminación de defectos mediante la escritura de incrementos de código desde un primer momento, y mediante la verificación de su corrección antes de las pruebas. Su modelo de proceso incluye la certificación estadística de calidad de los incrementos de código, a medida que estos se van añadiendo en el sistema. La utilización integrada de IS convencional, métodos formales, verificación de programas (demostración de corrección) y estadística SQA se combinan en una técnica que pueda producir un producto de calidad.
- **Enfoque de la Sala Limpia:** La filosofía de la «sala limpien en las técnicas de fabricación de hardware es en realidad algo bastante sencillo: Se trata de una forma rentable y eficiente, en términos de tiempo, de establecer un enfoque de fabricación que impida la introducción de defectos de producción. En lugar de fabricar un producto y dedicarse después a eliminar defectos, el enfoque de sala limpia demanda la disciplina necesaria para eliminar errores en las especificaciones y en el diseño, fabricando entonces el producto de forma «limpia».
- **Estrategia de la Sala Limpia:** El enfoque de sala limpia hace uso de una versión especializada del modelo incremental de software. Se desarrolla un «cauce de incrementos de software» por parte de equipos de ingeniería del software pequeños e independientes. A medida que se va certificando cada incremento, se integra en el todo. Consiguientemente, la funcionalidad del sistema va creciendo con el tiempo.
- **Tareas que se realizan**
 - **Planificación de Incrementos:** Se desarrolla un plan de proyecto que adopta la estrategia incremental. Se van estableciendo las funcionalidades de cada uno de los incrementos, su tamaño estimado y un plan de desarrollo de sala limpia. Es preciso tener especial cuidado para asegurar que los incrementos certificados se vayan integrando de forma temporalmente oportuna.
 - **Recolección de Requisitos:** Mediante el uso de técnicas, se desarrolla una descripción más detallada de requisitos del nivel del usuario (para cada incremento).
 - **Especificación de la Estructura de Cajas:** Se utiliza un método de especificación que hace uso de estructuras de caja para describir la especificación funcional. Ajustado a los principios de análisis operacional, la estructura de caja «aísla y separa la definición creativa del comportamiento, de los datos, y de los procedimientos para cada nivel de refinamiento».
 - **Caja negra.** Esta caja especifica el comportamiento del sistema, o de parte de un sistema. El sistema (o parte de él) responde a estímulos específicos (sucesos) mediante la aplicación de un conjunto de reglas de transición que hacen corresponder el estímulo con la respuesta.
 - **Caja de estado.** Esta caja encapsula los datos de estados y de servicios (operaciones) de forma análoga a los objetos. En esta vista de especificación, se representan las entradas de la caja de estados (los estímulos) y sus salidas (las respuestas). La caja de estados también representa la «historia de estímulos» de la caja
 - **negra,** es decir, los datos encapsulados en la caja de estado que deben ser mantenidos entre las transiciones implicadas
 - **Caja limpia.** Las funciones de transición que están implicadas en la caja de estado se definen en la caja limpia. Dicho literalmente, la caja limpia contiene el diseño procedimental correspondiente a la caja de estados.
 - **Diseño formal:** Mediante el uso del enfoque de estructura de cajas, el diseño de sala limpia es una extensión natural y sin discontinuidades de la especificación.
 - **Verificación de corrección:** El equipo de sala limpia lleva a cabo una serie de rigurosas actividades de verificación de corrección aplicadas primero al diseño y después al código.
 - **Generación de código, inspección y verificación:** Las especificaciones de estructura de caja, que se representan mediante un lenguaje especializado, se traducen al lenguaje de programación adecuado. Se

utilizan entonces técnicas estándar de recorrido o de inspección para asegurar el cumplimiento semántico de las estructuras de código y de cajas, y la corrección sintáctica de código.

- **Planificación de la comprobación estadística:** La utilización estimada del software se analiza, se planifica y se diseña un conjunto de casos de prueba que ejerciten la «distribución de probabilidad» de esa utilización, esta actividad de sala limpia se realiza en paralelo con la especificación, la verificación y la generación de código.
- **Comprobación estadística de utilización:** Las técnicas estadísticas de utilización ejecutan una colección de pruebas derivadas de una muestra estadística de todas las posibles ejecuciones del programa por parte de todos los usuarios de una cierta población objetivo.
- **Certificación:** Una vez que se ha finalizado la verificación, la inspección y la comprobación de utilización (y después de corregir todos los errores) se certifica el incremento como preparado para su integración.

➤ **Ingeniería Inversa:** El objetivo de la **ingeniería inversa** es obtener información o un diseño a partir de un producto, con el fin de determinar de qué está hecho, qué lo hace funcionar y cómo fue fabricado. La ingeniería inversa es un método de resolución. Aplicar ingeniería inversa a algo supone profundizar en el estudio de su funcionamiento, hasta el punto de que podamos llegar a entender, modificar y mejorar dicho modo de funcionamiento. La ingeniería inversa es un método de resolución. Aplicar ingeniería inversa a algo supone profundizar en el estudio de su funcionamiento, hasta el punto de que podamos llegar a entender, modificar y mejorar dicho modo de funcionamiento.

➤ **Reingeniería del Software:** Reingeniería del software se puede definir como: “modificación de un producto software, o de ciertos componentes, usando para el análisis del sistema existente técnicas de Ingeniería Inversa y, para la etapa de reconstrucción, herramientas de Ingeniería Directa, de tal manera que se oriente este cambio hacia mayores niveles de facilidad en cuanto a mantenimiento, reutilización, comprensión o evaluación.” Beneficios de aplicar reingeniería: Pueden reducir los riesgos evolutivos de una organización. Puede ayudar a las organizaciones a recuperar sus inversiones en software. Puede hacer el software más fácilmente modificable Amplía las capacidades de las herramientas CASE. Es un catalizador para la automatización del mantenimiento del software. Puede actuar como catalizador para la aplicación de técnicas de inteligencia artificial para resolver problemas de reingeniería.

➤ **Sistemas en Tiempo Real (STR):** Cualquier sistema en el que el tiempo en el que se produce la salida es significativo. Esto es generalmente porque la entrada corresponde a algún movimiento en el mundo físico, y la salida está relacionada con dicho movimiento. El intervalo entre el tiempo de entrada y el de salida debe ser lo suficientemente para un a temporalidad aceptable.

- **Young (1982):** Cualquier actividad o sistema de proceso de información que tiene que responder a un estímulo de entrada generado externamente en periodo finito y especificado.
- **Randell (1995):** Un sistema de tiempo real es aquel a1 que se le solicita que reaccione a estímulos del entorno (incluyendo el paso de tiempo físico) en intervalos del tiempo dictados por el entorno.

- **Tipos de Sistemas:**

- **Sistemas en Tiempo Real Estrictos (hard):** Los sistemas de tiempo real estrictos son aquéllos en los que es absolutamente imperativo que las respuestas se produzcan dentro del tiempo límite especificado.
- **Sistemas en tiempo Real No Estrictos (soft):** Los sistemas de tiempo real no estrictos son aquéllos en los que los tiempos de respuesta son importantes pero el sistema seguirá funcionando correctamente aunque los tiempos límite no se cumplan ocasionalmente. Los sistemas no estrictos se pueden distinguir de los interactivos, en los que no hay tiempo límite explícito.

- **Ejemplos de STR:**

- **Sistemas de Control de Procesos:** Controlan sistemas industriales externos que requieren una respuesta en tiempo finito y predecible, y dependen de que se dé la respuesta en ese tiempo.
- **Sistemas de Control de Fabricación:** Coordinan conjuntos de dispositivos para la fabricación de productos. Componentes: sistema de control, dispositivos (manipuladores, cintas transportadoras, etc.). Si se usan

sistemas informáticos, se denominan CIM (Computer Integrated Manufacturing). Se persigue reducir costes y aumentar la productividad.

- **Sistemas de Comunicación, Mando y Control:** Constituidos por sistemas de toma de decisiones conectados en red. Utilidades: tráfico aéreo, transacciones bancarias, reserva de billetes, gestión de energía, etc.).
- **Sistemas Empotrados:** El sistema de control forma parte del sistema a controlar. Normalmente sólo proveen entornos de ejecución, no de depuración ni desarrollo. El diseño debe incluir el hardware concreto a controlar. Diseño multidisciplinar. Presencia de reloj(es) de tiempo real. Potentes mecanismos de E/S e interrupciones y pocas facilidades de propósito general.
- **Tareas de tiempo real:** las actividades de un sistema de tiempo real se llaman “tareas”, tienen varios tipos de propiedades, funcionales (que hacen), temporales (cuando lo hacen), fiabilidad, seguridad, etc., el comportamiento temporal de las tareas se especifica mediante sus atributos temporales, cuando se ejecutan (esquema de activación) y que plazo tienen para ejecutar cada acción.
- **Características de los Sistemas en Tiempo Real**
 - **Concurrencia:** se componen de un conjunto de actividades realizándose en paralelo: el mundo real es concurrente, así que los S.T.R. deben serlo. Además, es necesaria para tratar con múltiples bucles de control simultáneamente.
 - **Dependencia del Tiempo:** se ejecutan tareas en respuesta a señales externas, o periódicamente.
 - **Fiabilidad y Tolerancia a Fallos:** deben funcionar aún en presencia de fallos o averías parciales (normalmente mediante elementos redundantes), ya que es imposible prever completamente el comportamiento del mundo real.
 - **Interacción con el Hardware:** para la conexión de la CPU con el exterior. Interacciones activas y pasivas:
 - Activa, el sistema decide cuándo accede al exterior o cuándo el exterior puede devolver información. A través de Interfaces de Entrada y Salida.
 - Pasiva, el sistema es interrumpido en el momento en que el exterior tenga algo que decir. A través de interrupciones, que pueden ser Simples o Múltiples.
 - **Manipulación de Números Reales:** adquiridos del exterior mediante interfaces.
 - **Eficiencia:** es deseable una implementación eficiente (que consuma pocos recursos), sobre todo en sistemas empotrados, debido al pequeño tamaño y al deseable ahorro de costes.
 - **Complejidad:** suelen ser complejos y por lo tanto es deseable usar técnicas modulares. Entra en conflicto con la eficiencia.
 - **Involucran todos los aspectos de la Ingeniería Informática:** el tiempo real debe estar contemplado en el Hardware (CPU, circuitería, redes) y en el Software (S.O., lenguajes, aplicaciones).
 - **Requisitos Generales del Hardware:** Tolerancia a Fallos (excepciones, multiproceso, etc.), Protección de Memoria para Concurrencia, Relojes de Tiempo Real y Temporizadores, Proceso en Coma Flotante (Números Reales), Controladores de Interrupciones, Interfaces de E/S,
 - **Requisitos Generales del Software:** Planificación de Tareas para Tiempo Real, Concurrencia, Mecanismos de Temporización, Mecanismos para la Portabilidad, Capacidad de Interrupciones, Acceso a E/S, Tolerancia a Fallos (excepciones, redundancia, etc.)
- **Diseños de STR:** La etapa más importante del desarrollo de cualquier sistema de tiempo real es la generación de un diseño consistente que satisfaga una especificación acreditada de los requisitos.
- **Niveles de Notación:** Hay varios modos de clasificar las formas de notación. McDermid (1989) proporciona una útil descomposición para nuestros fines y destaca tres técnicas:
 - **Los métodos informales** suelen hacer uso del lenguaje natural y de diversos tipos de diagramas imprecisos.
 - **Los métodos estructurados** suelen emplear notación gráfica bien definidos. Dichos esquemas se construyen a partir de un pequeño número de componente, predefinidas que pueden conectarse de una forma controlada. La forma gráfica puede tener también una representación sintáctica en algún lenguaje bien definido.
 - **Métodos formales:** A pesar de que pueden idearse métodos estructurados muy rigurosos, no pueden, en sí, ser analizados o manipuladas. Para poder efectuar este tipo de operaciones, la notación debe tener una base matemática. Los métodos que presentan dichas propiedades matemáticas se conocen como

métodos formales. Éstos tienen la ventaja evidente de que mediante estas notaciones es posible realizar descripciones precisas.

- **Visión General**

- **Especificación de requisitos:** Es en esta etapa donde se define la funcionalidad del sistema. En cuanto a los factores concretos de tiempo real, deberá hacerse suficientemente explícito el comportamiento temporal del sistema, así como los requisitos de fiabilidad y el comportamiento deseado del sistema en caso de fallos en los componentes. En estos análisis se usan técnicas estructuradas, como PSL y CORE. También son populares otras técnicas, como UML (orientado a objetos), etc. La más popular es VDM.

- **Actividades de Diseño:**

- **Descomposición:** Se descompone el sistema en componentes lo suficientemente pequeños como para que puedan ser comprendidos y abordados por individuos o pequeños equipos.
- **Abstracción:** Permite dejar para más adelante los detalles (especialmente los de implementación), lo que permite simplificar la visión del sistema.

Si se usase una notación formal para la especificación de requisitos, debe usarse también para el diseño de alto nivel (para demostrar que cumplen la especificación).

- **Encapsulamiento:** Los componentes de un sistema tienen funciones bien definidas y un grupo de interfaces claras. Si la especificación puede verificarse por la verificación de los subcomponentes inmediatos sin necesitar nada más; se dice que es una descomposición composicional. Podemos encontrar diversos lenguajes, como Modula-2, C++, Java, Eiffel, Ada...
- **Cohesión:** Se conoce como cohesión al grado de unión interna de un módulo; es decir, a la relación existente entre sus funciones.
- **Acoplamiento:** Se conoce como acoplamiento a la interdependencia entre los módulos de un programa. Conviene tener una alta cohesión y un bajo acoplamiento.
- **Aproximaciones Formales:** Se pueden utilizar redes sitio-transición (Place-Transition) (Brauer, 1980) C. A. Petri, que consiste en grafos con dos tipos de nodos: S para estados atómicos locales y T para transiciones; y que están relacionados por arcos. Aunque puede ser una representación compleja, su capacidad de abstracción y visualización gráfica lo convierten en un buen marco de trabajo. Hay pocos lenguajes de implementación con una descripción formal. Tenemos, por ejemplo, el CSP, en el que cada proceso se describe en términos de eventos externos. Dos propiedades importantes en la comprobación de modelos: la seguridad (asegurarse de que no va a fallar) y la vivacidad (asegurarse de que los resultados van a ser positivos). Los STR también tienen requisitos de temporalidad, por lo que se requiere el uso de la lógica temporal, con operadores como siempre, en-algún-momento, hasta, desde... Suele decirse que la lógica temporal requiere del programa completo para poder usarla; es por ello que suele utilizarse más en la verificación de programas existentes que en la especificación y desarrollo de otros nuevos. Aunque lo ideal sería usar técnicas formales, no están lo suficientemente maduras como para considerarse métodos probados y contrastados. En su lugar, suelen utilizarse métodos estructurados.
- **Métodos de Diseño:** Ya se comentó que la mayoría de los diseñadores de sistemas de tiempo real promueven un proceso de abstracción de objetos, y que existen técnicas formales que permiten especificar y analizar sistemas concurrentes con restricciones temporales. No cabe duda de que estas técnicas no están aun suficientemente maduras para constituir métodos de diseño probados y contrastados. Más bien, la industria de tiempo real usa, en el mejor de los casos, métodos estructurados y aproximaciones de ingeniería de software aplicables a la mayoría de los sistemas de procesamiento de información. Estos métodos no proporcionan un soporte específico para el dominio de tiempo real; y carecen -de-la riqueza necesaria para explotar completamente la potencia, de los lenguajes de implementación. Métodos de diseño estructurados orientados a los sistemas de tiempo real: MASCOT, JSD, Yourdon, MOON, OOD, RTSA, HOOD, DARTS, ADARTS, CODARTS, EPOS, MCSE, PAMELA, HRT-HOOD, Octopiis, etc.

- **Implementación**

- **Ensamblador:** Solía utilizarse inicialmente, ya que los lenguajes de alto nivel no tenían suficiente soporte para la mayoría de microcomputadores. Lo malo es que el ensamblador es un tipo de lenguaje orientado a la máquina. Es decir, no se puede llevar fácilmente de una máquina a otra, hace falta reescribirlo (con todas las complicaciones que conlleva).

- **Lenguajes de Implementación de Sistemas Secuenciales:** Los lenguajes de alto nivel comenzaron a tener más ventajas que inconvenientes. Aparecieron lenguajes nuevos específicos para la programación embebida, como Jovial o Coral66; y más recientemente C y C++. Son lenguajes secuenciales y tienen carencias de control y fiabilidad para tiempo real, por lo que suele ser necesario basarse en el soporte del SO y en fragmentos de ensamblador.
- **Lenguajes de Programación Concurrente de Alto Nivel:** Se buscaba un lenguaje de alto nivel que pueda servir para todas las aplicaciones; por lo que nació Ada, que fue actualizado una década después para aportar la experiencia de esos años transcurridos. Hay otros lenguajes, como Modula-1 y sus actualizaciones. Posteriormente aparecieron más: PEARL, CHILL, Java, Mesa... Existe también OCCAM, que es mucho más reducido y no da soporte a sistemas grandes y complejos.
- **Criterios Generales de Diseño de Lenguajes:** Los lenguajes para el diseño de sistemas embebidos no suelen limitarse a esos: también se usan para la implementación de sistemas para aplicaciones, como compiladores y SSOO. Podemos distinguir 6 criterios que deben seguir los lenguajes de tiempo real:
 - **Seguridad:** Consiste en el grado en que el compilador o el sistema de soporte en tiempo de ejecución pueden detectar los errores de programación, con los límites de sentido común.
 - **Legibilidad:** Las palabras clave, la facilidad de definición de tipos, mecanismos de modularización, etc. son determinantes para un lenguaje legible. Reduce los costes de la documentación, aporta seguridad y es más fácil de mantener. Puede provocar un aumento de la longitud de los programas.
 - **Flexibilidad:** El programador debe poder expresar directamente todas las operaciones oportunas, sin tener que recurrir a código máquina o al SO.
 - **Simplicidad:** Cuanto más simple sea, más sencillo es producir compiladores, menor es el coste de formación de programadores y evita en mayor medida errores de programación por malas interpretaciones en el lenguaje.
 - **Portabilidad:** Un ejemplo claro es Java. Es bueno que sea independiente del hardware sobre el que se ejecuta. En los STR es complicado, ya que estos programas se involucran en la manipulación de recursos hardware.
 - **Eficiencia:** Ya dijimos que un STR debe garantizar una respuesta dentro de los intervalos marcados. Deben evitarse sobrecargas que retarden esta respuesta. Es evidente que las pruebas de un STR deben ser muy exigentes y estrictas. Sin embargo, los errores más comunes de los STR suelen provocarse por interacciones sutiles entre procesos y en situaciones difíciles muy concretas de reproducir durante las pruebas.
- **Prueba:** Las pruebas no se hacen sólo sobre el sistema final, sino también sobre el diseño y sus módulos. No sólo hay que comprobar el sistema en el entorno correcto, sino también en entornos incorrectos e imprevisibles.
 - **Simuladores** Son los entornos de prueba para el software. Es un programa que imita las acciones que hará el sistema final. Pueden crear situaciones de ejecución normales y anormales (e incluso situaciones presuntamente imposibles). Para los STR, sería conveniente un simulador multiprocesador, y a veces es imposible construir uno adecuado por la complejidad del programa en cuestión. Los simuladores de STR se consideran sistemas STR por sí mismos, lo que implica que también deben ser probados.
- **Prototipado:** El prototipo debe tener un coste significativamente menor que el sistema final, utilizando lenguajes de más alto nivel (por ejemplo). Un lenguaje popular para el prototipado es el APL. Los prototipos no tienen en cuenta los aspectos del tiempo real de la aplicación; para ello se necesita una simulación, como ya se vio antes.
- **Interacción Hombre-Máquina:** Es importante la modularidad. Es decir, las actividades de HCI deben estar aisladas y con interfaces bien definidas. Además, la interacción debe estar controlada, por ejemplo para evitar órdenes humanas peligrosas o no autorizadas. La mayoría de STR se conoce como sistemas de iniciativa mixta; es decir, en ocasiones el computador tiene el control y otras veces el operador. Se intenta que todos los errores derivados del usuario se capturen en el software de control de la interfaz y no en el de aplicación o el resto del sistema.
- **Gestión del Diseño:** Debe asegurarse la aplicación correcta de las técnicas necesarias. Existen herramientas de apoyo para realizar todas estas comprobaciones, pero tienen que tener una calidad extremadamente alta. Recuérdese que estas herramientas no sustituyen al equipo humano; deben aplicarse rigurosamente las técnicas para producir STR de calidad y asequibles. Hay que evitar los métodos informales y los lenguajes inapropiados de bajo nivel.

➤ **Introducción al Diseño**

- **Diseño de datos:** El diseño de datos transforma el modelo del dominio de información que se crea durante el análisis en las estructuras de datos que se necesitarán para implementar el software. Los objetos de datos y las relaciones definidas en el diagrama relación entidad y el contenido de datos detallado que se representa en el diccionario de datos proporcionan la base de la actividad del diseño de datos.
- **Diseño arquitectónico:** El diseño arquitectónico define la relación entre los elementos estructurales principales del software, los patrones de diseño que se pueden utilizar para lograr los requisitos que se han definido para el sistema, y las restricciones que afectan a la manera en que se pueden aplicar los patrones de diseño arquitectónicos.
- **Diseño de la interfaz:** El diseño de la interfaz describe la manera de comunicarse el software dentro de sí mismo, con sistemas que inter - operan dentro de él y con las personas que lo utilizan. Los diagramas de flujo de control y de datos proporcionan gran parte de la información que se requiere para el diseño de la interfaz.
- **Diseño a nivel de componentes:** El diseño a nivel de componentes transforma los elementos estructurales de la arquitectura del software en una descripción procedimental de los componentes del software. La información que se obtiene de Especificaciones de Procesos y de Diagrama de Transición de Estados sirve como base para el diseño de los componentes.
 - **La capa subsistema:** Contiene una representación de cada uno de los subsistemas, para permitir al software conseguir sus requisitos definidos por el cliente e implementar la infraestructura que soporte los requerimientos del cliente.
 - **La capa de clases y objetos:** Contiene la jerarquía de clases, que permiten al sistema ser creado usando generalizaciones y cada vez especializaciones más acertadas.
 - **La capa de mensajes:** Contiene detalles de diseño, que permite a cada objeto comunicarse con sus colaboradores. Esta capa establece interfaces externas e internas para el sistema.
 - **La capa de responsabilidades:** Contiene estructuras de datos y diseños algorítmicos, para todos los atributos y operaciones de cada objeto.
- **Diseño y calidad del software:** Características que sirven como guía para la evaluación de un buen diseño: El diseño deberá implementar todos los requisitos explícitos del modelo de análisis, y deberán ajustarse a todos los requisitos implícitos que desea el cliente. El diseño deberá ser una guía legible y comprensible para aquellos que generan código y para aquellos que comprueban y consecuentemente, dan soporte al software. El diseño deberá proporcionar una imagen completa del software, enfrentándose a los dominios de comportamiento, funcionales y de datos desde una perspectiva de implementación.

«El comienzo de la sabiduría para un ingeniero del software es reconocer la diferencia entre hacer que un programa funcione y conseguir que lo haga correctamente» M.A. Jackson.

➤ **Conceptos de Diseño**

- **Abstracción:** Cuando se tiene en consideración una solución modular a cualquier problema, se pueden exponer muchos niveles de abstracción. En el nivel más alto de abstracción, la solución se pone como una medida extensa empleando el lenguaje del entorno del problema. En niveles inferiores de abstracción, se toma una orientación más procedimental. Finalmente, en el nivel más bajo de abstracción, se establece la solución para poder implementarse directamente.
 - **Abstracción procedimental:** es una secuencia nombrada de instrucciones que tiene una función específica y limitada.
 - **Abstracción de datos:** es una colección nombrada de datos que describe un objeto de datos.
 - **La abstracción de control:** este tipo de abstracción implica un mecanismo de control de programa sin especificar los datos internos.
- **Refinamiento:** En cada paso (del refinamiento), se descompone una o varias instrucciones del programa dado en instrucciones más detalladas. Esta descomposición sucesiva o refinamiento de especificaciones termina

cuando todas las instrucciones se expresan en función de cualquier computadora subyacente o de cualquier lenguaje de programación. Todos los pasos del refinamiento implican decisiones de diseño.

- **Modularidad:** La arquitectura de computadora expresa la modularidad; es decir, el software se divide en componentes nombrados y abordados por separado, llamados frecuentemente módulos, que se integran para satisfacer los requisitos del problema. Se ha afirmado que la modularidad es el único atributo del software que permite gestionar un programa intelectualmente. Un buen método de diseño permite, la descomposición modular, el empleo de componentes modulares, capacidad de compresión modular, continuidad modular y protección modular.
- **Arquitectura del Software:** La arquitectura del software alude a la «estructura global del software y a las formas en que la estructura proporciona la integridad conceptual de un sistema» En su forma más simple, la arquitectura es la estructura jerárquica de los componentes del programa (módulos), la manera en que los componentes interactúan y la estructura de datos que van a utilizar los componentes, en un sentido más amplio, los «componentes» se pueden generalizar para representar los elementos principales del sistema y sus interacciones. Se ha desarrollado un conjunto de lenguajes de descripción arquitectónica (LDAs) para representar los siguientes modelos. Diseño Arquitectónico, su representación:
 - **Modelos estructurales:** representan la arquitectura como una colección organizada de componentes de programa.
 - **Modelos del marco de trabajo:** aumentan el nivel de abstracción del diseño en un intento de identificar los marcos de trabajo (patrones) repetibles del diseño arquitectónico que se encuentran en tipos similares de aplicaciones.
 - **Modelos dinámicos:** tratan los aspectos de comportamiento de la arquitectura del programa, indicando cómo puede cambiar la estructura o la configuración del sistema en función de los acontecimientos externos.
 - **Modelos de proceso:** se centran en el diseño del proceso técnico de negocios que tiene que adaptar el sistema.
 - **Modelos funcionales:** se pueden utilizar para representar la jerarquía funcional de un sistema.
- **Jerarquía de control:** La jerarquía de control, denominada también estructura de programa, representa la organización de los módulos e implica una jerarquía de control. La jerarquía de control también representa dos características sutiles diferentes de la arquitectura del software: visibilidad y conectividad.
 - **La visibilidad** indica el conjunto de componentes de programa que un componente dado puede invocar o utilizar como datos, incluso cuando se lleva a cabo indirectamente.
 - **La conectividad** indica el conjunto de componentes que un componente dado invoca o utiliza directamente como datos. Por ejemplo, un módulo que hace directamente que otro módulo empiece la ejecución está conectado a él.
- **División estructural**
 - **División Horizontal:** define tres particiones -entrada, procesamiento y salida. La división horizontal de la arquitectura proporciona diferentes ventajas: proporciona software más fácil de probar, conduce a un software más fácil de mantener, propaga menos efectos secundarios, proporciona software más fácil de ampliar. En la parte negativa la división horizontal suele hacer que los datos pasen a través de interfaces de módulos y que puedan complicar el control global del flujo del programa.
 - **División vertical:** frecuentemente llamada factorización, sugiere que dentro de la estructura de programa el control y el trabajo se distribuyan de manera descendente. Los módulos del nivel superior deberán llevar a cabo funciones de control y no realizarán mucho trabajo de procesamiento. Los módulos que residen en la parte inferior de la estructura deberán ser los trabajadores, aquellos que realizan todas las tareas de entrada, proceso y salida.
- **Estructura de datos:** La estructura de datos es una representación de la relación lógica entre elementos individuales de datos, dicta las alternativas de organización, métodos de acceso, grado de capacidad de asociación y procesamiento de la información final, la estructura de datos es tan importante como la estructura de programa para la representación de la arquitectura del software. Las estructuras de datos se pueden representar a diferentes niveles de abstracción.
- **Procedimiento de software:** La estructura de programa define la jerarquía de control sin tener en consideración la secuencia de proceso y de decisiones. El procedimiento de software se centra en el procesamiento de cada módulo individualmente. El procedimiento debe proporcionar una especificación

precisa de procesamiento, incluyendo la secuencia de sucesos, los puntos de decisión exactos, las operaciones repetitivas e incluso la estructura/organización de datos.

➤ **Diseño Modular Efectivo:** La modularidad se ha convertido en un enfoque aceptado en todas las disciplinas de ingeniería. Un diseño modular reduce la complejidad, facilita los cambios, y da como resultado una implementación más fácil al fomentar el desarrollo paralelo de las diferentes partes de un sistema.

- **Independencia Funcional:** El concepto de independencia funcional es la suma de la modularidad y de los conceptos de abstracción y ocultación de información. La independencia funcional es la clave para un buen diseño y el diseño es la clave para la calidad del software. La independencia se mide mediante dos criterios cualitativos: la cohesión y el acoplamiento.

- **Cohesión:** La cohesión es una extensión natural del concepto de ocultación de información. Un módulo cohesivo lleva a cabo una sola tarea dentro de un procedimiento de software, lo cual requiere poca interacción con los procedimientos que se llevan a cabo en otras partes de un programa. Dicho de manera sencilla, un módulo cohesivo deberá (idealmente) hacer una sola cosa. La cohesión tiene que ver con que cada módulo del sistema se refiera a un único proceso o entidad. A mayor cohesión, mejor: el módulo en cuestión será más sencillo de diseñar, programar, probar y mantener.
- **Acoplamiento:** El acoplamiento es una medida de interconexión entre módulos dentro de una estructura de software. El acoplamiento depende de la complejidad de interconexión entre los módulos, el punto donde se realiza una entrada o referencia a un módulo, y los datos que pasan a través de la interfaz. En el diseño del software, intentamos conseguir el acoplamiento más bajo posible. Una conectividad sencilla entre los módulos da como resultado un software más fácil de entender y menos propenso a tener un «efecto ola» causado cuando ocurren errores en un lugar y se propagan por el sistema.

➤ **Documentación del Diseño**

- **La Especificación del diseño:** establece que todos los requisitos se satisfagan mediante el diseño del software e indica cuales son los componentes críticos para la implementación de requisitos específicos.
- **La documentación de pruebas:** también se encuentra dentro del documento del diseño. Una vez que se han establecido las interfaces y la estructura de programa podremos desarrollar las líneas generales para comprobar los módulos individuales y la integración de todo el paquete.
- **Las restricciones de diseño:** tales como limitaciones físicas de memoria o la necesidad de una interfaz externa especializada, podrán dictar requisitos especiales para ensamblar o empaquetar el software.
- **Consideraciones especiales:** originadas por la necesidad de superposición de programas, gestión de memoria virtual, procesamiento de alta velocidad u otros factores podrán originar modificaciones en diseño derivadas del flujo o estructura de la información. También se describe el enfoque que se utilizará para transferir software al cliente.
- **La última sección:** de la Especificación del diseño contiene datos complementarios. También se presentan descripciones de algoritmos, procedimientos alternativos, datos tabulares, extractos de otros documentos y otro tipo de información relevante, todos mediante notas especiales o apéndices separados. Será aconsejable desarrollar un Manual preliminar de Operaciones e instalación e incluirlo como apéndice para la documentación del diseño.

➤ **Diseño Arquitectónico:** El diseño construye representaciones coherentes y bien planificadas de los programas, concentrándose en las interrelaciones de los componentes de mayor nivel y en las operaciones lógicas implicadas en los niveles inferiores.

- **Arquitectura:** La arquitectura «constituye un modelo relativamente pequeño e intelectualmente comprensible de cómo está estructurado el sistema y de cómo trabajan juntos sus componentes». El estilo y estructura de una aplicación puede depender de los requerimientos no funcionales del sistema, lo que implica que, el Rendimiento, Protección, Seguridad, Disponibilidad y Mantenimiento serán tomados como críticos.

- **Organización del sistema:** La organización de un sistema refleja la estrategia básica usada para estructurar dicho sistema. Deben tomarse decisiones sobre la totalidad del modelo organizacional de un sistema al principio del proceso de diseño arquitectónico.
- **El modelo de repositorio:** Todos los datos compartidos se almacenan en una base de datos central a la que puede acceder por todos los subsistemas. Un modelo de sistema basado en una base de datos compartida se denomina algunas veces modelo de repositorio. Cada subsistema mantiene su propia base de datos. Los datos se intercambian con otros subsistemas mediante el paso de mensajes entre ellos.
- **El modelo de Cliente – Servidor:** La arquitectura cliente-servidor es un modelo de aplicación distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes. Un cliente realiza peticiones a otro programa, el servidor, quien le da respuesta. Esta idea también se puede aplicar a programas que se ejecutan sobre una sola computadora, aunque es más ventajosa en un sistema operativo multiusuario distribuido a través de una red de computadoras.
- **El modelo de capas:** El modelo de capas de una arquitectura (algunas veces denominada modelo de máquina abstracta) organiza el sistema en capas, cada una de las cuales proporciona un conjunto de servicios. Cada capa puede pensarse como una máquina abstracta cuyo lenguaje máquina se define por los servicios proporcionados por la capa.
- **Estilos de Descomposición Modular**
 - **Descomposición orientada a objetos:** en la que se descompone un sistema en un conjunto de objetos que se comunican.
 - **Descomposición orientada a flujos de funciones:** en la que se descompone un sistema en módulos funcionales que aceptan datos y los transforman en datos de salida.
- **Estilos de Control**
 - **Control centralizado:** Un subsistema tiene toda la responsabilidad para controlar, iniciar y detener a otros subsistemas. También puede devolver el control a otro subsistema, pero esperará que le sea devuelta la responsabilidad del control.
 - **El modelo de llamada-retorno:** Es el modelo usual de subrutina descendente en donde el control comienza al inicio de una jerarquía de subrutinas y, a través de las llamadas a subrutinas, el control pasa a niveles inferiores en el árbol de la jerarquía (sistemas secuenciales).
 - **El modelo del gestor:** (sistemas concurrentes). Un componente del sistema se diseña como un gestor del sistema y controla el inicio, parada y coordinación del resto de los procesos del sistema. Un proceso es un subsistema o módulo que puede ejecutarse en paralelo con otros procesos
 - **Control basado en eventos:** En lugar de que la información de control esté embebida en un subsistema, cada subsistema puede responder a eventos generados externamente. Estos eventos podrían provenir de otros subsistemas o del entorno del sistema.
 - **Modelos de transmisión (broadcast):** En estos modelos, un evento se transmite a todos los subsistemas. Cualquier subsistema que haya sido programado para manejar ese evento puede responder a él.
 - **Modelos dirigidos por interrupciones:** Éstos se usan exclusivamente en sistemas de tiempo real, en donde las interrupciones externas son detectadas por un manejador de interrupciones.
- **Arquitectura de Referencia:** Hay dos tipos de modelos arquitectónicos de dominio específico:
 - **Modelos genéricos:** Son abstracciones obtenidas a partir de varios sistemas reales. Encapsulan las características principales de estos sistemas. Por ejemplo, en sistemas de tiempo real, podría haber modelos arquitectónicos genéricos de diferentes tipos de sistemas tales como sistemas de recolección de datos o sistemas de monitorización.
 - **Modelos de referencia:** Son más abstractos y describen una clase más amplia de sistemas. Constituyen un modo de informar a los diseñadores sobre la estructura general de esta clase de sistemas. Los modelos de referencia normalmente se obtienen a partir de un estudio del dominio de la aplicación. Representan una arquitectura ideal que incluye todas las características que los sistemas podrían incorporar.
- **Arquitectura de Sistemas Distribuidos:** Un sistema distribuido es un sistema en el que el procesamiento de información se distribuye sobre varias computadoras en vez de estar confinado en una única máquina.
 - **Ventajas:** Compartición de recursos, apertura en base a protocolos de estándares abiertos, concurrencia, escalabilidad, tolerancia a fallos.

- **Desventajas:** Complejidad, riesgos de seguridad, inconvenientes de manejabilidad, impredecibilidad.
 - **Tipos genéricos de arquitectura de sistemas distribuidos**
 - **Arquitecturas cliente-servidor:** En esta aproximación, el sistema puede ser visto como un conjunto de servicios que se proporcionan a los clientes que hacen uso de dichos servicios. Los servidores y los clientes se tratan de forma diferente en estos sistemas.
 - **Arquitecturas de objetos distribuidos:** En este caso, no hay distinción entre servidores y clientes, y el sistema puede ser visto como un conjunto de objetos que interaccionan cuya localización es irrelevante. No hay distinción entre un proveedor de servicios y el usuario de estos servicios.
 - **Arquitectura Multiprocesador:** El modelo más simple de un sistema distribuido es un sistema multiprocesador en el que el sistema software está formado por varios procesos que pueden (aunque no necesariamente) ejecutarse sobre procesadores diferentes. Este modelo es común en sistemas grandes de tiempo real.
 - **CORBA:** Common Object Request Broker Architecture (CORBA) es un estándar definido por Object Management Group (OMG) que permite que diversos componentes de software escritos en múltiples lenguajes de programación y que corren en diferentes computadoras, puedan trabajar juntos; es decir, facilita el desarrollo de aplicaciones distribuidas en entornos heterogéneos
- **Diseño de Salidas:** La salida es cualquier información o dato útil que proporciona al usuario el Sistema de Información, puede tomar virtualmente cualquier forma, incluyendo: impresora, pantallas, audio y micro formas. En el diseño de la salida, el analista trata de satisfacer sus objetivos principales, ellos son: Un diseño de salida que satisfaga el propósito planteado, diseño de la salida para adaptarse al usuario, proveer la cantidad adecuada de información, que la salida esté disponible donde se requiera, proporcionando oportunamente la salida y elección del método correcto de salida. Es importante que el analista se percate de que el contenido de la salida se relaciona con el método de salida. La salida con diferentes tecnologías afectan de manera diferente al usuario.
- **Información Externa:** La información que sale de la Organización es llamada **externa**, difieren en mecanismo de distribución, su diseño y apariencia.
 - **Información Interna:** La información que permanece dentro de la Organizaciones llamada **interna**, se distribuyen a lo largo de la Organización, normalmente tiene pocos problemas de diseño.
 - **Tipos de Salidas:** Informe Impresos, pantallas (CRT, LCD), Audio, Micro-formas.
 - **Convenciones:** incluye como resaltar el tipo de datos, mostrar las dimensiones exactas, indicar la continuación de los datos, podemos destacar: Información constante, información variable, estimación del ancho del reporte, calidad de papel, tipo y tamaño, formas especiales.
 - **Atributos:** al diseñar el reporte, el analista incorpora elementos funcionales y estéticos para que la información tenga un formato legible.
 - **Funcionales:** incluyen el encabezado, numero de página, fechas, rótulos de columnas, agrupamiento de datos relacionados y el uso de elementos de pausa.
 - **Estéticos:** Debe ser atractivo la lectura del reporte para que este sea utilizado. Lectura de arriba hacia abajo, de izquierda a derecha. Espacios blancos entre columnas. Códigos y/o logotipo.
 - **Pasos para determinar hoja de salida:** Determinar necesidad del reporte, identificar a los usuarios, determinar la información, dimensión global del reporte, título y numero de página, fecha y rotulo de columnas, definir la línea de detalle, revisar el boceto con los usuarios (factibilidad, utilidad, legibilidad, comprensión y apariencia estética)
 - **Lineamientos para un buen diseño de pantalla:** Mantener una pantalla sencilla, conservar consistencia entre una y otra, facilitar el movimiento entre pantallas, pantallas atractivas.
- **Diseño de Entrada:** La calidad de la salida está determinada por la calidad de su entrada. Las pantallas con buen diseño de entrada deben satisfacer las metas de eficiencia, precisión, facilidad de uso, consistencia y atractivo, el conocimiento de los diferentes elementos de diseño permitirán el logro de estos objetivos. Los cuatros lineamientos para un buen diseño de entrada son: Formas fáciles de llenar, Formas que satisfagan el propósito para el cual fueron diseñadas, Asegurar el llenado preciso, Formas atractivas.
- **Diseño de procedimientos precisos de captura de datos:** La codificación eficaz es el proceso de re expresar datos ambiguos o complicados en dígitos o letras que se capturan con facilidad. Como ventajas podemos mencionar, la reducción en el tiempo de captura, facilidad en el ordenamiento, reducción de espacio de

almacenamiento. Permite mantener el seguimiento de algo, la clasificación de la información, el ocultamiento de información en formato cifrado, resalta la información y permite acciones apropiadas a las solicitudes dadas.

- **Lineamientos para la codificación:** ser conciso, estabilizar los códigos, utilizar códigos únicos, utilizar códigos que tengan un cierto orden, evitar códigos confusos, utilizar codificación uniforme, permitir modificaciones y ofrecer un significado lógico.
 - **Captura de datos eficaz y eficiente:** Es importante capturar los datos con efectividad con el fin de asegurar la calidad de los datos que entran al sistema. Existen dos tipos de datos a introducir: Los datos que cambian o varían en cada una de las transacciones y los datos que distinguen de otros artículos, de manera concisa, al artículo particular que se procesa, se debe evitarlos los cuellos de botella. Los métodos de entrada son: Teclas de almacenamiento, Tarjetas, OCR, MICR, etc.
 - **Validación de entrada:** Los errores no pueden evitarse por completo y es sumamente importante detectar los mismos en el momento mismo de la captura. Se debe evaluar si existen datos faltantes, si la longitud de los datos es correcta, si el tipo o composición, rangos son correctos, realizar pruebas para la comparación de datos ya almacenados, entre otros tipos de validaciones.
- **Diseño de Interfaz del Usuario:** Desde el punto de vista de la Ingeniería del Software, la interfaz de usuario juega un papel en extremo importante en el desarrollo y puesta en marcha de todo sistema, es la carta de presentación del sistema y en ocasiones resulta determinante para la aceptación o rechazo de todo un proyecto. La idea fundamental en el concepto de interfaz es la mediación, entre hombre y máquina. La interfaz es lo que “media”, lo que facilita la comunicación, la interacción, entre dos sistemas de diferente naturaleza, típicamente el ser humano y una máquina como el computador, esto implica que es un sistema de traducción debido a que ambas partes manejan lenguajes diferentes. Es posible realizar desarrollo de “prototipos” de Interfaz de Usuario para resolver problemas de requerimientos e inclusive especificar parte del sistema.
- El diseño de la interfaz de usuario es el proceso de determinar los distintos componentes, tanto hardware como software, sus características y su disposición, que se utilizarán para interactuar con una serie de usuarios determinados en un medio ambiente determinado. Se debe tener como objetivo, el diseño de una interfaz que ayude a los usuarios y a sus empresas a obtener e introducir información al sistema. Una interfaz debe ser eficaz ofreciendo acceso al sistema manteniendo la congruencia de requisitos, debe ser eficiente mejorando la velocidad y captura y reduciendo los errores, debe poseer un diseño adecuado y permitir una favorable retroalimentación hacia el usuario y debe ser productiva.
 - Una buena interfaz debe usar términos y conceptos familiares al usuario y al dominio de la aplicación, debe ser uniforme y toda operación comparable debe funcionar de la misma manera, siempre debe comportarse de la forma más predecible posible evitando sorpresas no deseadas. Se deberá hacer hincapié en establecer mensajes informativos, de error, de advertencias de una forma clara y precisa y en lenguaje que sea claramente entendible por el usuario.
- **Diseño de Archivos y de las Bases de Datos:** Los objetivos generales del diseño de la organización de los datos son, la disponibilidad de los datos, el almacenamiento eficiente de los datos, la recuperación dirigida de la información la actualización y recuperación eficientes y la integración de los datos.
- **Almacenamiento Convencional:** almacenar los datos en archivos individuales, exclusivos para una aplicación en particular. Se diseñan y elaboran de manera rápida, reduciendo los problemas de disponibilidad de datos y seguridad. También la velocidad de procesamiento es otra ventaja de estos tipos de archivos. Uno de los problemas de estos archivos es la falta potencial para evolucionar, con frecuencia los archivos se diseñan por una necesidad inmediata. Hace tediosa la consulta de sistema que involucre a varios archivos separados. El rediseño de archivos, normalmente implica rehacer los programas que los acceden.
 - **Base de datos:** almacenamiento de datos formalmente definidos, controlado centralmente para intentar servir a múltiples y diferentes aplicaciones. La esencia de una Base de datos es el DBMS (Data Base Management System), el cual permite la creación, modificación y actualización de la base. Los objetivos son: asegurar que los datos puedan ser compartidos por varias aplicaciones, que el mantenimiento sea preciso y consistente, asegurar que los datos se encuentren siempre disponibles, permitir que la base evolucione y se

adapte a las necesidades, permitir que los usuarios desarrollen su propia visión de los datos, sin preocuparse por la manera que los datos se encuentran almacenados físicamente. La desventaja es que los datos se almacenan en un solo lugar físico, por lo tanto son más vulnerables a accidentes y requieren de mayor respaldo, el tiempo de insertar, actualizar, eliminar y recuperar son considerables y el costo de mantenimiento es una cantidad razonable.

- **Conceptos base**

- **Entidad:** un objeto o evento, acerca del cual, se recolectan datos.- (una persona, un lugar, un objeto).
- **Relaciones:** asociaciones entre entidades.
- **Atributos:** es una característica de una entidad. Puede haber muchos atributos para una entidad.
- **Registros:** colección de datos elementales que tienen algo en común con la entidad descripta.
- **Llaves:** es un dato elemental en un registro que se utiliza como criterio de identificación para este.
- **Metadatos:** son datos acerca de los datos presentes en el archivo o en la base. Los metadatos describen el nombre que se les da y la longitud asignada a cada dato elemental y de cada uno de los registros.

- **Tipos de Archivos**

- **Archivos maestros:** contienen registros para un grupo de entidades. Pueden actualizarse frecuentemente, pero los registros en sí, se mantienen permanentes.
- **Archivos de tablas:** contiene datos que se utilizan para calcular otros datos o más parámetros de desempeño.
- **Archivos de transacción:** se utiliza para introducir cambios para la actualización del archivo maestro.
- **Archivos de trabajo:** contiene datos reordenados de formas distintas o clasificadas en formas distintas.
- **Archivos Spooling (para impresión):** el envío de la salida de un proceso a un archivo en lugar de la impresora por falta de disponibilidad.

- **Tipos de Organización**

- **Secuencial**
- **Listas de enlace:** Las listas de enlace se consultan para utilizar un conjunto de apuntadores que dirigen al siguiente registro lógico localizado en cualquier parte del archivo.-
- **Disperso o atomizado:** La dispersión es el proceso de cálculo de una dirección para un registro llave.
- **Indexada:** es diferente al apuntador debido a que se almacena en un archivo independiente al de datos.
- **Índices secuenciales: (ISAM)** Los registros se ordenan en bloque. Los registros dentro del bloque se almacenan en un orden físico, pero los bloques de registros pueden presentarse en cualquier orden. Utilizan un índice para localizar el bloque de registro.

- **Organización de Bases de Datos:** se basa en una visión lógica y una visión física de los datos, las Bases de datos la comparten numerosos usuarios, y cada usuario ve los datos de manera diferente. Este modelo lógico es el que debe transformarse en modelo físico de la Base. El diseño físico considera la forma de almacenarse los datos y de sus interrelaciones, así como la mecánica del acceso.

- **Tipos básicos de base de datos con una estructuración lógica:**

- **Estructura de datos jerárquica:** Implica que una entidad no puede tener más de una entidad propia. Esto es, una estructura hecha de varias asociaciones 1: M o 1:1. No se permite: M: 1 o M: M.
- **Estructuras de Red:** Una estructura reticular permite que cualquier entidad cuente con cualquier número de subordinados o de superiores.- Las entidades se conectan mediante el uso de enlaces de red, los cuales son datos comunes a ambas entidades conectadas.
- **Estructuras de datos relacionales:** Consiste en una o más tablas bidimensionales, las cuales se refieren como relaciones. Los renglones de las tablas representan los registros y las columnas contienen los atributos.

- **Normalización:** Es un proceso de transformación de las complejas presentaciones de usuarios y de los almacenamientos de datos en conjuntos estables de estructuras de datos de menor tamaño. Además de ser más sencillas, son más estables e inclusive son más fáciles de mantener. Los aspectos de normalización incluye, la eliminación de grupos repetidos y la identificación de la llave que define el criterio primario, asegurar que todos los atributos no-llave, o sin llave, sean completamente dependiente de la llave de criterio primario, eliminar cualquier dependencia transitoria.

- **Etapas del Diseño de Bases de Datos**

- **Etapas del Diseño Conceptual:** en esta etapa se obtiene una estructura de la información de la futura BD independiente de la tecnología que hay que emplear. No se tiene en cuenta todavía qué tipo de base de datos se utilizará –relacional, orientada a objetos, jerárquica, etc.; en consecuencia, tampoco se tiene en cuenta con qué SGBD ni con qué lenguaje concreto se implementará la base de datos.
 - **Etapas del diseño lógico:** en esta etapa se parte del resultado del diseño conceptual, que se transforma de forma que se adapte a la tecnología que se debe emplear. Más concretamente, es preciso que se ajuste al modelo del SGBD con el que se desea implementar la base de datos. Esta etapa parte del hecho de que ya se ha resuelto la problemática de la estructuración de la información en un ámbito conceptual, y permite concentrarnos en las cuestiones tecnológicas relacionadas con el modelo de base de datos.
 - **Etapas del diseño físico:** en esta etapa se transforma la estructura obtenida en la etapa del diseño lógico, con el objetivo de conseguir una mayor eficiencia; además, se completa con aspectos de implementación física que dependerán del SGBD. Con el objetivo de conseguir un buen rendimiento de la base de datos, se deben tener en cuenta las características de los procesos que consultan y actualizan la base de datos, como por ejemplo los caminos de acceso que utilizan y las frecuencias de ejecución. También es necesario considerar los volúmenes que se espera tener de los diferentes datos que se quieren almacenar.
- **Diseño Conceptual:** el diseño conceptual de una base de datos mediante el modelo ER es aplicable al diseño de cualquier tipo de bases de datos –relacional, jerárquica, etc. Su legibilidad se ve favorecida porque proporciona una notación diagramática muy comprensiva. Es una herramienta útil tanto para ayudar al diseñador a reflejar en un modelo conceptual los requisitos del mundo real de interés como para comunicarse con el usuario final sobre el modelo conceptual obtenido y, de este modo, poder verificar si satisface sus requisitos. En el Modelo Conceptual se utiliza el Modelo Entidad Relación que permite generar esquemas conceptuales que se caracterizan por ser, claros, coherentes, plenos, fieles y sencillos.
 - **Modelo Entidad Relación – Esquema Conceptual**
 - **Clave primaria:** Una clave primaria es aquella columna (o conjunto de columnas) que identifica únicamente a una fila. La clave primaria es un identificador que va a ser siempre único para cada fila. Se acostumbra a poner la clave primaria como la primera columna de la tabla pero es más una conveniencia que una obligación. Muchas veces la clave primaria es numérica auto-incrementada, es decir, generada mediante una secuencia numérica incrementada automáticamente cada vez que se inserta una fila.
 - **Clave candidata:** En una tabla puede que tengamos más de una columna que puede ser clave primaria por sí misma. En ese caso se puede escoger una para ser la clave primaria y las demás claves serán claves candidatas.
 - **Interrelación:** se define interrelación como una asociación entre entidades. Una interrelación puede asociar dos o más entidades, el número de entidades que asocia una interrelación es el grado de interrelación.
 - **Interrelaciones Binarias:** Conectividad uno a uno (1:1), Conectividad uno a muchos (1:N) y Conectividad muchos a muchos (M:N)
 - **Interrelaciones n-arias:** cada una de las tres entidades asociadas con una interrelación ternaria puede estar conectada con conectividad “uno” o bien con conectividad “muchos”. En consecuencia, las interrelaciones ternarias pueden tener cuatro tipos de conectividad: M:N:P, M:M:1, N:1:1 y 1:1:1.
 - **Interrelaciones Recursivas:** una interrelación recursiva es una interrelación en la que alguna entidad está asociada más de una vez.
 - **Entidades Débiles:** Una entidad débil es aquella que no puede existir sin participar en la relación; es decir, aquella que no puede ser unívocamente identificada solamente por sus atributos.
 - **Entidades Asociativas:** la entidad que resulta de considerar una interrelación entre entidades como si fuese una entidad es una entidad asociativa, y tendrá el mismo nombre que la interrelación sobre la que se define.
 - **Generalización/especialización:** permite reflejar el hecho de que hay una entidad general, que denominamos superclase que se puede especializar en entidades de subclase.
 - **Diseño Lógico:** Transformación del modelo Entidad-Relación al modelo relacional.
 - **Diseño Lógico Estándar:** a partir del esquema conceptual, y teniendo los requisitos de proceso y de entorno, se elabora un Esquema Lógico Estándar (ELS), que se apoya en un Modelo Lógico Estándar (MLS),

que será el mismo modelo de datos soportado por el SGBD (Sistema de Gestión de Base de Datos) que se vaya a utilizar. En nuestro caso el MLS es el Modelo Relacional.

- **Diseño Lógico Específico:** con el Esquema Lógico Estándar (ELS), y teniendo en cuenta el Modelo Lógico Específico (MLE) propio del SGBD (Sistema de Gestión de Base de Datos), se elabora el Esquema Lógico Específico (ELE), que será descrito en el Lenguaje de Definición de Datos (LDD) del producto comercial que estemos utilizando.
- **Transformaciones desde Esquema Entidad-Relación a Relacional**
 - **Transformación de Entidades:** cada tipo de entidad se convierte en una tabla. La tabla se llamara igual que el tipo de entidad de donde proviene.
 - **Transformación de Atributos de Entidades:** cada atributo de una entidad se transforma en una columna de la tabla a la que ha dado lugar la entidad. Los atributos que son identificadores principales pasan a formar la clave primaria de la tabla. Los atributos que no son identificadores se representan como columnas de la tabla correspondiente.
 - **Transformación de Atributos Multivaluados:** puesto que el Modelo Relacional no permite dominios multivaluados, deberá crearse una nueva tabla cuyos únicos atributos (y clave primaria) serán la concatenación de la clave primaria de la entidad original y el atributo multivaluado. Además, se debe crear una clave ajena referenciando a la tabla primaria.
 - **Transformación de Atributos Derivados:** No existe para los atributos derivados una representación directa y concreta en el Modelo Lógico Estándar, sino que se pueden tratar como atributos normales, que pasarán a ser columnas de la relación que corresponda.
 - **Transformación - Interrelaciones Binarias N:M:** Un tipo de interrelación N:M (muchos a muchos) se transforma en una relación que tendrá como clave primaria la concatenación de los Identificadores principales (IP) de los tipos de entidad que asocia. Cada uno de los atributos que forman la clave primaria de esta relación son clave ajena respecto a cada una de las tablas donde este atributo es clave primaria.
 - **Transformación - Interrelaciones Binarias 1:N:** Propagar los Identificadores Primarios del tipo de entidad que tiene la cardinalidad máxima 1 a la que tiene N (propagación de clave). Esta es la regla habitual.
 - **Transformación - Interrelaciones Binarias 1:1:** una interrelación de tipo 1:1 es un caso particular de una 1:N, por lo que se pueden aplicar dos opciones siguientes, crear una nueva tabla, o establecer propagación de clave, teniendo en cuenta que ahora la propagación de clave puede efectuarse en ambos sentidos.
 - **Transformación - Interrelaciones Binarias grado $n > 2$:** debido a su complejidad, las interrelaciones de grado mayor que 2 se representan igual a las interrelaciones N:M, es decir, creando una nueva tabla cuya clave primaria será la concatenación de las claves primarias de los tipo de entidades participantes.
 - **Transformación - Interrelaciones Atributos:** Si la interrelación se transforma en una tabla, todos sus atributos pasan a ser columnas de la tabla, en caso de la interrelación se transforme mediante propagación de clave, sus atributos migran junto a la clave a la tabla correspondiente.
 - **Transformación en Dependencias de Existencia e Identificación:** La manera de transformar una interrelación de este tipo es utilizar el mecanismo de propagación de claves, creando una clave ajena, con nulos no permitidos, en la relación de la entidad dependiente, con la característica de obligar a una modificación y un borrado en cascada.
 - **Transformación – Generalizaciones (Tipos y Subtipos):**
 - **Opción A)** Englobar todos los atributos de la entidad y sus subtipos en UNA SOLA RELACIÓN cuando los subtipos se diferencien en muy pocos atributos y las interrelaciones que los asocian sean las mismas para todos los subtipos.
 - **Opción B)** Crear una relación para el supertipo y tantas relaciones como subtipos haya, con sus atributos correspondientes. Esta es la solución cuando existen muchos atributos distintos entre los subtipos y se quieren mantener los atributos comunes a todos ellos en una relación.
 - **Opción C)** Considerar relaciones distintas para cada subtipo, que contengan además los atributos comunes. Se elegiría esta opción cuando se dieran las mismas condiciones que en el caso anterior – muchos atributos distintos – y los accesos realizados sobre los datos de los distintos supertipos siempre afectan a los atributos comunes.

Introducción al Diseño Webapps

"La ingeniería web es la aplicación de metodologías sistemáticas, disciplinadas y cuantificables al desarrollo eficiente, operación y evolución de aplicaciones de alta calidad en la World Wide Web"

- **Ingeniería Web:** uno de los aspectos más tenidos en cuenta, en el desarrollo de sitios web es sin duda alguna el diseño gráfico y la organización estructural del contenido. **Una página web** es un documento al que se puede acceder a través de un navegador. La información de las páginas web es, normalmente, estática (sólo se puede leer, no interactuar con ella). **Una aplicación web** es una página web especial, que tiene información sobre la que se puede interactuar e incluso cambiar.
 - **La Ingeniería Web es Multidisciplinar** e incluye los siguientes aspectos: arquitectura de la información, ingeniería de hipermedia/hipertexto, ingeniería de requisitos, diseño de interfaz de usuario, usabilidad, diseño gráfico y de presentación, diseño análisis de sistemas, ingeniería de software, ingeniería de datos, indexado y recuperación de información, testeo, modelado y simulación, despliegue de aplicaciones, operación de sistemas y gestión de proyectos.
 - **Los principales aspectos de la ingeniería de la Web** incluyen, entre otros, los siguientes temas: diseño de procesos de negocio para aplicaciones web, herramientas CASE para aplicaciones web, generación de código para aplicaciones web, desarrollo web colaborativo, modelado conceptual de aplicaciones web, diseño de Modelos de datos para sistemas de información web, ingeniería web empírica, entornos de desarrollo de aplicaciones web integrados, herramientas de autor para contenido multimedia, pruebas de rendimiento de aplicaciones basadas en web, personalización y adaptación de aplicaciones web, herramientas y métodos de prototipado, control de calidad y pruebas de sistemas, ingeniería de requisitos para aplicaciones web, aplicaciones para la Web Semántica, factorías de software para la web, métodos, herramientas y automatización de pruebas para aplicaciones web, aplicaciones web móviles y ubicuas, usabilidad de aplicaciones web, accesibilidad para la web, metodologías de diseño web, formación en ingeniería de la web, diseño de interfaces de usuario, métricas para la web, estimación de costes y medición, gestión de proyectos web y gestión de riesgos, desarrollo y despliegue de servicios web.
 - **categorización:**
 - **Estático:** Se enfoca en la organización de la estructura y el contenido, en la forma como se va a presentar la información y que sea fácil de manejar para cualquier usuario, pero debe tener en cuenta la eficiencia y la confiabilidad.
 - **Sitio estático con formularios de entrada:** Tiene las mismas características que el anterior, adicionándole que permite a los usuarios la interacción por medio de cuestionarios, comentario y sugerencias.
 - **Sitio con acceso de datos dinámicos:** Incluye lo anterior, además cuenta con bases de datos en las cuales el usuario puede realizar consultas y búsquedas.
 - **Sitio creado dinámicamente** en este sitio los requerimientos son parecidos pero deben suplir con las necesidades de cada usuario; creando sitios dinámicos que sean compatibles con el entorno de navegación de cada usuario.
 - **Aplicación de software basada en la Web** este sitio puede tener todas las características antes mencionadas, pero logrando un parecido con una implementación cliente/servidor comúnmente conocido que a un sitio web estático.
 - **Características:** Intensivas de Red, Inmediatez, Evolución Continua, Controlada por el Contenido, Seguridad y Estética.
- **Etapas de la Ingeniería Web**
 - **Formulación:** Clientes y Equipo de Desarrollo establecen, las necesidades del negocio, los requisitos, las funciones, características, las metas y objetivos. Permite establecer el alcance de la primera entrega.
 - **Planificación:** Se establece la estimación de costos, evaluación de riesgos, el calendario de desarrollo y fechas de entregas.
 - **Análisis:** se especifican los requerimientos y se identifican los contenidos.

- **Modelización:** Se compone de dos secuencias paralelas de tareas. Una consiste en el Diseño y Producción de Contenidos que forma parte de la aplicación. La otra, en el Diseño de la Arquitectura, Navegación e Interfaz de usuario.
- **Generación de Páginas:** Se integra, contenido, arquitectura, navegación e interfaz para crear estática o dinámicamente el aspecto más visible de la aplicación, las páginas.
- **Test:** el test busca errores a todos los niveles, contenido, funcional, navegacional, rendimiento, etc.
- **Evaluación del Cliente:** el resultado final es sometido a la evaluación del cliente y aceptación por el mismo.

➤ Modelo de Proceso de la Ingeniería de la Usabilidad

- **Análisis de Requisitos:** se inicia estableciendo el perfil de los usuarios del sistema y llevando acabo el análisis conceptual de las tareas, la definición de las restricciones y necesidades de la plataforma de uso y de los principios de diseño a aplicar. Todo ello da lugar a una serie de objetivos de usabilidad que el sistema debe cumplir y que, junto a las guías de estilo, conforman el conjunto estable de requisitos a considerar en el proceso de desarrollo.
- **Diseño, Evaluación y Desarrollo:** se definen tres niveles de desarrollo, cada uno de los cuales hace uso de maquetas y prototipos como artefactos destinados a ser evaluados. En el primer nivel, se realiza el diseño conceptual, en el segundo nivel se centran en el diseño de las ventanas y en la construcción de los prototipos que van a permitir comprobar que se han cubierto todos los aspectos del nivel anterior, y por último, la interfaz se va refinando de manera iterativa hasta conseguir un sistema que incorpore todas las funcionalidades requeridas.
- **Fase de Instalación:** la fase de instalación del mismo en el entorno de exploración en el que el usuario real interactuara con el producto. El objetivo principal de esta fase es detectar y corregir posibles problemas.

➤ Ingeniería de Requisitos en Aplicaciones para la Web

- **Captura de Requisitos:** Entrevistas, JAD, Brainstorming, Concept Mapping, Sketches y Storyboards, Casos de Uso, Cuestionarios y Checklists, Comparación de terminología.
- **Definición de Requisitos:** lenguaje natural, glosario y ontologías, plantillas o patrones, escenarios, casos de uso, lenguajes formales.
- **Validación de Requisitos:** Reviews (evaluación) o Walk-throughs (tutoriales), auditorías, matrices de trazabilidad, prototipos.
- **Clasificación de Requisitos relevantes en sistemas web:** requisitos de datos (contenido, información), requisitos de interfaz (interacción con el usuario), requisitos navegacionales (navegación del usuario), requisitos de personalización (interacción y adaptación del usuario), requisitos transaccionales o funcionales internos (lo que hace el sistema internamente, requisitos de servicios) y requisitos no funcionales (portabilidad, usabilidad, reutilización, etc.)

➤ Técnicas y Modelos para el Desarrollo de Aplicaciones Web

- **WSDM: Web Site Design Method:** De Troyer & Leune, 1997) Es una propuesta para el desarrollo de sitios web, en la que el sistema se define en base a los grupos de usuarios. Su proceso de desarrollo se divide en cuatro fases: modelo de Usuario, diseño conceptual, diseño de la implementación, implementación. La fase que más repercusión tiene para este trabajo es la primera en la que intenta detectar los perfiles de usuarios para los cuales se construye la aplicación. Para ello, se deben realizar dos tareas: clasificación de usuarios y descripción de los grupos de usuarios. El resto de las fases se basan en la clasificación de usuarios de la primera fase.
- **SOHDM: Scenario-based Object-Oriented Hypermedia Design Methodology:** (Lee, Lee & Yoo, 1998) Presenta la necesidad de disponer de un proceso que permita capturar las necesidades del sistema. Para ello, propone el uso de escenarios. El proceso de definición de requisitos parte de la realización de un diagrama de contexto tal y como se propone en los diagramas de flujos de datos (DFD) de Yourdon (1989). En este diagrama de contexto se identifican las entidades externas que se comunican con el sistema, así como los eventos que provocan esa comunicación. Se propone por cada evento diferente se desarrollar un escenario.

- **RNA: Relationship-Navegational Analysis:** RNA (Bieber, Galnares & Lu, 1998) plantea una secuencia de pasos para el desarrollo de aplicaciones web, centrándose fundamentalmente en el flujo de trabajo de análisis. Fase 1- Análisis del entorno (características de la audiencia), Fase 2- Elementos de interés (documentos, pantallas, la información, etc.), Fase 3- Análisis del conocimiento (desarrollar un esquema que represente a la aplicación), Fase 4- Análisis de la navegación (posibilidades de navegación dentro de la aplicación), Fase 5- Implementación del análisis (esquema final en el que ya se encuentran incluidos los aspectos de navegación, se pasa el esquema a un lenguaje entendible por la máquina).
- **HFBPM: Hypermedia Flexible Process Modeling** La propuesta de HFBPM (Olsina, 1998) describe un proceso detallado que cubre todo el ciclo de vida de un proyecto software. Propone un total de trece fases para las cuales se especifican a su vez una serie de tareas. Para este estudio es principalmente relevante la primera fase, denominada de modelado de requisitos, cuyas tareas son las siguientes: descripción breve del problema. (lenguaje natural), descripción de los requisitos funcionales mediante casos de uso, realizar un modelo de datos para esos casos de uso (Modelo de Clases), modelar la interfaz de usuario. (uso de sketches y prototipos), modelar los requisitos no funcionales. En éstos incluyen la navegación, la seguridad, etc.
- **OOHDM: Object Oriented Hypermedia Design Model:** es una propuesta metodológica ampliamente aceptada para el desarrollo de aplicaciones de la web (Schwabe & Rossi, 1998). En sus comienzos no contemplaba la fase de captura y definición de requisitos, pero actualmente propone el uso de User Interaction Diagrams (UIDs) definidos por Vilain, Schwabe & Sieckenius (2000). Esta propuesta parte de los casos de uso, igualmente, resalta la necesidad de empezar el diseño del sistema, especialmente en los entornos web, teniendo un claro y amplio conocimiento de las necesidades de interacción, o lo que es lo mismo de la forma en la que el usuario va a comunicarse con el sistema. Partiendo de estas dos premisas, OOHDM propone que la comunicación con el usuario se realice utilizando los casos de uso y a partir de ellos los analistas elaboran los UIDs. OOHDM centra el desarrollo de un sistema de información web entorno al modelo conceptual de clases.
- **UWE: UML-Based Web Engineering:** Es una propuesta metodológica basada en el Proceso Unificado (Jacobson, Booch & Rumbaugh, 1999) y UML para el desarrollo de aplicaciones web (Hennicker & Koch, 2000, Koch, 2001), UWE cubre todo el ciclo de vida de este tipo de aplicaciones, centrando además su atención en aplicaciones personalizadas (adaptivas), concretamente para la captura de requisitos, esta metodología distingue entre la tarea de educir requisitos, definir y validar los requisitos. El resultado final de la captura de requisitos en UWE es un modelo de casos de uso acompañado de documentación que describe los usuarios del sistema, las reglas de adaptación, los casos de uso y la interfaz. UWE clasifica los requisitos en dos grandes grupos: funcionales y no funcionales.
- **W2000:** (Baresi, Garzotto & Paolini, 2001) Supone una propuesta que amplía la notación de UML con conceptos para modelar elementos de multimedia heredados de la propuesta HDM (Hypermedia Design Model) (Garzotto, Schwabe & Paolini, 1993). El proceso de desarrollo deW2000 se divide en tres etapas: Análisis de requisitos, Diseño de hipermedia y Diseño funcional. La especificación de requisitos enW2000 se divide en dos sub-actividades: Análisis de requisitos funcionales (modelo de casos de uso UML) y Análisis de requisitos navegacionales (modelo de caso de uso no funcional de una extensión de UML).
- **UWA: Ubiquitous Web Applications:** ha nacido de la colaboración entre diferentes grupos de trabajo, por lo que resulta realmente una agrupación de propuestas y técnicas. En concreto, la propuesta de W2000 se encuentra incluida en UWA. Sin embargo, W2000 ha sido incluida en UWA sólo en la fase de diseño hipermedia, siendo ambas propuestas diferentes en la fase de definición de requisitos. En el proceso de captura de requisitos en UWA (Uwa Requirements Elicitation, 2001) se definen los diferentes roles de usuario que pueden interactuar con el sistema, los objetivos globales de éste y la relación entre ellos. Los requisitos son clasificados en varios tipos: de contenido, de estructura de contenido, de acceso, de navegación, de presentación, de operaciones de usuario y de operaciones del sistema. De esta forma, los requisitos se van refinando hasta que solo pertenezcan a uno de estos grupos. Para definir los objetivos, UWA propone una notación propia, basada en una plantilla. La definición de los actores y la relación con los objetivos se hace usando un diagrama basado en casos de uso. Una vez que los requisitos son detectados, hacen uso de XML para definirlos de una manera formal.
- **NDT - Navigational Development Techniques:** (Escalona, Torres & Mejías, 2002) Es una técnica para especificar, analizar y diseñar el aspecto de la navegación en aplicaciones web. Para este trabajo, solo es relevante la propuesta que ofrece para la definición y captura de requisitos. El flujo de especificación de requisitos de NDT comienza con la fase de captura de requisitos y estudio del entorno. Para ello, plantea el

uso de técnicas como las entrevistas o el Brainstorming y JAD. Tras esta fase, se propone la definición de los objetivos del sistema. En base a estos objetivos, el proceso continúa definiendo los requisitos que el sistema debe cumplir para cubrir los objetivos marcados. NDT clasifica los requisitos en:

- **Requisitos de almacenamiento de información**
 - **Requisitos de actores**
 - **Requisitos funcionales**
 - **Requisitos de interacción**, representados mediante:
 - **Frases**, que recogen cómo se va a recuperar la información del sistema utilizando un lenguaje especial denominado BNL (Bounded Natural Language) (Brisaboa, Penabad, Places & Rodríguez, 2001).
 - **Prototipos de visualización**, que representan la navegación del sistema, la visualización de los datos y la interacción con el usuario.
 - **Requisitos no funcionales**
 - **Design-driven Requirements Elicitation**: Es parte del proceso design-driven que proponen Lowe y Eklund (2002) para el desarrollo de aplicaciones en el entorno Web. La propuesta consiste en realizar la captura, definición y validación de requisitos durante el proceso de diseño. El proceso se basa en el uso de prototipos para ayudar al cliente en la exploración de las posibles soluciones y de los problemas que tienen que ser resueltos. Los usuarios o clientes definen sus requerimientos basándose en la observación o trabajo con estos prototipos. Es un proceso iterativo que consiste en reducir la incertidumbre del cliente. El ciclo tiene tres fases: evaluación, especificación y construcción.
- **NDT (Navigational Development Techniques)**: NDT es una propuesta metodológica compuesta por un proceso en el que se plantean técnicas para capturar, describir y validar los requisitos de un sistema Web, y partiendo de esos requisitos, generar de manera sistemática los modelos de análisis de la aplicación Web. NDT no contempla las fases avanzadas del ciclo de vida de desarrollo de una aplicación Web, como podrían ser implementación o mantenimiento. El ciclo de vida de NDT está compuesto por dos fases: la ingeniería de requisitos y el análisis. Aunque, en principio, ambas son secuenciales, el proceso de NDT no lo es, puesto que en muchos momentos se puede realizar la vuelta atrás para corregir errores o incongruencias.
- **Ingeniería de Requisitos**: en esta fase se definen cuáles son los objetivos de la aplicación Web a desarrollar y en base a ellos se capturan y definen los diferentes requisitos de esta. Los requisitos son agrupados en:
 - **Requisitos de almacenamiento de información**: constituye uno de los elementos relevantes para la navegación. Estos representan las necesidades de almacenamiento de información que tiene el sistema y define las características de la información que se va a manejar y a mostrar.
 - **Requisitos de actores**: son los roles que podrán interactuar con la aplicación Web. Dichos roles surgen por las necesidades de trabajo de los diferentes roles de usuario con la aplicación. La definición del sistema de navegación debe basarse en los diferentes roles de usuario que pueden interactuar con el sistema para que se adecue a las necesidades establecidas por cada uno de ellos. Es posible diferenciar dos tipos de actores: actores básicos; que corresponden a un rol individualizado de interacción con la aplicación Web, y actores derivados; que se pueden definir a partir de otros actores, como conjunción de los roles correspondientes a los actores componentes.
 - **Requisitos funcionales**: representa las posibilidades funcionales que debe ofrecer la aplicación Web durante la navegación. Estas posibilidades funcionales van a depender directamente del actor que en cada momento interactúe con el sistema.
 - **Requisitos de interacción**: definen la estructura de navegación a alto nivel del sistema, así como los criterios de recuperación que se van a ofrecer a los diferentes actores. Estos requisitos recogen la manera en la que los actores van a interactuar con la aplicación durante la navegación.

Una vez capturados y definidos los requisitos se pasa a la validación de los mismos. Si durante la validación se detectan errores, se vuelve a la captura y definición hasta llegar al resultado final adecuado. Este resultado final queda plasmado en el documento de requisitos del sistema.

- **Análisis**: durante esta fase se generan varios modelos. El primero de ellos es el modelo conceptual, el cual describe la información que maneja la aplicación Web. El modelo conceptual representa y describe el tipo de información que va a ser tratada, presentada o modificada durante el proceso de navegación. De esta forma,

la navegación de un sistema se define en base a la información que maneja. El segundo modelo que se genera durante el análisis es el modelo de navegación, que representa la estructura de navegación de la aplicación Web, es decir, cómo se va a poder navegar a través de la información.

La aplicación de NDT genera tres resultados finales:

1. **El documento de requisitos de la aplicación Web**, donde se detallan los objetivos y requisitos que debe cumplir la aplicación Web.
 2. **El documento de análisis de la aplicación Web**, donde se recogen el modelo conceptual y el modelo de navegación del sistema.
 3. **Los prototipos del sistema**, que muestran la estructura de la interfaz de la aplicación Web.
- **RNA (Relationship Navigational Analysis):** La propuesta de RNA es quizás, junto a NDT, dos de las que más ha resaltado la necesidad de trabajar con la especificación de requisitos, dejando atrás las fases posteriores.
 - **Fase de Análisis del Entorno:** El propósito de esta fase es el de estudiar las características de la audiencia. Se determina y clasifica a los usuarios finales de la aplicación en grupos según sus perfiles.
 - **Fase de Definición de Elementos:** En esta fase se listan todos los elementos de interés de la aplicación. Por elementos de interés se entienden los documentos, las pantallas que se van a requerir, la información, etc.
 - **Fase de Análisis del Conocimiento:** se desarrolla un esquema que represente a la aplicación. Para ello RNA propone identificar los objetos, los procesos y las operaciones que se van a poder realizar en la aplicación, así como las relaciones que se producen entre estos elementos.
 - **Fase de Análisis de Navegación:** se verifica que el esquema obtenido en la fase anterior sea enriquecido con las posibilidades de navegación dentro de la aplicación.
 - **Fase de Implementación del Análisis:** cuando una vez obtenido el esquema final en el que ya se encuentran incluidos los aspectos de navegación, se pasa el esquema a un lenguaje entendible por la máquina.